

UNIVERSITÀ DEGLI STUDI DI VERONA

Corso di Laurea in Bioinformatica

Algoritmi per Bioinformatica

e Laboratorio di Programmazione II

I seguenti appunti sono preparati tramite i lucidi del professore e il libro di riferimento

Per la parte di laboratorio il professore prende gli esercizi da:

<https://leetcode.com/problemset/algorithms/>

Anno Accademico 2025/2026

Indice

1	Fondamenti	6
1.1	Introduzione al corso	6
1.1.1	Requisiti	6
1.1.2	Programma del corso	6
1.1.3	Struttura del modulo	7
1.1.4	Modalità di esame	7
1.2	Il ruolo degli algoritmi nell'informatica	7
1.2.1	Cos'è un algoritmo?	7
1.3	Esempio di algoritmo ricorsivo	8
1.4	Esempi di problemi computazionali da risolvere in aula	13
2	InsertionSort e MergeSort	21
2.0.1	InsertionSort	21
2.1	MergeSort	29
2.1.1	Metodo Divide et Impera	29
2.1.2	Descrizione dell'algoritmo	29
2.1.3	Pseudocodice	29
2.1.4	Correttezza: Invariante di ciclo di Merge	31
2.1.5	Complessità	31
2.1.6	Confronto con InsertionSort	32
3	Notazione Asintotica	34
3.1	Introduzione	34
3.2	Notazione O — Limite superiore	34
3.3	Notazione Ω — Limite inferiore	35
3.4	Notazione Θ — Limite stretto	36
3.5	Uso nelle equazioni e disequazioni	37
3.6	Proprietà di confronto tra funzioni	37
3.7	Limiti asintotici delle funzioni elementari	37
3.8	Equazioni e disequazioni asintotiche	38
3.9	Proprietà delle notazioni asintotiche	39
3.10	Limiti asintotici delle funzioni elementari	39
4	Divide et Impera	41
4.1	Idea Generale	41
4.2	Schema Generale	41
4.3	Costo Computazionale	42
4.4	Esempi di Applicazione	42
4.4.1	Ricorsione o iterazione?	42
4.4.2	Problema Prodotto — Algoritmo di Karatsuba	43

4.4.3	Tabella di esempi divide et impera	44
4.4.4	Prodotto di Interi Grandi	44
4.4.5	MergeSort	45
4.4.6	Massimo SottoArray	46
4.5	Equazioni di Ricorrenza	47
4.5.1	Metodo di Sostituzione	47
4.5.2	Metodo dell'Albero di Ricorsione	47
4.5.3	Metodo dell'Esperto	48
4.6	Riepilogo: Problemi risolvibili con Divide et Impera	49
5	Analisi Probabilistica e Algoritmi Randomizzati	49
5.1	Preliminari: variabili casuali indicatrici	49
5.2	Analisi probabilistica: il Problema delle Assunzioni	50
5.3	Algoritmi randomizzati	51
5.4	Permutazione casuale: <code>RandomizeInPlace</code>	51
5.5	QuickSort	52
5.6	Idea Generale	52
5.7	$\text{Partition}(A, p, r)$	53
5.7.1	Invariante di ciclo di Partition	54
5.8	Analisi di Complessità di QuickSort	55
5.8.1	Caso peggiore	55
5.8.2	Caso migliore	56
5.8.3	Verso il caso medio	56
5.9	QuickSort Randomizzato	57
5.10	Analisi del Caso Medio di QuickSort	57
6	Statistiche d'Ordine e Selezione	59
6.1	Definizioni	59
6.2	Problema della Selezione	60
6.3	Minimo, Massimo e MinMax	60
6.4	Idea Divide et Impera per la Selezione	61
6.5	RandSelect	62
6.6	Analisi della Complessità di RandSelect	62
6.6.1	Caso peggiore	63
6.6.2	Caso medio	63
7	HeapSort	64
7.1	Idea Generale	64
7.2	Struttura Dati Heap	64
7.2.1	Navigazione dell'albero tramite indici	65
7.2.2	Tipi di heap e proprietà	65
7.2.3	Altezza dello heap	65

7.2.4	Operazioni fondamentali sullo heap	66
7.3	MaxHeapify	66
7.3.1	Analisi di complessità di MaxHeapify	67
7.4	BuildMaxHeap	67
7.4.1	Correttezza di BuildMaxHeap	68
7.4.2	Complessità di BuildMaxHeap	68
7.5	Algoritmo HeapSort	69
7.5.1	Correttezza di HeapSort	69
7.5.2	Complessità di HeapSort	69
7.6	Code di Priorità	70
7.6.1	Operazioni fondamentali	70
7.6.2	Implementazione delle operazioni	70
8	Ordinamento in Tempo Lineare	72
8.1	Limiti Inferiori per l'Ordinamento	72
8.1.1	Alberi di Decisione	72
8.1.2	Limite Inferiore per il Caso Peggior	73
8.2	Counting Sort	73
8.2.1	Idea Generale	73
8.2.2	Algoritmo	74
8.2.3	Esempio Visivo	74
8.2.4	Complessità	75
8.2.5	Stabilità	76
8.3	Bucket Sort	76
8.3.1	Idea Generale	76
8.3.2	Algoritmo	76
8.3.3	Esempio Visivo	77
8.3.4	Correttezza	77
8.3.5	Complessità: Analisi del Caso Medio	78
8.4	Confronto tra gli Algoritmi	79
9	Hashing e Tabelle Hash	79
9.1	Tavole a indirizzamento diretto	79
9.2	Tabelle hash	80
9.3	Funzioni hash: scelta e qualità	80
9.3.1	Conversione verso gli interi	80
9.4	Hashing statico	80
9.4.1	Metodo della divisione	81
9.4.2	Metodo della moltiplicazione	81
9.4.3	Metodo moltiplica-e-scorri	81
9.5	Hashing randomizzato	82

9.6	Gestione delle collisioni	83
9.6.1	Risoluzione mediante concatenamento	83
9.6.2	Risoluzione mediante indirizzamento aperto	84
9.6.3	Ispezione lineare	86
9.6.4	Doppio hashing	86
10	Alberi Binari di Ricerca	87
10.1	Definizione di ABR	87
10.2	Attraversamento	87
10.2.1	Attraversamento Simmetrico (in-order)	88
10.2.2	Pre-ordine e Post-ordine	88
10.3	Interrogazione di un ABR	89
10.3.1	Search	89
10.3.2	Minimum e Maximum	89
10.3.3	Successor e Predecessor	90
10.4	Inserimento e Cancellazione	91
10.4.1	Inserimento	91
10.4.2	Cancellazione	91
10.5	Costruzione di un ABR	93
10.6	Cenno agli ABR Rosso-Neri	94
10.7	Cenno ai Radix Tree	94
10.8	Esercizi	95
11	Programmazione Dinamica	95
11.1	Idea Generale	95
11.1.1	Confronto con Divide et Impera	96
11.1.2	Quando si Usa	96
11.2	Esempi di Applicazione	96
11.2.1	Massima Sottosequenza Comune (LCS)	97
11.2.2	Massima Sottosequenza Crescente (MSC)	100
11.2.3	String Matching Approssimato	102
11.2.4	Problema dello Zaino 0-1	104
11.3	Casi Comuni di Sottoproblemi	106
11.4	Memoization	106
11.5	Esercizi	107
12	Può tornare utile	107
12.1	Appendice A (Sommatorie)	107
12.1.1	Formule fondamentali	108
12.1.2	Serie geometrica	108
12.1.3	Serie armonica	108
12.1.4	Tecniche di manipolazione	109

12.1.5	Approssimazione con integrali	109
12.2	Appendice B (Insiemi, relazioni, funzioni, grafi, alberi)	110
12.2.1	Insiemi	110
12.2.2	Relazioni	110
12.2.3	Funzioni	111
12.2.4	Grafi	111
12.2.5	Alberi	112
12.3	Appendice C (Calcolo combinatorio e probabilità)	113
12.3.1	Principi di conteggio	113
12.3.2	Permutazioni e combinazioni	113
12.3.3	Probabilità	114
12.3.4	Probabilità condizionata e indipendenza	114
12.3.5	Variabili casuali e valore atteso	115
12.4	Appendice D (Matrici)	115
12.4.1	Definizioni base	115
12.4.2	Operazioni fondamentali	116
12.4.3	Proprietà del prodotto	116
12.4.4	Determinante e matrice inversa	117
12.4.5	Applicazioni agli algoritmi	117
12.5	Domande d'esame (Appello 2)	118

1. Fondamenti

Questa sezione introduce i concetti fondamentali necessari per comprendere il resto del corso. Non preoccuparti se all'inizio alcune definizioni sembrano astratte: diventeranno chiare con gli esempi concreti nelle sezioni successive.

1.1 Introduzione al corso

1.1.1 Requisiti

- Fondamenti di matematica discreta e grafi
- Calcolo delle probabilità
- Basi di C $\rightarrow$ si studia Java (Classi e Oggetti)¹
- Le procedure ricorsive sono come l'Ave Maria (STUDIA: c'è nell'esame)

Nota

La ricorsione è **fondamentale** in questo corso. Se non ti senti sicuro, ripassa prima di procedere: fibonacci, fattoriale, e i concetti base di chiamate ricorsive.

1.1.2 Programma del corso

- Divide et Impera
- Programmazione dinamica
- Algoritmi Greedy
- Grafi $\rightarrow$ Nodi e archi

Questi quattro argomenti sono le **tecniche algoritmiche** principali. Ogni tecnica è un "modo di pensare" per risolvere problemi complessi:

- **Divide et Impera:** spezza il problema in pezzi piccoli
- **Programmazione Dinamica:** memorizza soluzioni parziali per evitare ricalcoli
- **Greedy:** fai sempre la scelta localmente migliore
- **Grafi:** rappresenta problemi come reti di connessioni

¹Java è il male

1.1.3 Struttura del modulo

48 ore di lezione: 4 ore a settimana + 9 ore di studio personale.² Non usare ChatGPT o altri strumenti simili.

I lucidi non bastano (purtroppo).

Se proprio devi, usa **Anthropic CLAUDE**.

1.1.4 Modalità di esame

- **Prima parte:** 15 domande a risposta multipla (almeno 4 su codice Java – CODE RUNNER)
- **Seconda parte:** Progettare una soluzione algoritmica
- **Se va male:** Può fare orale se ha dubbi (Ave o Maria...)

1.2 Il ruolo degli algoritmi nell'informatica

Prima di tuffarci nei dettagli tecnici, capiamo **perché** gli algoritmi sono importanti. Un algoritmo è semplicemente una "ricetta" per risolvere un problema al computer.

1.2.1 Cos'è un algoritmo?

Definizione 1.1 (Algoritmo). Procedura di calcolo ben definita: dato un input (anche nullo), produce un output in tempo finito.

Nota

In parole semplici: un algoritmo è una sequenza di passi precisi che trasforma dei dati di partenza (input) in un risultato (output). Deve sempre finire in tempo ragionevole.

Esempio quotidiano: una ricetta di cucina è un algoritmo:

- **Input:** ingredienti (farina, uova, zucchero)
- **Passi:** mescola, cuoci a 180°C per 30 minuti
- **Output:** torta

Definizione 1.2 (Problema computazionale). Relazione input/output desiderata. L'algoritmo specifica *come* ottenerla.

Differenza chiave:

- **Problema:** “Voglio ordinare una lista di numeri” (COSA)
- **Algoritmo:** “Confronta elementi a coppie e scambiali se necessario” (COME)

Il problema dice *cosa* vogliamo ottenere, l'algoritmo dice *come* farlo.

²Si, lo so, è una noia, ma serve.

Definizione 1.3 (Algoritmo corretto). Termina producendo il risultato corretto per ogni istanza valida.

Osservazione 1. Non sempre esiste un algoritmo: il problema della fermata è indecidibile.

Nota

Cosa significa “indecidibile”? Esistono problemi per cui è matematicamente impossibile scrivere un algoritmo generale che funzioni sempre. Il famoso “problema della fermata” chiede: “Dato un programma qualsiasi, terminerà mai?” È stato dimostrato che non esiste un algoritmo che possa rispondere a questa domanda per tutti i programmi possibili.

1.3 Esempio di algoritmo ricorsivo

Ora vediamo un esempio concreto: calcolare i numeri di Fibonacci. Questo ci permetterà di capire la differenza tra un algoritmo **corretto** (che funziona) e uno **efficiente** (che funziona velocemente).

Nota

Cosa sono i numeri di Fibonacci?

È una sequenza di numeri dove ogni numero è la somma dei due precedenti:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$$

Definizione formale:

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \text{ per } n \geq 2$$

Esempio: $F_5 = F_4 + F_3 = 3 + 2 = 5$

Nota

Le convenzioni usate per il pseudocodice sono a pag. 19 del [CLRS].

Algorithm 1: FibonacciR

Input: $n \geq 0$

Output: F_n

```
1 if  $n = 0$  then
2   return 0;
3 if  $n = 1$  then
4   return 1;
5 return FibonacciR( $n - 1$ ) + FibonacciR( $n - 2$ );
```

Come funziona questo algoritmo?

Traduce direttamente la definizione matematica in codice:

1. Se $n = 0$ restituisci 0 (caso base)
2. Se $n = 1$ restituisci 1 (caso base)
3. Altrimenti, calcola F_{n-1} e F_{n-2} ricorsivamente e sommal

Esempio per $n = 4$:

- Calcola $F_3 + F_2$
- $F_3 = F_2 + F_1$, quindi serve calcolare F_2 due volte!
- $F_2 = F_1 + F_0 = 1 + 0 = 1$
- Risultato finale: $F_4 = 3$

L'algoritmo è corretto? **Sì!**

Per ogni input valido, termina? **Sì, assumendo che input sia sempre un intero $n \geq 0$**

Se sì, calcola il valore giusto? **Sì, perché l'algoritmo codifica la definizione ricorsiva.**

Quanto **tempo** richiede per terminare?

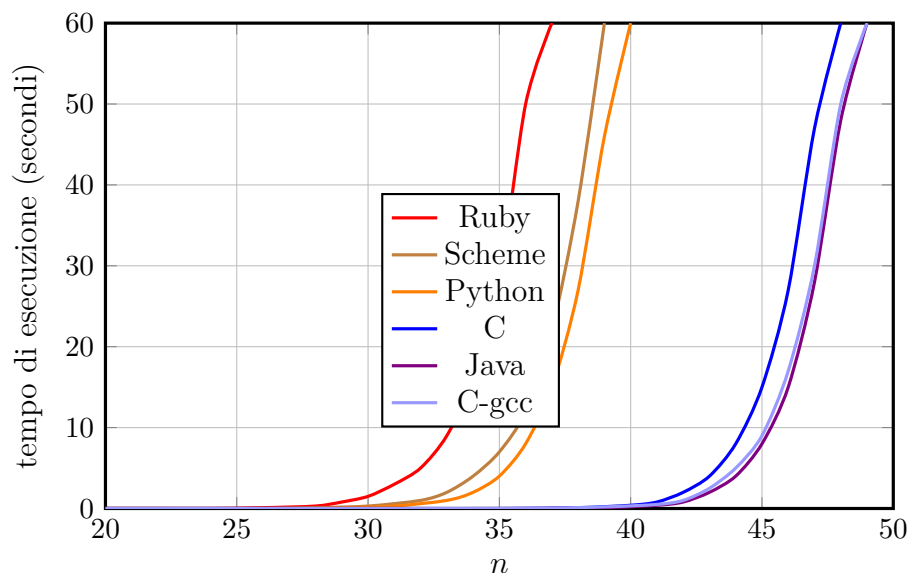


Figura 1. Confronto dei tempi di esecuzione dell'algoritmo ricorsivo `FibonacciR` in diversi linguaggi. I linguaggi interpretati (Ruby, Scheme, Python) saturano a valori di n molto più bassi rispetto ai linguaggi compilati (C, Java, C-gcc).

Problema grave: anche se l'algoritmo è corretto, è **terribilmente lento**.

Per $n = 50$ impiega diversi minuti, mentre un algoritmo migliore lo calcola istantaneamente.

Il problema è che ricalcola gli stessi valori moltissime volte.

Calcolo della complessità computazionale

La **complessità** misura quanto “costa” eseguire un algoritmo. Non misuriamo i secondi (dipendono dal computer), ma il numero di operazioni in funzione della dimensione dell’input.

Definizione 1.4 (Efficienza). Risorse (tempo e spazio) richieste in funzione della dimensione n dell’input.

Definizione 1.5 (Algoritmo vs. Implementazione). L’algoritmo è un concetto **astratto** ($\neq$ implementazione): descrive *come* risolvere un problema indipendentemente dal linguaggio. L’implementazione è un **programma concreto**, ovvero la traduzione dell’algoritmo in un linguaggio comprensibile dalla macchina.

Nota

Analogia:

- **Algoritmo** = ricetta scritta in italiano (“mescola gli ingredienti”)
- **Implementazione** = ricetta tradotta in giapponese per un cuoco giapponese

L’idea (algoritmo) è la stessa, cambia solo la forma (implementazione).

Nozione 1.1 (Algoritmo efficiente). Andiamo a cercare l’algoritmo efficiente:

- **Efficiente:** usa poche risorse
- **Risorse:** tempo e spazio richiesti dall’algoritmo per produrre l’output

Teorema 1.1 (Algoritmo vs. hardware). Con $n = 10^7$ interi da ordinare:

$$t_A = \frac{2(10^7)^2}{10^{10}} \approx 5.5 \text{ ore} \quad (\text{InsSort}, 10^9 \text{ istr/s})$$

$$t_B = \frac{50 \cdot 10^7 \log_2 10^7}{10^6} \approx 20 \text{ min} \quad (\text{MergeSort}, 10^6 \text{ istr/s})$$

L’algoritmo migliore batte l’hardware più veloce.

Lezione fondamentale: un algoritmo efficiente su un computer lento batte un algoritmo inefficiente su un computer veloce.

Nell’esempio:

- Computer A: 10× più veloce, ma usa InsertionSort (algoritmo lento)
- Computer B: 10× più lento, ma usa MergeSort (algoritmo veloce)

Risultato: B finisce in 20 minuti, A impiega 5.5 ore!

Possiamo fare meglio?

Ottimizzazione: FibonacciI**Algorithm 2:** FibonacciI

Input: Intero positivo n
Output: Il valore F_n

```

1 if  $n = 0$  then
2   return 0;
3 if  $n = 1$  then
4   return 1;
5  $pprev = 0$ ;
6  $prev = 1$ ;
7 for  $i = 2$  to  $n$  do
8    $f = prev + pprev$ ;
9    $pprev = prev$ ;
10   $prev = f$ ;
11 return  $f$ ;
```

Come funziona FibonacciI (iterativo)?

Invece di chiamare ricorsivamente la funzione, mantiene in memoria solo gli ultimi due valori e li aggiorna ad ogni passo:

- Inizia con $F_0 = 0$ e $F_1 = 1$
- Calcola $F_2 = F_1 + F_0 = 1$
- Calcola $F_3 = F_2 + F_1 = 2$
- Calcola $F_4 = F_3 + F_2 = 3$
- E così via fino a F_n

Vantaggio: ogni numero viene calcolato **una sola volta**, non milioni di volte come nella versione ricorsiva!

La complessità è:

$$T(0) = 2, \quad T(1) = 3, \quad T(n) = 4 + 2n + 4(n-1) + 1 = 6n + 1$$

La crescita è **lineare in n** .

Algoritmo	Complessità	$n = 50$
Ricorsivo	$(1.4)^n$	minuti
Iterativo	$\Theta(n)$	istantaneo

Confronto drammatico:

- FibonacciR (ricorsivo): per $n = 50$ fa circa $2^{25} \approx 33$ milioni di chiamate $\rightarrow$ minuti
- FibonacciI (iterativo): per $n = 50$ fa esattamente 50 iterazioni $\rightarrow$ istantaneo

Questo dimostra che due algoritmi che risolvono lo stesso problema possono avere prestazioni **radicalmente diverse**.

Modello RAM: ogni operazione elementare ha costo 1; parola di memoria a dimensione fissa.

Nota

Il **modello RAM** (Random Access Machine) è una semplificazione che assumiamo per analizzare gli algoritmi:

- Ogni operazione base (somma, confronto, assegnazione) costa 1
- La memoria è infinita e accedervi costa 1
- Un solo processore esegue un'istruzione alla volta

Non è realistico (nella realtà ci sono cache, processori multipli, ecc.), ma è utile per confrontare algoritmi in modo uniforme.

1.4 Esempi di problemi computazionali da risolvere in aula

Ora vediamo alcuni problemi classici che ti aiuteranno a capire come si progettano e analizzano gli algoritmi. Studia bene questi esempi: sono fondamentali per l'esame!

Indici dei due numeri che sommati danno N

Problema: Data una matrice di interi `nums` e un intero `target`, restituisci gli indici dei due numeri tali che la loro somma sia uguale a `target`. Puoi assumere che ogni input abbia esattamente una soluzione, e non puoi usare lo stesso elemento due volte. Puoi restituire la risposta in qualsiasi ordine.

Nota

In parole semplici: hai un array di numeri e un numero obiettivo. Devi trovare due numeri nell'array che sommati danno il numero obiettivo, e restituire le loro posizioni.

Esempio:

- Input: `nums = [2, 7, 11, 15]`, `target = 9`
- Output: `[0, 1]`
- Spiegazione: `nums[0] + nums[1] = 2 + 7 = 9`

Algorithm 3: twoSum (Brute Force)

Input: `nums[0...n-1]`, `target`
Output: $\{i, j\}$ tali che $\text{nums}[i] + \text{nums}[j] = \text{target}$, $i \neq j$

```

1 for  $i = 0$  to  $n - 2$  do
2   for  $j = i + 1$  to  $n - 1$  do
3     if  $\text{nums}[i] + \text{nums}[j] == \text{target}$  then
4       return  $\{i, j\}$ ;

```

Come funziona l'algoritmo?

Prova tutte le possibili coppie di numeri:

1. Fisso il primo numero (indice i)
2. Controllo tutti i numeri successivi (indice $j > i$)
3. Se $\text{nums}[i] + \text{nums}[j] = \text{target}$, ho trovato la coppia!

Si chiama “brute force” (forza bruta) perché prova semplicemente tutte le combinazioni possibili.

Invariante: Dopo ogni iterazione esterna, tutti gli elementi da 0 a $i - 1$ sono stati confrontati con tutti gli elementi successivi.

Nota

L'**invariante di ciclo** è una proprietà che rimane vera ad ogni iterazione. Serve per dimostrare che l'algoritmo è corretto. In questo caso: “alla fine dell'iterazione i , abbiamo controllato tutte le coppie che iniziano con indici da 0 a i .”

Complessità:

- Caso migliore: $\Theta(1)$ (se la coppia è trovata subito)
- Caso peggiore: $\Theta(n^2)$

Calcolo della complessità temporale (caso peggiore):

L'algoritmo esegue due cicli annidati. Il ciclo esterno itera $n - 1$ volte, e per ogni i il ciclo interno itera $n - 1 - i$ volte. Quindi:

$$T(n) = \sum_{i=0}^{n-2} (n - 1 - i) = \sum_{k=1}^{n-1} k = \frac{(n-1)n}{2} \in \Theta(n^2)$$

Perché $\Theta(n^2)$?

Il ciclo esterno fa n passi, e per ognuno il ciclo interno fa in media $n/2$ passi. In totale: $n \times n/2 = n^2/2$ operazioni. Siccome ignoriamo le costanti, rimane n^2 .

Esempio concreto: con $n = 100$ elementi, l'algoritmo fa circa $100 \times 100/2 = 5000$ confronti.

La complessità spaziale è $\Theta(1)$, poiché usiamo solo variabili costanti.

Codice JAVA

```
class twoSum {
    public int[] twoSum(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[]{i, j};
                }
            }
        }
        return new int[]{}; // nessuna soluzione trovata
    }
}
```

Nota**Nota sul codice Java:**

- `nums.length` restituisce la lunghezza dell'array
- `new int[]{i, j}` crea un nuovo array con due elementi
- Il `return` interrompe immediatamente l'esecuzione quando trova la soluzione

Problema dell'inversione numerica

Questo problema insegna a manipolare le cifre di un numero usando aritmetica modulare. È un classico per imparare operazioni su interi.

Algorithm 4: Inversione numerica**Input:** Intero x rappresentato con segno a 32 bit**Output:** Intero y con le cifre di x invertite

```

1  $y = 0;$  // Inizializza risultato
2  $sign = 1;$  // Memorizza segno
3  $sign = (x < 0) ? -1 : 1;$  // Memorizza segno di x
4  $x = |x|;$  // Rende x positivo
5 while  $x > 0$  do
6    $digit = x \bmod 10;$  // Estrae ultima cifra
7    $y = y \times 10 + digit;$  // Aggiunge a y
8    $x = \lfloor x/10 \rfloor;$  // Rimuove cifra
9 return  $y \times sign;$  // Restituisce con segno

```

Come funziona l'inversione?**Esempio:** inverti 1234

1. $digit = 1234 \bmod 10 = 4$, $y = 0 \times 10 + 4 = 4$, $x = 123$
2. $digit = 123 \bmod 10 = 3$, $y = 4 \times 10 + 3 = 43$, $x = 12$
3. $digit = 12 \bmod 10 = 2$, $y = 43 \times 10 + 2 = 432$, $x = 1$
4. $digit = 1 \bmod 10 = 1$, $y = 432 \times 10 + 1 = 4321$, $x = 0$

Trucchi usati:

- $n \bmod 10$ estrae l'ultima cifra
- $\lfloor n/10 \rfloor$ rimuove l'ultima cifra
- Moltiplicare per 10 e sommare "costruisce" il numero invertito

Complessità. Sia k il numero di cifre decimali di $|x|$. Allora

$$T(k) = \begin{cases} 6 & \text{se } x = 0, \\ 5k + 6 & \text{se } x > 0, \\ 5k + 8 & \text{se } x < 0, \end{cases} \Rightarrow T(k) = \mathcal{O}(k).$$

La complessità dell'algoritmo è lineare nel numero di cifre.

Nota

Se il numero ha k cifre, il ciclo while viene eseguito esattamente k volte (una per ogni cifra). Quindi la complessità è $\mathcal{O}(k)$, dove $k = \log_{10}(n)$ per un numero n .

Esempio: il numero 12345 ha 5 cifre, quindi $k = 5$ e il ciclo fa 5 iterazioni.

Nota

L'**operatore ternario** è definito come:

$x = (\text{espressione booleana}) ? v1 : v2$

- se l'espressione booleana è vera viene usata $v1$
- se l'espressione booleana è falsa viene usata $v2$

Esempio: $\max = (a > b) ? a : b$ assegna a $\max$ il valore più grande tra a e b .

Container con più acqua

Questo è un problema di ottimizzazione: dobbiamo trovare il rettangolo di area massima. Introduciamo la tecnica dei “due puntatori” che è molto utile per problemi su array.

Si consideri un array di interi non negativi $height$ di lunghezza n . Per ogni i , la retta verticale ha estremi $(i, 0)$ e $(i, height[i])$. Si vogliono trovare due rette che, insieme all'asse x , formino un contenitore che possa contenere la massima quantità d'acqua.

Nel nostro esempio poniamo

$$A = [4, 6, 3, 5].$$

L'area del contenitore formato dalle rette in posizione i e j ($i < j$) è data da

$$\text{area}(i, j) = (j - i) \min(A[i], A[j]).$$

Nota**Perché questa formula?**

L'area di un rettangolo è: $\text{base} \times \text{altezza}$

- **Base** = distanza orizzontale = $(j - i)$
- **Altezza** = il lato più basso = $\min(A[i], A[j])$

Usiamo il minimo perché l'acqua “trabocca” dal lato più basso, quindi l'altezza effettiva è limitata dalla barra più corta.

Per $A = [4, 6, 3, 5]$ otteniamo:

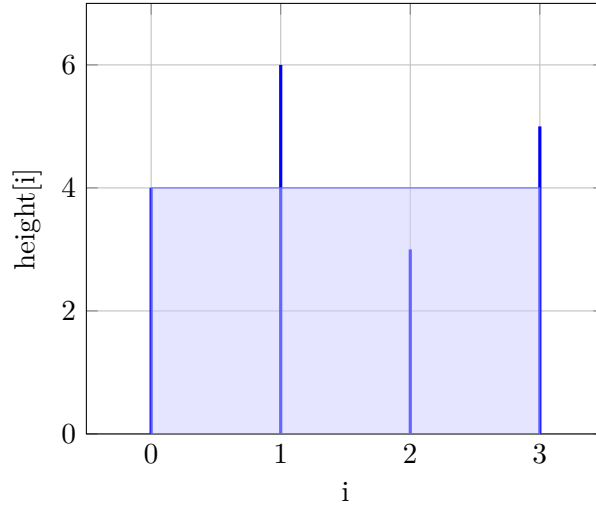


Figura 2. Container con area massima per $A = [4, 6, 3, 5]$.

Il valore massimo è quindi

$$\max_{0 \leq i < j \leq 3} \text{area}(i, j) = 12,$$

ottenuto per la coppia di indici $(i, j) = (0, 3)$.

Nell'esempio: $\text{area}(0, 3) = (3 - 0) \times \min(4, 5) = 3 \times 4 = 12$ è la massima possibile.

Quindi il pseudocodice per questo problema computazionale è il seguente:

Algorithm 5: Container With Most Water

Input: Array $\text{height}[0 \dots n - 1]$ di interi non negativi

Output: Massima area d'acqua contenibile

```

1  $left \leftarrow 0;$                                      // Puntatore sinistro
2  $right \leftarrow n - 1;$                              // Puntatore destro
3  $maxArea \leftarrow 0;$                                // Area massima trovata finora
4 while  $left < right$  do
5    $width \leftarrow right - left;$                      // Larghezza del contenitore
6    $h \leftarrow \min(\text{height}[left], \text{height}[right]);$  // Altezza = lato più basso
7    $area \leftarrow width \cdot h;$                      // Area del contenitore corrente
8    $maxArea \leftarrow \max(maxArea, area);$              // Aggiorna massimo
9   if  $\text{height}[left] < \text{height}[right]$  then
10     $left \leftarrow left + 1;$                        // Sposta il lato più basso verso il centro
11  else
12     $right \leftarrow right - 1;$                      // Sposta il lato più basso verso il centro
13 return  $maxArea;$                                    // Restituisce l'area massima
```

Tecnica dei due puntatori:

1. Partiamo dai due estremi dell'array (left=0, right=n-1)
2. Calcoliamo l'area del contenitore corrente
3. Spostiamo verso il centro il puntatore che punta alla barra **più bassa**
4. Ripetiamo fino a quando i puntatori si incontrano

Perché muoviamo il lato più basso? Se muovessimo il lato più alto, la larghezza diminuirebbe e l'altezza resterebbe limitata dal lato basso: l'area peggiorerebbe sicuramente. Muovendo il lato basso abbiamo la *possibilità* di trovare un lato più alto che migliori l'area.

Complessità. Ad ogni iterazione del ciclo **while** viene spostato solo uno dei due puntatori, che possono muoversi ciascuno al più $n - 1$ volte. Pertanto il numero di iterazioni è al più $n - 1$ e il costo è

$$T(n) = c_1 + c_2(n - 1) = \Theta(n).$$

Lo spazio aggiuntivo utilizzato è costante, quindi $S(n) = \Theta(1)$.

Nota

Complessità $\Theta(n)$ significa che l'algoritmo è **lineare**: raddoppiando la dimensione dell'input, il tempo raddoppia. Molto meglio di $\Theta(n^2)$ dove raddoppiando l'input il tempo quadruplica!

Prefisso più lungo comune fra più sequenze

Questo problema è utile per capire come elaborare stringhe. L'idea è trovare la parte iniziale condivisa da tutte le stringhe in input.

Algorithm 6: Prefisso comune più lungo

Input: Array $S[1 \dots n]$ di stringhe
Output: Prefisso più lungo comune a tutte le stringhe
 // Assumo che le stringhe di S siano tutte diverse da null

```

1  $p \leftarrow S[1];$                                 // Prefisso corrente
2 for  $i \leftarrow 2$  to  $n$  do
3    $p \leftarrow \text{PrefissoComune}(p, S[i]);$           // Prefisso tra  $p$  e  $S[i]$ 
4   if  $p = \text{stringa vuota}$  then
5     return  $p;$                                 // Nessun prefisso comune
6 return  $p;$                                     // Prefisso comune più lungo
```

Come funziona?

Esempio: $S = [\text{"fiore"}, \text{"fiorire"}, \text{"fioraio"}]$

1. $p = \text{"fiore"}$ (prima stringa)
2. $\text{PrefissoComune}(\text{"fiore"}, \text{"fiorire"}) = \text{"fiori"}$
3. $\text{PrefissoComune}(\text{"fiori"}, \text{"fioraio"}) = \text{"fior"}$
4. Risultato: "fior"

L'idea è semplice: confrontiamo il prefisso corrente con ogni stringa successiva, accorciandolo se necessario.

Invariante: Dopo ogni iterazione del ciclo, p è il prefisso comune più lungo delle prime i stringhe.

Complessità. Sia n il numero di stringhe in S e sia m la lunghezza della stringa più corta. Ad ogni passo del ciclo la funzione **PrefissoComune** confronta al più m caratteri, quindi

$$T(n, m) = c_1 + c_2(n - 1)m = \Theta(nm).$$

Lo spazio aggiuntivo utilizzato è costante, dunque $S(n, m) = \Theta(1)$.

Nota

La complessità $\Theta(nm)$ significa che nel caso peggiore confrontiamo m caratteri per ognuna delle n stringhe. Se abbiamo 100 stringhe di 50 caratteri, facciamo al massimo $100 \times 50 = 5000$ confronti.

2. InsertionSort e MergeSort

Ora studiamo due algoritmi di ordinamento fondamentali. InsertionSort è semplice ma lento, MergeSort è più complesso ma veloce. Capirai la differenza tra approccio **incrementale** e **divide et impera**.

2.0.1 InsertionSort

Inserisce ogni elemento nella posizione corretta tra quelli già ordinati (come le carte da gioco).

Nota

Analogia con le carte: quando ordini una mano di carte, prendi una carta alla volta e la inserisci nella posizione giusta tra quelle già ordinate. Questo è esattamente quello che fa InsertionSort!

Algorithm 7: InsertionSort

Input: $A = \langle a_1, \dots, a_n \rangle$, array di elementi confrontabili con $\leq$

Output: A ordinato rispetto a $\leq$

```
1 for  $i = 2$  to  $n$  do
2    $key = A[i]$  ;           // Inserisce  $A[i]$  nella parte ordinata  $a_1, \dots, a_{i-1}$ 
3    $j = i - 1$ ;
4   while  $j > 0$  and  $A[j] > key$  do
5      $A[j + 1] = A[j]$ ;
6      $j = j - 1$ ;
7    $A[j + 1] = key$ ;
8 return  $A$ 
```

Come funziona InsertionSort passo p

asso:

1. Parte dall'elemento in posizione 2 (il primo è già "ordinato")
2. Prende l'elemento corrente (**key**)
3. Lo confronta con gli elementi precedenti (da destra a sinistra)
4. Sposta gli elementi più grandi verso destra
5. Inserisce **key** nella posizione corretta
6. Ripete per tutti gli elementi

Nota

Ordina gli elementi **sul posto** (in-place), cioè non usa array aggiuntivi ma modifica direttamente l'array di input.

Ciclo Algoritmo

$$\boxed{5 \mid 2 \mid 4 \mid 6 \mid 1} = A$$

Ogni blocco rappresenta un ciclo for (0 o più cicli while)

1	2	3	4	5	Commento
5	2	4	6	1	Seq. ord.: 5 Scambio 5→2
j	i				
2	5	4	6	1	Seq. ord.: 2,5 Scambio 5→4
←	j	i			
2	4	5	6	1	Seq. ord.: 2,4,5 Nessun scambio
	←	j	i		
2	4	5	6	1	Seq. ord.: 2,4,5,6 1 → pos. 1
		←	j	i	
1	2	4	5	6	Seq. ordinata
				i	

Nella tabella:

- Rosso = elemento che stiamo inserendo (key)
- i = indice elemento corrente
- j = indice di confronto (parte da $i - 1$ e va a ritroso)
- Le frecce ← mostrano il movimento degli elementi verso destra

Invariante di ciclo

Definizione 2.1 (Invariante di ciclo). Un'asserzione (proprietà booleana) sempre vera all'inizio di ogni iterazione di un ciclo.

Nota

L'invariante di ciclo è come una **promessa** che l'algoritmo mantiene: “prima di ogni giro del ciclo, questa proprietà è vera”. Serve per dimostrare matematicamente che l'algoritmo funziona correttamente.

La validità si dimostra in tre passi:

1. **Inizializzazione:** è vera prima della prima iterazione.
2. **Conservazione:** se è vera prima di un'iterazione, rimane vera prima di quella successiva.
3. **Conclusione:** quando il ciclo termina, l'invariante fornisce una proprietà utile a dimostrare la correttezza.

Esempio 2.1 (Invariante di InsertionSort). “All'inizio del ciclo *for*, il sottoarray $A[1 : i - 1]$ è ordinato e formato dagli stessi elementi che erano in $A[1 : i - 1]$.”

- **Inizializzazione:** $i = 2$, quindi $A[1 : 1]$ è un solo elemento $\Rightarrow$ ordinato. ✓
- **Conservazione:** $\text{key} = A[i]$, il ciclo **while** sposta $A[i - 1], A[i - 2], \dots$ a destra finché non trova la posizione corretta per **key**. Quindi $A[1 : i]$ diventa ordinato con gli stessi elementi. All'iterazione successiva i aumenta di 1, l'invariante vale su $A[1 : i - 1]$. ✓
- **Conclusione:** il ciclo termina con $i = n + 1$. Per l'invariante, $A[1 : n]$ è ordinato con gli stessi elementi $\Rightarrow$ l'algoritmo è corretto. ✓

In parole semplici:

- **Inizializzazione:** all'inizio funziona (un elemento è sempre ordinato)
- **Conservazione:** se funziona al passo i , funziona anche al passo $i + 1$
- **Conclusione:** alla fine, l'intero array è ordinato

È come l'effetto domino: se il primo pezzo cade e ogni pezzo fa cadere il successivo, alla fine cadono tutti!

Modello RAM

Per analizzare la complessità si usa un modello astratto di computer: la **Random Access Machine (RAM)**.

- Un solo processore con memoria ad accesso casuale.
- Numeri interi e a virgola mobile con dimensione fissa (es. 64 bit per parola).
- Istruzioni elementari (lettura, scrittura, somma, sottrazione, moltiplicazione, divisione, confronto, if, salto, chiamata a procedura).
- Ogni istruzione ha **costo costante** = 1.

Nota

È una semplificazione: nella realtà cache, pipeline e parallelismo influenzano i tempi reali. In questo corso si ignorano questi dettagli.

Perché usiamo il modello RAM? Perché ci permette di confrontare algoritmi in modo uniforme, indipendentemente dall'hardware specifico su cui girano.

Dimensione dell'input e tempo di esecuzione

Definizione 2.2 (Dimensione dell'input). Misura della quantità di bit/word necessari per rappresentare l'input. Un indicatore è **corretto** se la dimensione reale è in relazione polinomiale con esso.

Definizione 2.3 (Tempo di esecuzione). Numero totale di operazioni elementari eseguite dall'algoritmo nel modello RAM, in funzione della dimensione n dell'input.

Nota

Indicatore corretto: una misura semplificata della dimensione dell'input che mantiene una relazione ragionevole con la dimensione reale.

Esempi:

- Per un array di n numeri: indicatore = n (numero di elementi)
- Per una matrice $n \times n$: indicatore = n (anche se contiene n^2 elementi)
- Per un numero intero: indicatore = numero di bit necessari a rappresentarlo

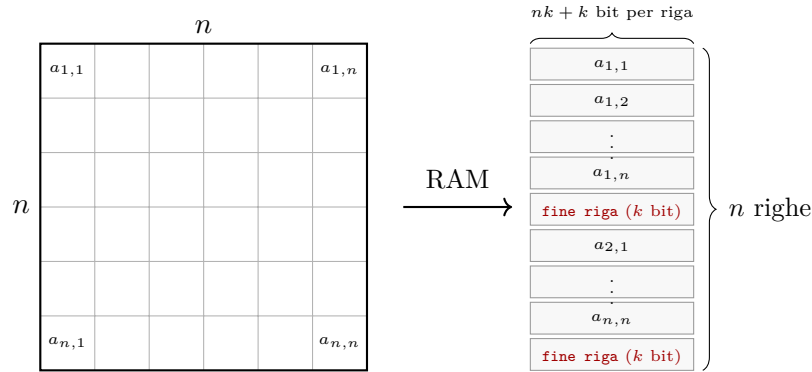
Esempi di indicatori corretti:

- **Array di n elementi:** indicatore = n . ✓
- **Matrice $n \times n$:** indicatore = n (la dimensione reale $n \cdot k(n+1)$ è polinomiale in n). ✓
- **Intero n :** indicatore = $\lceil \lg n \rceil + 1$ (numero di bit). **Non** si può usare n come indicatore perché $2^{|I|} \approx n$, relazione esponenziale. ✓

Esempio 2.2 (Matrice $n \times n$). Sia A una matrice $n \times n$ di interi, dove ogni intero richiede k bit. La RAM memorizza la matrice riga per riga.

Ogni riga occupa nk bit per gli interi, più k bit per il marcatore di fine riga. L'intera matrice richiede:

$$|I| = n(n+1)k \text{ bit} = n(kn+k) \text{ bit} = \boxed{kn^2 + kn \text{ bit}} = \Theta(n)$$



Si usa come indicatore $|I| = n$. È **corretto** perché la dimensione reale $kn^2 + kn$ è un **polinomio** in n .

Nota

L'errore comune è usare il *valore* di un intero n come dimensione dell'input invece del numero di bit necessari a rappresentarlo. Questo porta a sottostimare la complessità.

Esempio: il numero 1000 ha valore 1000, ma occupa solo $\lceil \log_2(1000) \rceil = 10$ bit. La dimensione dell'input è 10, non 1000!

Analisi di InsertionSort

c_k = costo esecuzione istruzione alla riga k . $t_i - 1 = \#$ cicli **while** nel i -esimo ciclo **for**.

INSERTION-SORT(A, n)		$cost$	$times$
1	for $i = 2$ to n	c_1	n
2	$key = A[i]$	c_2	$n - 1$
3	// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$	0	$n - 1$
4	$j = i - 1$	c_4	$n - 1$
5	while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6	$A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7	$j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8	$A[j + 1] = key$	c_8	$n - 1$

Contare le iterazioni:

for $i = \ell$ **to** n $\Rightarrow$ #Cicli = $n - \ell + 1$

Esempio: **for** $i = 2$ **to** n $\Rightarrow$ $n - 1$ iterazioni

$$T(n) = c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{i=2}^n t_i + c_6 \sum_{i=2}^n (t_i - 1) + c_7 \sum_{i=2}^n (t_i - 1) + c_8(n - 1)$$

Indichiamo con c_k il costo dell'istruzione alla riga k , e con t_i il numero di volte che il ciclo **while** viene eseguito durante l' i -esimo ciclo **for**.

Nota

t_i dipende dai dati:

- Se $A[i]$ è già al posto giusto: $t_i = 1$ (un solo confronto)
- Se $A[i]$ deve andare all'inizio: $t_i = i$ (confronta con tutti)

Caso migliore: array già ordinato

Se l'array è già ordinato, il corpo del **while** non viene mai eseguito: $t_i = 1$ per ogni i . Le somme $\sum(t_i - 1) = 0$, quindi:

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= \underbrace{(c_1 + c_2 + c_4 + c_5 + c_8)}_a n - \underbrace{(c_2 + c_4 + c_5 + c_8)}_b \\ &= an - b \quad \Rightarrow \quad \boxed{T(n) = \Theta(n)} \end{aligned}$$

Nota

Nel caso migliore la crescita è **lineare**: ogni elemento viene confrontato una sola volta con il suo predecessore e nessuno scambio viene effettuato.

Esempio pratico: se l'array è $[1, 2, 3, 4, 5]$, l'algoritmo fa solo $n - 1$ confronti e nessuno spostamento. Velocissimo!

Caso peggiore: array ordinato al contrario

Se l'array è in ordine decrescente, ogni $A[i]$ deve essere confrontato con **tutti** gli elementi in $A[1 : i - 1]$, quindi $t_i = i$. Usando $\sum_{i=2}^n i = \frac{n(n+1)}{2} - 1$:

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) + (c_6 + c_7)\frac{n(n-1)}{2} + c_8(n-1)$$

Raccogliendo i termini per grado:

$$\begin{aligned} T(n) &= \underbrace{\frac{c_5 + c_6 + c_7}{2}}_a n^2 + \underbrace{\left(c_1 + c_2 + c_4 + \frac{c_5 - c_6 - c_7}{2} + c_8\right)}_b n - \underbrace{(c_2 + c_4 + c_5 + c_8)}_c \\ &\Rightarrow \quad \boxed{T(n) = \Theta(n^2)} \end{aligned}$$

Nota

Nel caso peggiore la crescita è **quadratica**: ogni nuovo elemento deve “scendere” fino in fondo alla parte già ordinata, confrontandosi con tutti gli elementi precedenti.

Esempio pratico: se l'array è $[5, 4, 3, 2, 1]$:

- Il 4 fa 1 confronto
- Il 3 fa 2 confronti
- Il 2 fa 3 confronti

- L'1 fa 4 confronti

Totale: $1 + 2 + 3 + 4 = 10 = \frac{n(n-1)}{2}$ confronti. Molto lento!

Caso	Input	Complessità
Migliore	Array già ordinato	$\Theta(n)$
Peggior	Array ordinato inverso	$\Theta(n^2)$

Solitamente si considera come complessità di un algoritmo il tempo di esecuzione con input che rappresenta il caso peggiore.

1. È una garanzia sulla descrizione del comportamento dell'algoritmo.
2. In alcuni casi, il caso peggiore è il più frequente.
3. Il caso medio spesso è simile al caso peggiore (vedi InsertionSort).

Per InsertionSort: il caso medio è $\Theta(n^2)$ come il caso peggiore, perché in media metà degli elementi vanno spostati.

2.1 MergeSort

MergeSort è un algoritmo di ordinamento basato sul metodo **divide et impera**, più efficiente di InsertionSort nel caso peggiore: $\Theta(n \lg n)$ contro $\Theta(n^2)$.

2.1.1 Metodo Divide et Impera

Il metodo divide et impera è un paradigma di progettazione degli algoritmi che si articola in tre fasi ricorsive:

- **Divide:** il problema di dimensione n viene suddiviso in a sotto-problemi di dimensione n/b .
- **Impera:** ogni sotto-problema viene risolto ricorsivamente. La ricorsione si ferma quando il problema è sufficientemente piccolo da essere risolto direttamente (caso base).
- **Combina:** le soluzioni dei sotto-problemi vengono unite per ottenere la soluzione del problema originale.

Nota

InsertionSort usa un approccio **incrementale**: estende la soluzione di $A[1 \dots i-1]$ inserendo $A[i]$ nella posizione corretta. MergeSort usa invece **divide et impera**: risolve due metà indipendentemente e le fonde.

2.1.2 Descrizione dell'algoritmo

Applicando divide et impera al problema dell'ordinamento:

- **Divide:** l'array $A[p \dots r]$ viene spezzato in due sotto-array $A[p \dots q]$ e $A[q+1 \dots r]$, dove $q = \lfloor (p+r)/2 \rfloor$.
- **Impera:** i due sotto-array vengono ordinati ricorsivamente con MergeSort. Quando $p = r$ (sotto-array di un solo elemento), la ricorsione termina.
- **Combina:** i due sotto-array ordinati vengono fusi dall'operazione $\text{MERGE}(A, p, q, r)$.

L'operazione chiave è MERGE: dati due sotto-array adiacenti già ordinati, li fonde in un unico sotto-array ordinato.

2.1.3 Pseudocodice

Dato $A[1 \dots n]$, MergeSort si invoca con $\text{MERGE-SORT}(A, 1, n)$.

Algorithm 8: MERGE-SORT(A, p, r)

Input: Array A , indici $p \leq r$ **Output:** $A[p \dots r]$ ordinato in loco

```
1 if  $p < r$  then
2    $q = \lfloor (p + r)/2 \rfloor$ ;
3   MERGE-SORT( $A, p, q$ );
4   MERGE-SORT( $A, q + 1, r$ );
5   MERGE( $A, p, q, r$ );
```

Algorithm 9: MERGE(A, p, q, r)

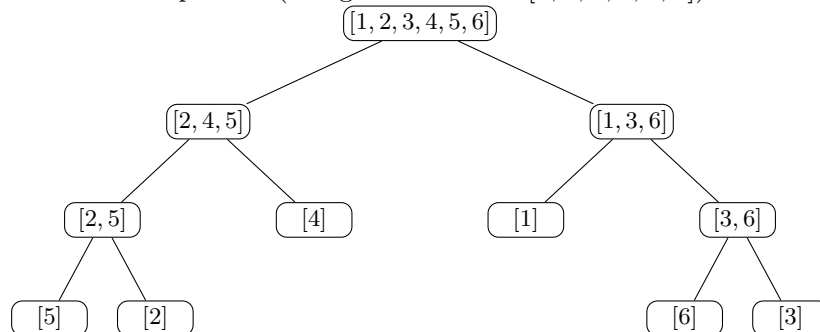
Input: $A[p \dots q]$ e $A[q + 1 \dots r]$ già ordinati**Output:** $A[p \dots r]$ ordinato

```
1  $n_L = q - p + 1$ ;
2  $n_R = r - q$ ;
3 crea array  $L[0 \dots n_L - 1]$  e  $R[0 \dots n_R - 1]$ ;
4 for  $i = 0$  to  $n_L - 1$  do
5    $L[i] = A[p + i]$ ;
6 for  $j = 0$  to  $n_R - 1$  do
7    $R[j] = A[q + 1 + j]$ ;
8  $i = 0$ ;  $j = 0$ ;  $k = p$ ;
9 while  $i < n_L$  and  $j < n_R$  do
10  if  $L[i] \leq R[j]$  then
11     $A[k] = L[i]$ ;
12     $i = i + 1$ ;
13  else
14     $A[k] = R[j]$ ;
15     $j = j + 1$ ;
16   $k = k + 1$ ;
17 while  $i < n_L$  do
18   $A[k] = L[i]$ ;
19   $i = i + 1$ ;
20   $k = k + 1$ ;
21 while  $j < n_R$  do
22   $A[k] = R[j]$ ;
23   $j = j + 1$ ;
24   $k = k + 1$ ;
```

Nota

Questa è la versione più semplice di MERGE. Nel Cormen (ed. IV) e in rete esistono versioni più compatte che usano la tecnica della **sentinella**: si aggiunge $+\infty$ in fondo a L e R , eliminando i due cicli **while** finali.

Esempio 2.3 (MergeSort su $A = [5, 2, 4, 6, 1, 3]$).



Le foglie sono i sotto-array di dimensione 1 (già ordinati). La fase di *merge* risale l'albero dal basso verso la radice, fondendo i fratelli a ogni livello. Con $n = 6$ ci sono $\lceil \lg 6 \rceil = 3$ livelli di merge, ognuno di costo $\Theta(n)$.

2.1.4 Correttezza: Invariante di ciclo di Merge

Definizione 2.4 (Invariante di MERGE). All'inizio di ogni iterazione del ciclo **while** principale, $A[p \dots k-1]$ contiene, ordinati, gli $i+j$ elementi minori di tutti gli elementi in $L[0 \dots n_L-1]$ e $R[0 \dots n_R-1]$. $L[i]$ e $R[j]$ sono gli elementi più piccoli dei rispettivi array non ancora copiati in A .

Dimostrazione. 1. **Inizializzazione:** $k = p$, quindi $A[p \dots k-1]$ è vuoto (0 elementi copiati).

Con $i = j = 0$, $L[0]$ e $R[0]$ sono i minimi non ancora copiati. L'invariante è vera. ✓

2. **Conservazione:** supponiamo $L[i] \leq R[j]$ (il caso $L[i] > R[j]$ è simmetrico). $L[i]$ è il minimo tra tutti gli elementi non ancora copiati, quindi viene copiato in $A[k]$. Incrementando k e i , l'array $A[p \dots k-1]$ contiene ora $i+j$ elementi ordinati e $L[i]$ è il nuovo minimo non copiato di L . L'invariante è mantenuta. ✓

3. **Conclusione:** il ciclo termina quando $i = n_L$ oppure $j = n_R$, cioè quando uno dei due array è esaurito. I rimanenti elementi dell'altro array (già ordinati e tutti maggiori di quelli copiati) vengono aggiunti dai due cicli **while** finali. Al termine, $A[p \dots r]$ è ordinato e contiene gli stessi elementi originali. ✓

□

2.1.5 Complessità

Complessità di Merge. Sia $n = r - p + 1$ la dimensione del sotto-array da fondere:

- Copiare in L e R : $n_L + n_R = n$ operazioni

- Ciclo **while** principale: al più n iterazioni
- Cicli **while** finali: al più n iterazioni totali

$$T_{\text{MERGE}}(n) = \Theta(n)$$

Equazione di ricorrenza di MergeSort. Con $a = 2$ sotto-problemi di dimensione $n/2$, costo di divisione $D(n) = \Theta(1)$ e costo di fusione $C(n) = \Theta(n)$:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T(n/2) + \Theta(n) & \text{altrimenti} \end{cases}$$

Applicando il **Teorema dell'Esperto**:

$$\boxed{T(n) = \Theta(n \lg n)} \quad \text{dove } \lg = \log_2$$

Soluzione per albero di ricorsione.

Nota

Ad ogni livello i dell'albero vi sono 2^i sotto-problemi di dimensione $n/2^i$, ciascuno con costo di fusione $\Theta(n/2^i)$. Il costo totale per livello è:

$$2^i \cdot \Theta\left(\frac{n}{2^i}\right) = \Theta(n)$$

Il numero di livelli è $\lg n + 1$ (le foglie hanno dimensione 1, con $n/2^{\lg n} = 1$). Sommando su tutti i livelli:

$$T(n) = (\lg n + 1) \cdot \Theta(n) = \Theta(n \lg n)$$

2.1.6 Confronto con InsertionSort

Nota

I due algoritmi rappresentano due filosofie opposte: InsertionSort usa un approccio **incrementale**, MergeSort usa **divide et impera**. La scelta dipende dal contesto.

Proprietà	InsertionSort	MergeSort
Paradigma	Incrementale	Divide et impera
Caso migliore	$\Theta(n)$	$\Theta(n \lg n)$
Caso peggiore	$\Theta(n^2)$	$\Theta(n \lg n)$
Caso medio	$\Theta(n^2)$	$\Theta(n \lg n)$
Memoria extra	$\Theta(1)$ (in-place)	$\Theta(n)$ (array L, R)
Stabile	Sì	Sì
Adatto a input piccoli	Sì	No

Quando usare InsertionSort?

- Array piccoli ($n \leq 20-30$): la costante nascosta in $\Theta(n^2)$ è talmente piccola che batte MergeSort in pratica.
- Array **quasi ordinati**: con pochi elementi fuori posto, InsertionSort è quasi $\Theta(n)$.
- Ambienti con memoria limitata: nessun array aggiuntivo.

Quando usare MergeSort?

- Array grandi: $\Theta(n \lg n)$ diventa rapidamente più conveniente di $\Theta(n^2)$.
- Quando si vuole una **garanzia** sul caso peggiore (non dipende dall'ordine dell'input).
- Ordinamento esterno (dati su disco): MergeSort si adatta naturalmente alla fusione di blocchi.

Perché $\Theta(n \lg n)$ è asintoticamente ottimale?

Qualsiasi algoritmo di ordinamento **basato su confronti** richiede nel caso peggiore almeno $\Omega(n \lg n)$ confronti (dimostrazione tramite alberi di decisione, cap. 8 CLRS). MergeSort è quindi **asintoticamente ottimale** per questa classe di algoritmi.

InsertionSort, con il suo $\Theta(n^2)$, è lontano dall'ottimo per input grandi, ma rimane utile grazie alle sue costanti moltiplicative molto piccole e alla semplicità implementativa.

Esempio 2.4 (Crescita a confronto). Per $n = 10^6$ elementi:

- InsertionSort (caso peggiore): $\sim 10^{12}$ operazioni
- MergeSort: $\sim 2 \times 10^7$ operazioni ($n \lg_2 n \approx 2 \times 10^7$)

La differenza è di **cinque ordini di grandezza**. Su un processore da 10^9 operazioni/secondo, InsertionSort impiegherebbe ~ 16 minuti, MergeSort $\sim 0,02$ secondi.

3. Notazione Asintotica

Questa sezione introduce gli strumenti matematici per confrontare la **velocità di crescita** degli algoritmi. Non ci interessa il tempo esatto in secondi (dipende dall'hardware), ma *come* cresce il tempo all'aumentare della dimensione dell'input n .

3.1 Introduzione

Quando si analizza il tempo di computazione di un algoritmo, per input sufficientemente grandi, è rilevante soltanto il **tasso di crescita**. Studiare il tasso di crescita significa studiare l'**efficienza asintotica** degli algoritmi.

Nota

Perché “asintotica”? Ignoriamo le costanti e i termini di ordine inferiore, che contano poco per input grandi.

Esempio:

- $T(n) = 3n^2 + 100n + 5$ funzione precisa
- $T(n) \in \Theta(n^2)$ classe asintotica

Per $n = 10000$, il termine $3n^2 = 3 \cdot 10^8$ domina completamente $100n = 10^6$. La costante e i termini minori diventano irrilevanti.

Anziché dare una funzione precisa per il tempo di esecuzione, si preferisce dare una **classe di funzioni**: la notazione asintotica.

3.2 Notazione O — Limite superiore

O -grande risponde a: “*al massimo quanto cresce questo algoritmo?*”. È la notazione più usata perché fornisce una **garanzia sul caso peggiore**.

Definizione 3.1 (O -grande). Data una funzione $g(n)$:

$$O(g(n)) = \{ f(n) \mid \exists c > 0, n_0 > 0 : 0 \leq f(n) \leq c g(n) \forall n \geq n_0 \}$$

Si scrive $f(n) = O(g(n))$ (abuso di notazione per $f(n) \in O(g(n))$).

Nota

In parole semplici: f cresce *al più* come g , a meno di una costante moltiplicativa, per n abbastanza grande.

Analogia: O è come il prezzo massimo di un menù fisso. Se so che costa $O(20\text{€})$, non pagherò mai di più (a meno di una costante).

Esempio 3.1. Dimostriamo che $\frac{1}{2}n^2 + 3n \in O(n^2)$.

Dobbiamo trovare $c > 0$, $n_0 > 0$ tali che $\frac{1}{2}n^2 + 3n \leq cn^2$ per ogni $n \geq n_0$.

Dividendo per n^2 : $\frac{1}{2} + 3/n \leq c$.

Per $n \geq 7$: $\frac{1}{2} + 3/7 < 1$. Quindi $c = 1$, $n_0 = 7$. ✓

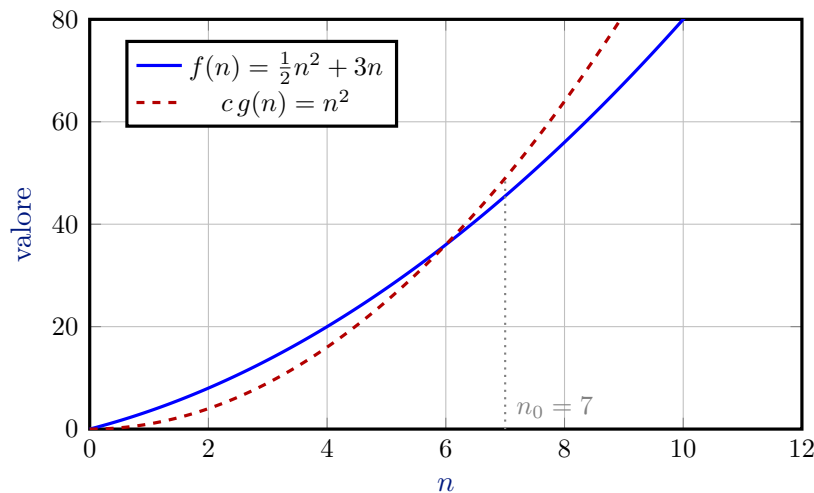


Figura 3. Da $n_0 = 7$ la curva rossa (n^2) domina quella blu ($\frac{1}{2}n^2 + 3n$), confermando $\frac{1}{2}n^2 + 3n = O(n^2)$.

O viene usata quasi sempre per i tempi di esecuzione perché è **facile da determinare**: basta trovare *un* valore di c e n_0 che funzionino — non è necessario trovare i valori ottimali.

3.3 Notazione Ω — Limite inferiore

Ω risponde a: “*almeno quanto cresce questo algoritmo?*”. Fornisce un **lower bound** sulla complessità.

Definizione 3.2 (Ω -grande). Data una funzione $g(n)$:

$$\Omega(g(n)) = \{ f(n) \mid \exists c > 0, n_0 > 0 : 0 \leq c g(n) \leq f(n) \forall n \geq n_0 \}$$

Nota

In parole semplici: f cresce *almeno* come g .

Attenzione: quando si caratterizza un tempo di esecuzione con Ω significativo (diverso da $\Omega(n)$), si deve specificare il tipo di analisi (caso peggiore o migliore), perché non esiste una prassi consolidata.

3.4 Notazione Θ — Limite stretto

Θ è la notazione più **precisa**: la funzione è “incastrata” tra due costanti di $g(n)$. Si usa quando si conosce sia il limite superiore che inferiore.

Definizione 3.3 (Θ). Data una funzione $g(n)$:

$$\Theta(g(n)) = \{ f(n) \mid \exists c_1 > 0, c_2 > 0, n_0 > 0 : 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0 \}$$

Teorema 3.1. Date $f(n), g(n)$:

$$f(n) \in \Theta(g(n)) \iff f(n) \in \Omega(g(n)) \text{ e } f(n) \in O(g(n))$$

Esempio 3.2. Dimostriamo che $\frac{1}{2}n^2 - 3n \in \Theta(n^2)$.

Si devono trovare c_1, c_2, n_0 tali che $c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$.

Dividendo per n^2 : $c_1 \leq \frac{1}{2} - 3/n \leq c_2$.

Per $n \geq 7$: scegliendo $c_1 = \frac{1}{14}$ e $c_2 = \frac{1}{2}$, la disuguaglianza è soddisfatta. ✓

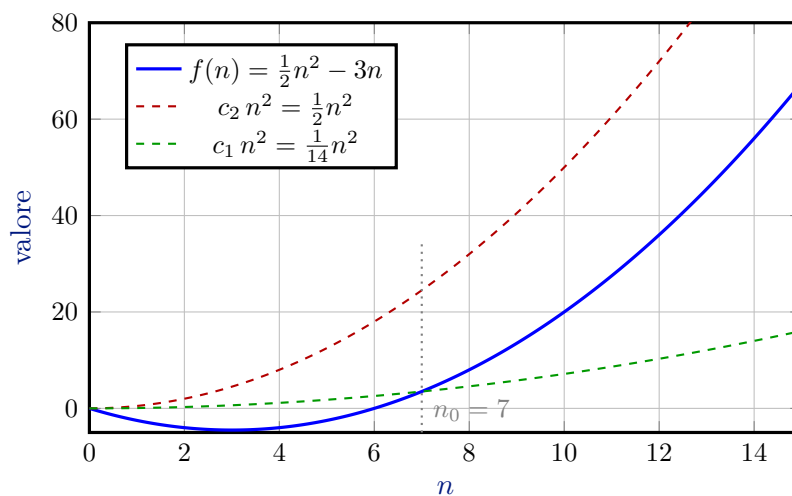


Figura 4. Per $n \geq 7$, $f(n)$ è compresa tra le due curve tratteggiate: $\frac{1}{14}n^2 \leq \frac{1}{2}n^2 - 3n \leq \frac{1}{2}n^2$, confermando $\Theta(n^2)$.

Nota

Esempio pratico — InsertionSort:

- caso migliore (array già ordinato): $\Omega(n)$
- caso peggiore (array inverso): $O(n^2)$
- se consideriamo *solo* il caso peggiore: $\Theta(n^2)$

3.5 Uso nelle equazioni e disequazioni

La notazione asintotica può comparire dentro espressioni algebriche. La convenzione è che il **lato destro** sia sempre meno preciso del sinistro: $\Theta(n)$ a destra significa “una qualche funzione in $\Theta(n)$ ”.

Osservazione 2. Si scrive $2n^2 + \Theta(n) = \Theta(n^2)$ con la seguente lettura: per ogni funzione $f(n) \in \Theta(n)$, la funzione $2n^2 + f(n)$ appartiene a $\Theta(n^2)$. Il lato sinistro è più preciso del destro: $\Theta(n)$ a destra indica “una qualche funzione in quella classe”, non un’identità tra insiemi.

Nota

Regola pratica: quando la notazione appare a sinistra dell’uguale, indica un membro anonimo della classe; a destra indica la classe complessiva. Usare $=$ è un abuso di notazione comodo ma non simmetrico: $\Theta(n) = \Theta(n^2)$ sarebbe privo di senso.

3.6 Proprietà di confronto tra funzioni

Le relazioni asintotiche si comportano in modo analogo ai confronti tra numeri reali ($\leq$, $\geq$, $=$), ma **non tutte le proprietà valgono**.

Siano f, g, h funzioni asintoticamente positive.

Teorema 3.2 (Proprietà). 1. **Transitività:** $f = O(g)$ e $g = O(h) \Rightarrow f = O(h)$ (vale anche per $\Omega, \Theta, o, \omega$).

2. **Riflessività:** $f = \Theta(f)$, $f = O(f)$, $f = \Omega(f)$.

3. **Simmetria:** $f = \Theta(g) \Leftrightarrow g = \Theta(f)$.

4. **Proprietà trasposta:** $f = O(g) \Leftrightarrow g = \Omega(f)$; $f = o(g) \Leftrightarrow g = \omega(f)$.

Nota

La **proprietà trasposta** è la più utile: dimostrare un limite superiore su g equivale a dimostrare un limite inferiore su f , e viceversa.

Osservazione 3 (Funzioni non confrontabili). Non tutte le coppie di funzioni sono confrontabili asintoticamente. Ad esempio, $f(n) = n$ e $g(n) = n^{1+\sin n}$ non soddisfano né $f = O(g)$ né $f = \Omega(g)$, perché l’esponente $1 + \sin n$ oscilla tra 0 e 2 senza stabilizzarsi.

3.7 Limiti asintotici delle funzioni elementari

I tre teoremi seguenti forniscono la **gerarchia fondamentale** tra classi di funzioni: polinomi, logaritmi e esponenziali. Sono usati continuamente nell’analisi degli algoritmi.

Teorema 3.3 (Polinomi). Sia $f(n) = a_d n^d + a_{d-1} n^{d-1} + \dots + a_0$ un polinomio di grado d con $a_d > 0$. Allora $f(n) = \Theta(n^d)$.

Dimostrazione. Basta mostrare $f(n) = O(n^d)$ e $f(n) = \Omega(n^d)$. Per il limite superiore: $f(n) \leq (|a_d| + \dots + |a_0|) n^d$ per ogni $n \geq 1$. Per il limite inferiore: $f(n) \geq \frac{a_d}{2} n^d$ per n sufficientemente grande, poiché i termini minori crescono più lentamente di $a_d n^d$. $\square$

Teorema 3.4 (Logaritmi e polinomi). Per ogni costante $a > 0$, $m \geq 0$ e $k \geq 0$:

$$(\log n^m)^k = (m \log n)^k = \Theta((\log n)^k) = O(n^a).$$

In particolare, $\log n = O(n)$.

Dimostrazione. Si dimostra $\log n = O(n^a)$ per ogni $a > 0$ usando i limiti: $\lim_{n \rightarrow \infty} \frac{\log n}{n^a} = 0$ (per L'Hôpital o per la definizione di esponenziale). $\square$

Teorema 3.5 (Esponenziali e polinomi). Per ogni costante $a > 1$ e $k \geq 0$:

$$n^k = O(a^n).$$

Dimostrazione. $\lim_{n \rightarrow \infty} n^k / a^n = 0$ per ogni $a > 1$ e k fissato, poiché gli esponenziali crescono più velocemente di qualunque polinomio. $\square$

Corollario 3.6. Per ogni $a > 1$:

$$(\log n)^k = O(n^a), \quad n^k = O(a^n), \quad a^n = O(n!).$$

Gerarchia da ricordare (dal più lento al più veloce):

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^k) \subset O(a^n) \subset O(n!)$$

Queste inclusioni derivano direttamente dai tre teoremi sopra.

3.8 Equazioni e disequazioni asintotiche

Le notazioni asintotiche servono per **eliminare dettagli non importanti**. Si trovano spesso a destra di un “=” che sostituisce $\in$:

$$2n^2 + \Theta(n) = \Theta(n^2)$$

Nota

Regola fondamentale: il lato destro di un'equazione asintotica è sempre *meno preciso* del lato sinistro. Si può leggere come: “il lato sinistro appartiene all'insieme descritto dal lato

destro”.

Esempio 3.3. Verificare le seguenti:

- $2n^2 + 3n + 1 = 2n^2 + \Theta(n) = \Theta(n^2)$ ✓
- $n^2 + 4n + 1 = n^2 + \Theta(n)?$ ✓
- $n^2 + \Theta(n) = O(n^2)?$ ✓

3.9 Proprietà delle notazioni asintotiche

Siano $f(n)$ e $g(n)$ asintoticamente positive ($f(n), g(n) > 0$ per n sufficientemente grande).

Proprietà	Θ	O / Ω
Transitiva	$f = \Theta(g), g = \Theta(h) \Rightarrow f = \Theta(h)$	vale anche per O e Ω
Riflessiva	$f = \Theta(f)$	$f = O(f), f = \Omega(f)$
Simmetrica	$f = \Theta(g) \Leftrightarrow g = \Theta(f)$	—
Trasposta	—	$f = O(g) \Leftrightarrow g = \Omega(f)$

Non tutte le funzioni sono confrontabili!

Ad esempio n e $n^{1+\sin n}$: poiché $\sin n$ oscilla tra -1 e $+1$, l'esponente oscilla tra 0 e 2 . Non si può affermare né $n = O(n^{1+\sin n})$ né il contrario.

3.10 Limiti asintotici delle funzioni elementari

Questa sezione stabilisce la **gerarchia di crescita** tra le funzioni più comuni. Da memorizzare: più si è a destra nella gerarchia, più velocemente cresce la funzione.

$$1 \prec \log n \prec n^a \prec n^b \prec c^n \prec n! \quad (0 < a < b, c > 1)$$

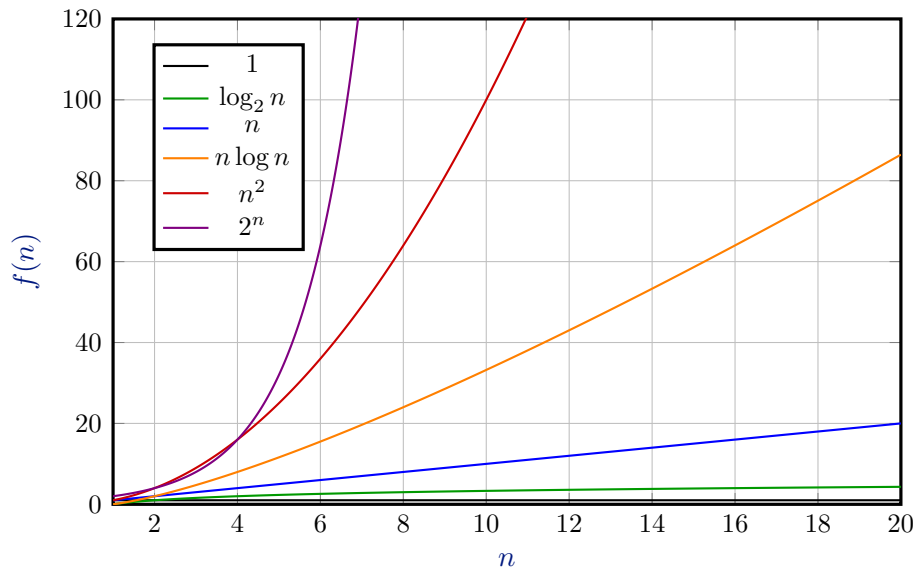


Figura 5. Confronto delle principali classi di complessità.

Teorema 3.7 (Polinomi). Sia $f(n) = a_0 + a_1n + \dots + a_dn^d$ con $a_d > 0$. Allora $f(n) \in \Theta(n^d)$.

Nota

Conseguenza pratica: per determinare la classe Θ di un polinomio basta guardare il termine di grado massimo.

Es.: $5n^3 - 200n^2 + 100n - 9 \in \Theta(n^3)$.

Teorema 3.8 (Logaritmi e polinomi). Per ogni $k, m, a > 0$: $\log^m n^k = O(n^a)$.

Qualsiasi funzione polilogaritmica è dominata da qualsiasi polinomio positivo in n .

Esempio 3.4. $\log^{107} n = O(n^{0,00001})$

Teorema 3.9 (Esponenziali e polinomi). Per ogni $a > 1, k > 0$: $n^k = O(a^n)$.

Qualsiasi funzione esponenziale con base > 1 cresce più rapidamente di qualsiasi polinomio.

Esempio 3.5. $n^{1000} = O(1,01^n)$

Gerarchia da memorizzare:

$$\log n \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \ll n!$$

4. Divide et Impera

Il **divide et impera** è una delle tecniche algoritmiche più potenti. L'idea è semplice: se un problema è troppo grande per essere risolto direttamente, spezzalo in pezzi più piccoli, risolvi i pezzi, e combina le soluzioni.

4.1 Idea Generale

Il paradigma divide et impera prevede tre passi:

1. **Divide:** suddividi il problema in sottoproblemi di dimensione inferiore;
2. **Impera:** risolvi ricorsivamente i sottoproblemi (quando la dimensione è abbastanza piccola, risolvi direttamente);
3. **Combina:** unisci i risultati per ottenere la soluzione al problema originale.

Nota

Analogia: immagina di dover ordinare un mazzo di 52 carte. Invece di ordinarle tutte in una volta (difficile), le dividi in due mazzi da 26, ordini ciascun mazzo separatamente, e poi fondi i due mazzi ordinati. Questo è esattamente MergeSort!

Aspetti cruciali

- **Suddivisione:** deve produrre sottoproblemi il più possibile *disgiunti*.
- **Ricorsione:** deve terminare quando la dimensione permette una risoluzione diretta (caso base).
- **Combinazione:** non deve richiedere un tempo maggiore di quello della risoluzione dei sottoproblemi.

4.2 Schema Generale

Algorithm 10: DivideEtImpera(X)

Input: X : istanza di dimensione $|X| = n$

Output: Y : soluzione

```
1 if  $|X| \leq n_0$  then
2   return ad hoc( $X$ );                                // Caso base: soluzione diretta
3 Dividi  $X$  in sottoistanze  $X_1, X_2, \dots, X_a$ ;
4 for  $i = 1$  to  $a$  do
5    $Y_i \leftarrow \text{DivideEtImpera}(X_i)$ ;                // Ricorsione
6  $Y \leftarrow \text{combina}(Y_1, \dots, Y_a)$ ;
7 return  $Y$ ;
```

- **ad hoc**(X): risolve istanze piccole in tempo costante o lineare.
- **dividi**: produce a sottoistanze di dimensione n/b ciascuna ($b > 1$).
- Se $a = 1$: si chiama *semplificazione* (es. ricerca binaria).
- **combina**: ha solitamente complessità $O(n^d)$.

4.3 Costo Computazionale

Il costo è la soluzione di un'equazione di ricorrenza:

$$T(n) = \begin{cases} g(n) & n \leq n_0 \\ aT(n/b) + f(n) & \text{altrimenti} \end{cases}$$

dove $f(n) = \text{tempo}(\text{dividi}) + \text{tempo}(\text{combina})$.

Nota

Ricorsione o iterazione?

- **Pro iterazione**: elimina l'overhead delle chiamate ricorsive; risparmia fino a $\Theta(h)$ spazio ausiliario.
- **Contro**: l'algoritmo diventa più complicato da scrivere.

4.4 Esempi di Applicazione

4.4.1 Ricorsione o iterazione?

Prima di entrare negli esempi, una domanda pratica: conviene sempre usare la ricorsione nei divide et impera?

È sempre possibile sostituire la ricorsione con un ciclo iterativo.

Pro	Si elimina l'overhead delle chiamate ricorsive; si risparmia fino a $\Theta(h)$ spazio ausiliario, dove h è la profondità della ricorsione.
Contro	L'algoritmo diventa più complesso da scrivere e da leggere.

4.4.2 Problema Prodotto — Algoritmo di Karatsuba

Il primo esempio concreto di divide et impera è la moltiplicazione di interi grandi. La moltiplicazione tradizionale è $O(n^2)$: possiamo fare meglio?

Definizione 4.1 (Problema del Prodotto). **Input:** due interi x, y di n bit ciascuno.
Output: il loro prodotto $x \cdot y$.

Il costo del prodotto tradizionale (moltiplicazione in colonna) è $O(n^2)$.

Idea divide et impera. Si decompongono x e y nelle loro metà alta e bassa:

$$x = 2^{n/2} x_H + x_L, \quad y = 2^{n/2} y_H + y_L.$$

Il prodotto diventa:

$$x \cdot y = 2^n x_H y_H + 2^{n/2} (x_H y_L + x_L y_H) + x_L y_L.$$

Questa decomposizione usa **4 moltiplicazioni** di dimensione $n/2$, quindi $T(n) = 4T(n/2) + \Theta(n) = \Theta(n^2)$: nessun miglioramento.

Osservazione di Gauss (Karatsuba). Con un trucco analogo a quello usato per i numeri complessi, si può calcolare il termine misto $x_H y_L + x_L y_H$ usando **una sola** moltiplicazione aggiuntiva:

$$x_H y_L + x_L y_H = (x_H + x_L)(y_H + y_L) - x_H y_H - x_L y_L.$$

Poiché $x_H y_H$ e $x_L y_L$ sono già stati calcolati, bastano **3 moltiplicazioni** in totale.

Algorithm 11: Prodotto(x, y, n)

Input: Interi x, y di n bit

Output: $x \cdot y$

```

1 if  $n = 1$  then
2   return  $x \cdot y$ ;
3  $x_H \leftarrow \lfloor x/2^{n/2} \rfloor$ ;  $x_L \leftarrow x \bmod 2^{n/2}$ ;
4  $y_H \leftarrow \lfloor y/2^{n/2} \rfloor$ ;  $y_L \leftarrow y \bmod 2^{n/2}$ ;
5  $p \leftarrow \text{Prodotto}(x_H, y_H, n/2)$ ;
6  $q \leftarrow \text{Prodotto}(x_L, y_L, n/2)$ ;
7  $r \leftarrow \text{Prodotto}(x_H + x_L, y_H + y_L, n/2)$ ;
8 return  $2^n p + 2^{n/2}(r - p - q) + q$ ;
```

Analisi.

$$T(n) = 3T(n/2) + \Theta(n).$$

Applicando il Teorema dell'Esperto (caso 1, con $\log_2 3 \approx 1.585 > 1$):

$$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}).$$

Nota

$\Theta(n^{1.585})$ è significativamente meglio di $\Theta(n^2)$ del metodo tradizionale. Per $n = 1000$ bit, il rapporto è circa $1000^{0.415} \approx 50 \times$ più veloce.

4.4.3 Tabella di esempi divide et impera

Algoritmo	a	b	Complessità
Ricerca binaria	1	2	$\Theta(\log n)$
MergeSort	2	2	$\Theta(n \log n)$
Karatsuba	3	2	$\Theta(n^{1.585})$
Massimo Sottoarray	2	2	$\Theta(n \log n)$
Selezione (mediana)	5	5	$\Theta(n)$
Coppia più vicina	2	2	$\Theta(n \log n)$

4.4.4 Prodotto di Interi Grandi

Definizione 4.2 (Problema Prodotto). **Input:** due interi x, y di n bit ciascuno ($n = 2^k$).

Output: il loro prodotto xy .

Costo naïve: $O(n^2)$.

Si suddivide $x = 2^{n/2}x_L + x_R$, $y = 2^{n/2}y_L + y_R$:

$$xy = 2^n(x_L y_L) + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$

Questo richiede 4 moltiplicazioni di dimensione $n/2$, dando $T(n) = 4T(n/2) + O(n) = O(n^2)$ — nessun miglioramento.

Trucco di Gauss (1777–1855): bastano **3 moltiplicazioni**:

$$P_1 = x_L y_L, \quad P_2 = x_R y_R, \quad P_3 = (x_L + x_R)(y_L + y_R)$$

$$xy = 2^n P_1 + 2^{n/2}(P_3 - P_1 - P_2) + P_2$$

Algorithm 12: Prodotto(x, y)**Input:** x, y interi non negativi di $n = 2^\ell$ bit**Output:** prodotto xy

```

1 if  $n \leq 64$  then
2   return  $x \cdot y$ ;                                     // Caso base  $\Theta(1)$ 
3  $x_L, x_R \leftarrow$  metà sinistra/destra di  $x$ ;         // Divide  $\Theta(n)$ 
4  $y_L, y_R \leftarrow$  metà sinistra/destra di  $y$ ;
5  $P_1 \leftarrow$  Prodotto( $x_L, y_L$ );
6  $P_2 \leftarrow$  Prodotto( $x_R, y_R$ );                     // Impera  $3T(n/2)$ 
7  $P_3 \leftarrow$  Prodotto( $x_L + x_R, y_L + y_R$ );
8 return  $(P_1 \ll n) + ((P_3 - P_1 - P_2) \ll n/2) + P_2$ ; // Combina  $\Theta(n)$ 

```

Complessità:

$$T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.585})$$

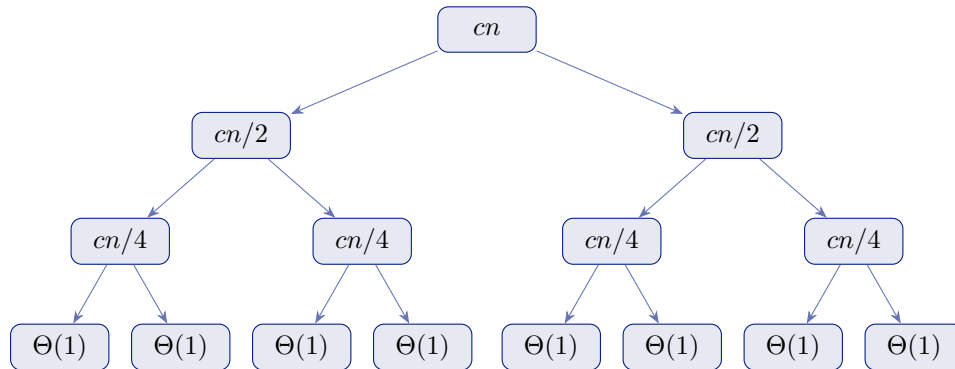
4.4.5 MergeSort

Lo schema divide et impera per MergeSort:

- **Divide:** 2 sottoarray di $n/2$ elementi — costo $\Theta(1)$.
- **Impera:** ordina i due sottoarray ricorsivamente.
- **Combina:** fondi i due sottoarray ordinati — costo $\Theta(n)$.

Analisi di complessità

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 2T(n/2) + \Theta(n) & n > 1 \end{cases} \Rightarrow T(n) = \Theta(n \log n)$$

**Figura 6.** Albero di ricorsione di MergeSort. Ogni livello costa cn ; con $\log_2 n$ livelli, il totale è $\Theta(n \log n)$.

È stato dimostrato che *nessun* algoritmo basato su confronti può ordinare in meno di $\Omega(n \log n)$ nel caso peggiore. MergeSort raggiunge questo limite!

4.4.6 Massimo SottoArray

Definizione 4.3 (Massimo SottoArray). **Input:** un array di interi $A[1 : n]$.

Output: il sottoarray non vuoto $A[i : j]$ con somma massima:

$$\max_{1 \leq i \leq j \leq n} \sum_{k=i}^j A[k]$$

Esempio 4.1. Profitti 2001–2012: il massimo sottoarray è 2008–2010 con somma $32 - 5 + 10 = 37$.

Strategia divide et impera

Posto $mid = \lfloor (low + high)/2 \rfloor$, la soluzione è in uno di tre casi:

1. Interamente in $A[low : mid]$
2. Interamente in $A[mid + 1 : high]$
3. **A cavallo** di mid

Algorithm 13: FindMaxCrossingSubarray($A, low, mid, high$)

Input: Array A , indici $low \leq mid < high$

Output: (i^*, j^*, somma)

```

1 leftSum  $\leftarrow -\infty$ ;
2 sum  $\leftarrow 0$ ;
3 for  $i \leftarrow mid$  downto  $low$  do
4   sum  $\leftarrow sum + A[i]$ ;
5   if sum > leftSum then
6     leftSum  $\leftarrow sum$ ;
7      $i^* \leftarrow i$ ;
8 rightSum  $\leftarrow -\infty$ ;
9 sum  $\leftarrow 0$ ;
10 for  $j \leftarrow mid + 1$  to  $high$  do
11   sum  $\leftarrow sum + A[j]$ ;
12   if sum > rightSum then
13     rightSum  $\leftarrow sum$ ;
14      $j^* \leftarrow j$ ;
15 return  $(i^*, j^*, leftSum + rightSum)$ ;
```

Algorithm 14: FindMaximumSubarray($A, low, high$)**Input:** Array $A[low : high]$ **Output:** $(i^*, j^*, \text{somma massima})$

```

1 if  $high = low$  then
2   return  $(low, high, A[low]);$  // Caso base
3  $mid \leftarrow \lfloor (low + high)/2 \rfloor;$ 
4  $(i_L, j_L, s_L) \leftarrow \text{FindMaximumSubarray}(A, low, mid);$ 
5  $(i_R, j_R, s_R) \leftarrow \text{FindMaximumSubarray}(A, mid + 1, high);$ 
6  $(i_C, j_C, s_C) \leftarrow \text{FindMaxCrossingSubarray}(A, low, mid, high);$ 
7 return  $\arg \max\{s_L, s_R, s_C\};$ 

```

$$T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$$

4.5 Equazioni di Ricorrenza

Definizione 4.4 (Equazione di ricorrenza algoritmica).

$$T(n) = \begin{cases} K & n \leq k \\ aT(n/b) + C(n) & \text{altrimenti} \end{cases}$$

4.5.1 Metodo di Sostituzione

(1) ipotizza una soluzione; **(2) dimostra** per induzione che è corretta.

Esempio 4.2. Risolvere $T(n) = 2T(\lfloor n/2 \rfloor) + n$.**Ipotesi:** $T(n) \leq cn \log n$. Passo induttivo:

$$\begin{aligned} T(n) &\leq 2(c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor) + n \\ &\leq cn \log(n/2) + n = cn \log n - cn + n \leq cn \log n \end{aligned}$$

valido per $c \geq 1$. ✓

Errore comune: se ipotizzo $T(n) = O(n)$ e ottengo $T(n) \leq cn + n = (c+1)n$, **non** ho dimostrato $T(n) \leq cn$. La soluzione $O(n)$ è quindi errata per questa ricorrenza.

4.5.2 Metodo dell'Albero di Ricorsione

Esempio 4.3. Risolvere $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$.Costo livello i : $3^i \cdot c(n/4^i)^2 = (3/16)^i cn^2$.

Totale (serie geometrica con ragione $3/16 < 1$):

$$\sum_{i=0}^{\log_4 n} (3/16)^i cn^2 < \frac{cn^2}{1 - 3/16} = \frac{16}{13}cn^2 = O(n^2)$$

4.5.3 Metodo dell'Esperto

Teorema 4.1 (Teorema dell'Esperto). *Date costanti $a > 0$, $b > 1$ e $f(n)$ asintoticamente positiva, per $T(n) = aT(n/b) + f(n)$:*

1. Se $f(n) = O(n^{(\log_b a) - \varepsilon})$: $T(n) = \Theta(n^{\log_b a})$ (foglie dominano)
2. Se $f(n) = \Theta(n^{\log_b a} \log^k n)$: $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$ (bilanciato)
3. Se $f(n) = \Omega(n^{(\log_b a) + \varepsilon})$ e $af(n/b) \leq cf(n)$: $T(n) = \Theta(f(n))$ (radice domina)

Esempio 4.4. **Caso 1** — $T(n) = 9T(n/3) + n$: $n^{\log_3 9} = n^2$, $f(n) = n = O(n^{2-1}) \Rightarrow T(n) = \Theta(n^2)$.

Esempio 4.5. **Caso 2** — $T(n) = T(n/2) + 1$ (ricerca binaria): $n^{\log_2 1} = 1$, $f(n) = \Theta(1) \Rightarrow T(n) = \Theta(\log n)$.

Esempio 4.6. **Caso 3** — $T(n) = 3T(n/4) + n^2$: $n^{\log_4 3} \approx n^{0.79}$, $f(n) = n^2$, $3(n/4)^2 \leq \frac{3}{16}n^2 \Rightarrow T(n) = \Theta(n^2)$.

Errore comune nel metodo di sostituzione. Si consideri $T(n) = 2T(\lfloor n/2 \rfloor) + n$ e si ipotizzi $T(n) = O(n)$, cioè $T(n) \leq cn$. Passo induttivo:

$$T(n) \leq 2c\lfloor n/2 \rfloor + n \leq cn + n = (c+1)n.$$

Questo **non** dimostra $T(n) \leq cn$, perché la costante è aumentata da c a $c+1$. L'ipotesi $O(n)$ è quindi **errata** per questa ricorrenza; la soluzione corretta è $T(n) = O(n \log n)$.

Osservazione 4. Il Teorema dell'Esperto **non si applica** quando $f(n)$ non è polinomialmente separabile da $n^{\log_b a}$. Esempio tipico:

$$T(n) = 2T(n/2) + n \log n.$$

Qui $f(n) = n \log n$ e $n^{\log_2 2} = n$, ma $n \log n / n = \log n$ non è $\Omega(n^\varepsilon)$ per nessun $\varepsilon > 0$: non si rientra in nessuno dei tre casi. La soluzione è $T(n) = \Theta(n \log^2 n)$, ottenibile con l'albero di ricorsione o con il metodo di sostituzione.

Caso non applicabile: $T(n) = 2T(n/2) + n \log n$.

$n^{\log_2 2} = n$. $f(n) = n \log n$ è più grande di n ma non *polinomialmente* più grande: non esiste $\varepsilon > 0$ con $n \log n = \Omega(n^{1+\varepsilon})$. Occorre il metodo di sostituzione.

4.6 Riepilogo: Problemi risolvibili con Divide et Impera

Problema	Costo naïve	Costo D&I
Ricerca in array ordinato	$O(n)$	$O(\log n)$
Ordinamento per confronto	$O(n^2)$	$\Theta(n \log n)$
Prodotto di interi n -bit	$O(n^2)$	$O(n^{1.585})$
Massimo SottoArray	$\Theta(n^3)$	$\Theta(n \log n)$
Ricerca coppia più vicina	$O(n^2)$	$O(n \log n)$
Ricerca della mediana	$\Theta(n \log n)$	$\Theta(n)$

5. Analisi Probabilistica e Algoritmi Randomizzati

Finora abbiamo analizzato gli algoritmi nel caso peggiore. Questa sezione introduce un'alternativa: l'analisi del **caso medio**, e gli **algoritmi randomizzati** che rendono l'analisi media valida per qualunque input.

5.1 Preliminari: variabili casuali indicatrici

Nota

Si assume che lo studente conosca il concetto di variabile casuale. Chi non lo conosce può studiare l'Appendice C di [CLRS] come riferimento sufficiente per questa sezione.

Definizione 5.1 (Variabile casuale indicatrice). Dato uno spazio campionario S e un evento A , la **variabile casuale indicatrice** $I\{A\}$ è:

$$I\{A\} = \begin{cases} 1 & \text{se } A \text{ si verifica} \\ 0 & \text{altrimenti} \end{cases}$$

Lemma 5.1. Dato uno spazio campionario S e un evento A , posto $X_A = I\{A\}$, si ha:

$$E[X_A] = \Pr\{A\}.$$

Dimostrazione. $E[X_A] = 1 \cdot \Pr\{A\} + 0 \cdot \Pr\{\bar{A}\} = \Pr\{A\}.$ □

Nota

Perché sono utili? Le v.c. indicatrici trasformano il calcolo di medie complesse in somme di probabilità semplici, grazie alla **linearità del valore atteso**: $E[\sum_i X_i] = \sum_i E[X_i]$, che vale anche quando le X_i sono *dipendenti*.

Esempio 5.1. Numero atteso di teste in n lanci di moneta. Sia $X_i = I\{\text{testa all}'i\text{-esimo lancio}\}$, e $X = \sum_{i=1}^n X_i$.

$$E[X] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n \frac{1}{2} = \frac{n}{2}.$$

Notare che si evita il calcolo diretto della distribuzione binomiale.

5.2 Analisi probabilistica: il Problema delle Assunzioni

Definizione 5.2 (Problema delle Assunzioni). **Input:** n candidati ordinabili per bravura; costo di colloquio $c_c > 0$, costo di assunzione $c_a \gg c_c$.

Output: assumere il candidato migliore con il minimo costo.

Strategia: si assume il primo candidato; si sostituisce il corrente ogni volta che si incontra uno migliore.

Se m è il numero di assunzioni effettuate, il costo totale è:

$$n \cdot c_c + m \cdot c_a.$$

Il costo $n \cdot c_c$ è fisso; quello che ci interessa è $m \cdot c_a$. Nel caso peggiore $m = n$, nel caso migliore $m = 1$. Vogliamo calcolare il **costo medio**, che dipende dalla distribuzione dei candidati.

Calcolo del costo medio. Si assuma che le $n!$ permutazioni dei candidati siano equiprobabili. Sia $X_i = I\{\text{il candidato } i\text{-esimo viene assunto}\}$, $X = \sum_{i=1}^n X_i$.

Il candidato in posizione i viene assunto se e solo se è il migliore tra i primi i . Poiché ogni permutazione dei primi i candidati è equiprobabile:

$$\Pr\{X_i = 1\} = \frac{1}{i}.$$

Quindi:

$$E[X] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n \frac{1}{i} \leq \ln n + 1.$$

Teorema 5.2 (Costo medio del Problema delle Assunzioni).

$$\text{Costo medio} \leq c_a(\ln n + 1) = O(c_a \log n),$$

contro il costo nel caso peggiore $O(c_a n)$.

Nota

$\sum_{i=1}^n 1/i$ è la **serie armonica**. Vale l'approssimazione: $\ln(n+1) \leq \sum_{i=1}^n \frac{1}{i} \leq \ln n + 1$. Ad esempio, per $n = 100$: $4.615 \leq 5.177 \leq 5.605$.

5.3 Algoritmi randomizzati

Assumere una distribuzione uniforme dell'input non è sempre realistico. Gli algoritmi randomizzati risolvono questo problema imponendo **essi stessi** la distribuzione, indipendentemente dall'input.

Analisi probabilistica	Algoritmo randomizzato
Algoritmo deterministico	Algoritmo non deterministico
Stesso input $\rightarrow$ stesso risultato	Stesso input $\rightarrow$ esecuzione diversa
Ipotesi sulla distribuzione dell'input	Nessuna ipotesi necessaria
Caso peggiore dipende dall'input	Il comportamento non dipende da un input specifico

Osservazione 5. Nel Problema delle Assunzioni, l'algoritmo randomizzato permuta casualmente i candidati *prima* di iniziare. In questo modo il costo medio $O(c_a \log n)$ è garantito **per qualunque input**, non solo per quelli ben distribuiti.

5.4 Permutazione casuale: RandomizeInPlace

Algorithm 15: RandomizeInPlace(A)

Input: Array $A[1 : n]$

Output: A con una permutazione casuale uniforme

```

1  $n \leftarrow |A|$ ;
2 for  $i = 1$  to  $n$  do
3   └─ scambia  $A[i]$  con  $A[\text{RANDOM}(i, n)]$ ;
4 return  $A$ ;
```

Complessità in tempo: $O(n)$ — n iterazioni, ciascuna con un numero costante di operazioni e una chiamata a $\text{RANDOM}(i, n)$, assunta a costo costante.

Complessità in spazio: $O(1)$ — nessun array ausiliario.

Lemma 5.3. *RandomizeInPlace genera una permutazione casuale uniforme (ogni delle $n!$ permutazioni ha probabilità $1/n!$).*

Dimostrazione. Si dimostra tramite invariante di ciclo.

Invariante: prima della i -esima iterazione, per ogni possibile $(i - 1)$ -permutazione, il sottoarray $A[1 : i - 1]$ la contiene con probabilità $\frac{(n-i+1)!}{n!}$.

Base ($i = 1$): il sottoarray $A[1 : 0]$ è vuoto; l'unica 0-permutazione (vuota) vi compare con probabilità $1 = n!/n!$. ✓

Conservazione: si assume che ogni $(i-1)$ -permutazione $\langle x_1, \dots, x_{i-1} \rangle$ compaia in $A[1 : i-1]$ con probabilità $(n-i+1)!/n!$. La probabilità che l'algoritmo scelga x_i come i -esimo elemento è $1/(n-i+1)$. Quindi la probabilità di ottenere $\langle x_1, \dots, x_{i-1}, x_i \rangle$ è:

$$\frac{(n-i+1)!}{n!} \cdot \frac{1}{n-i+1} = \frac{(n-i)!}{n!}.$$

Questo è esattamente l'invariante per $i+1$. ✓

Conclusione: alla fine ($i = n+1$) ogni n -permutazione compare in $A[1 : n]$ con probabilità $\frac{(n-n)!}{n!} = \frac{1}{n!}$. □

Da ricordare all'esame: la correttezza di `RandomizeInPlace` si dimostra con un invariante di ciclo, non semplicemente contando le permutazioni. Il punto chiave è che a ogni passo la scelta uniforme in $A[i : n]$ garantisce che tutte le possibili estensioni siano equiprobabili.

5.5 QuickSort

QuickSort è uno degli algoritmi di ordinamento più usati in pratica. A differenza di MergeSort, ordina **sul posto** e, pur avendo caso peggiore $\Theta(n^2)$, nel caso medio è $\Theta(n \log n)$ con costanti moltiplicative molto piccole.

5.6 Idea Generale

QuickSort è un algoritmo che:

- ordina **sul posto** come InsertionSort;
- è basato sulla tecnica **divide et impera** come MergeSort;
- ha complessità $\Theta(n^2)$ nel caso peggiore e $\Theta(n \log n)$ nel caso medio (qui si dimostra il caso medio con l'ulteriore assunzione che i valori siano tutti distinti);
- nel caso medio le costanti moltiplicative nascoste da Θ sono abbastanza piccole $\Rightarrow$ molto efficiente in pratica, tra i più usati per l'ordinamento interno.

Nota

In Java viene usato, nella sua versione parallela, quando si devono ordinare ≥ 44 elementi.

Schema divide et impera.

- **Divide:** partiziona l'array $A[p : r]$ in due sotto-array $A[p : q-1]$ e $A[q+1 : r]$ tali che ogni elemento del primo sia $\leq A[q]$ (il *pivot*) e ogni elemento del secondo sia $> A[q]$.

- **Impera:** ordina i due sotto-array in modo ricorsivo fino a quando la dimensione è 0 o 1 (quando $p \geq r$).
- **Combina:** dato che i due sotto-array sono già ordinati al ritorno della chiamata ricorsiva, non serve alcuna ulteriore fase di combinazione.

Algorithm 16: QuickSort(A, p, r)**Input:** Array A , indici p, r **Output:** $A[p : r]$ ordinato in loco

```

1 if  $p < r$  then
2    $q \leftarrow \text{Partition}(A, p, r)$ ;
3   QuickSort( $A, p, q - 1$ );
4   QuickSort( $A, q + 1, r$ );

```

Nota

Per ordinare un intero array A di n elementi si invoca QuickSort($A, 1, n$).

5.7 Partition(A, p, r)

Partition è la componente fondamentale di QuickSort. Sceglie un valore *pivot* (l'ultimo elemento $A[r]$), riorganizza l'array e restituisce l'indice q della posizione finale del pivot.

Algorithm 17: Partition(A, p, r)**Input:** Array A , indici p, r **Output:** q e A modificato con $A[p : q - 1] \leq A[q] < A[q + 1 : r]$

```

1  $x \leftarrow A[r]$ ;           // Pivot: scegliere l'ultimo valore è utile per il for
2  $i \leftarrow p - 1$ ;         // Indice ultimo elemento sotto-array sinistro
3 for  $j \leftarrow p$  to  $r - 1$  do
4   if  $A[j] \leq x$ ;           // Se l'elemento deve andare nel sotto-array sx
5   then
6      $i \leftarrow i + 1$ ;      // Posizione per l'elemento da spostare
7     scambia  $A[i]$  con  $A[j]$ ;
8  $q \leftarrow i + 1$ ;         // Posizione per il pivot
9 scambia  $A[q]$  con  $A[r]$ ;
10 return  $q$ ;

```

Esempio 5.2 (Partition su $A = [2, 8, 7, 1, 3, 5, 6, 4]$, pivot = $A[8] = 4$). Traccia di PARTITION($A, 1, 8$): il pivot è $p = 4$, i parte da 0 e j scorre da 1 a 7.

Passo	Array	Azione
Inizio	[2 , 8, 7, 1, 3, 5, 6 <u>4</u>]	$i = 0$
$j = 1$	[2 , 8, 7, 1, 3, 5, 6 4]	$A[1] = 2 \leq 4$: $i++$, swap(1, 1)
$j = 2$	[2, 8 , 7, 1, 3, 5, 6 4]	$A[2] = 8 > 4$: nulla
$j = 3$	[2, 8, 7 , 1, 3, 5, 6 4]	$A[3] = 7 > 4$: nulla
$j = 4$	[2, 8, 7, 1 , 3, 5, 6 4]	$A[4] = 1 \leq 4$: $i++$, swap(2, 4)
$j = 5$	[2, 1, 7, 3 , 8, 5, 6 4]	wait... ricordare: $A[4] \leftrightarrow A[2]$
$j = 5$	[2, 1, 7, 8, 3 , 5, 6 4]	$A[5] = 3 \leq 4$: $i++$, swap(3, 5)
$j = 6$	[2, 1, 3, 8, 7 , 5, 6 4]	—
$j = 7$	[2, 1, 3, 8, 7, 5 , 6 4]	$A[6] = 5 > 4$: nulla
Fine	[2, 1, 3, 4 , 7, 5, 6, 8]	swap($i+1, 8$): pivot in pos. 4

Risultato: $\underbrace{[2, 1, 3]}_{\leq 4}, \underbrace{[4, 7, 5, 6, 8]}_{> 4}$. Il pivot è nella posizione corretta; i due sotto-array vengono ordinati ricorsivamente.

Dopo PARTITION, il pivot si trova nella sua posizione *definitiva*: non verrà mai spostato nelle chiamate ricorsive.

5.7.1 Invariante di ciclo di Partition

Per dimostrare la correttezza di **Partition**(A, p, r), si considera la seguente invariante di ciclo:

Definizione 5.3 (Invariante di Partition). All'inizio di ogni iterazione del ciclo **for**, per qualsiasi indice k dell'array vale almeno una delle seguenti proprietà:

1. Se $p \leq k \leq i$, allora $A[k] \leq x$;
2. Se $i + 1 \leq k \leq j - 1$, allora $A[k] > x$;
3. Se $k = r$, allora $A[k] = x$ (il pivot).

Nota

Non si considerano i valori con indice *t.c.* $j \leq k \leq r - 1$ perché sono valori non ancora considerati e quindi non ancora posizionati correttamente.

Dimostrazione. 1. **Inizializzazione** (prima della I iterazione, $i = p - 1$, $j = p$): non ci sono valori tra p e $i + 1 = p$ né tra $i + 1 = p$ e $j - 1 = p - 1$. Le prime due proprietà dell'invariante sono banalmente soddisfatte. L'istruzione $x \leftarrow A[r]$ soddisfa la III proprietà. ✓

2. **Conservazione**: ci sono due possibilità.

- Se $A[j] > x$ (riga 4 non eseguita), allora non si fa nulla e $j' = j + 1$: la proprietà 2 dell'invariante è soddisfatta all'inizio del prossimo giro perché $j' - 1 = j$.
 - Se invece $A[j] \leq x$, allora $i' = i + 1$, vengono scambiati $A[i']$ e $A[j]$, poi $j' = j + 1$. In seguito allo scambio, $A[i'] \leq x$ e la proprietà 1 è soddisfatta. Anche $A[j' + 1] > x$ perché era l'elemento $A[i]$ prima dello scambio. Quindi anche la proprietà 2 è soddisfatta.
3. **Conclusione:** alla fine del ciclo $j = r$. Ogni posizione dell'array si trova in uno dei tre insiemi descritti nell'invariante. L'istruzione alla riga 8 posiziona l'elemento pivot nella giusta posizione: **Partition** fa esattamente quello che ci si aspetta, riordinando l'array rispetto all'elemento pivot.

□

Complessità di Partition. Il tempo di computazione ha sempre lo stesso ordine di grandezza per qualsiasi input di dimensione $n = r - p + 1$:

$$T_{\text{Partition}}(n) = \Theta(n).$$

5.8 Analisi di Complessità di QuickSort

Il tempo di esecuzione di QuickSort dipende, a parità di n , da quanto il partizionamento è bilanciato.

5.8.1 Caso peggiore

Analisi elementare. **Partition** restituisce sempre un sotto-array di dimensione $n - 1$ e uno di dimensione 0. In questo caso:

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n),$$

che è la somma aritmetica dei costi di **Partition**:

$$T(n) = \Theta(n^2). \quad (\text{caso peggiore})$$

Questo caso si verifica anche quando l'array è **completamente ordinato**! QuickSort è peggiore di InsertionSort in questo scenario.

Analisi formale. **Partition** restituisce sempre due sotto-array, uno di dimensione d e l'altro di dimensione $n - d - 1$, dove $0 \leq d \leq n - 1$:

$$T(n) = \max_{0 \leq d \leq n-1} [T(d) + T(n - d - 1)] + \Theta(n).$$

Si usa il metodo di sostituzione ipotizzando $T(n) \leq cn^2$:

$$T(n) \leq \max_{0 \leq d \leq n-1} [cd^2 + c(n-d-1)^2] + \Theta(n).$$

La quantità $d^2 + (n-d-1)^2$ ha il suo massimo quando $d = 0$ o $d = n-1$:

$$\max_{0 \leq d \leq n-1} [d^2 + (n-d-1)^2] = (n-1)^2 \leq n^2 - 2n + 1.$$

Quindi $T(n) \leq c(n^2 - 2n + 1) + \Theta(n) \leq cn^2$, confermando $T(n) = O(n^2)$.

5.8.2 Caso migliore

Partition restituisce sempre due sotto-array con dimensione $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil - 1$ rispettivamente. In questo caso (omettendo troncamenti e arrotondamenti):

$$T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n). \quad (\text{caso migliore})$$

5.8.3 Verso il caso medio

Caso con proporzionalità fissa. Si supponga che **Partition** restituisca sempre due sotto-array con dimensione $\frac{9}{10}n$ e $\frac{1}{10}n$ rispettivamente (caso sbilanciato):

$$T(n) = T\left(\frac{9}{10}n\right) + T\left(\frac{1}{10}n\right) + \Theta(n).$$

Sembrerebbe un caso simile al peggiore perché sbilanciato... ma analizzando l'albero di ricorsione si vede che ogni livello ha costo $\Theta(n)$ finché il ramo $T(\frac{1}{10}n)$ non raggiunge la dimensione 1 (dopo $\log_{10} n$ livelli). Dopo tale livello ogni livello costa ancora $\Theta(n)$. La ricorsione termina alla profondità $\log_{10/9} n = \Theta(\log n)$, quindi il costo totale è $\Theta(n \log n)$.

Analisi verso il caso medio generale. In generale non si può supporre che tutte le chiamate ricorsive siano sempre caso migliore, peggiore o con proporzionalità fissa. Assumendo che gli input siano distribuiti in modo equiprobabile, durante una computazione alcune ricorsioni divideranno bene, altre no.

Si definisce l'elemento pivot $A[q]$ trovato da **Partition** **buono** se $A[q]$ risiede tra il 25° e il 75° percentile:

$$A[q] \text{ buono} \Leftrightarrow |A[p : q-1]|, |A[q+1 : r]| \leq \frac{3}{4}|A[p : r]|.$$

Il 50% degli elementi sono buoni. Sia $X_q = I\{A[q] \text{ è un elemento buono}\}$:

$$\Pr\{X_q = 1\} = E[X_q] = \frac{1}{2}.$$

Dati due pivot $A[q]$ e $A[q']$ scelti casualmente uno dopo l'altro, il valore atteso del numero di

pivot buoni nella sequenza è:

$$E[X_q + X_{q'}] = E[X_q] + E[X_{q'}] = \frac{1}{2} + \frac{1}{2} = 1.$$

Quindi, mediamente ogni 2 scelte casuali del pivot si ottiene un valore buono. Due livelli consecutivi della ricorsione costano complessivamente $\Theta(n)$. Dopo una partizione buona, la dimensione del sotto-problema più grande è al più $\frac{3}{4}n$. Raggruppando i livelli due a due, la ricorrenza si comporta come:

$$T(n) = T\left(\frac{3}{4}n\right) + T\left(\frac{1}{4}n\right) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n).$$

5.9 QuickSort Randomizzato

Assumere che l'input abbia gli elementi distribuiti in modo casuale uniforme è troppo limitante. È preferibile mescolare l'input usando un generatore di numeri casuali, oppure scegliere il pivot casualmente prima di invocare **Partition**.

Algorithm 18: RandQuickSort(A, p, r)

Input: Array A , indici p, r

Output: $A[p : r]$ ordinato in loco

```

1 if  $p < r$  then
2    $q \leftarrow \text{RandPartition}(A, p, r);$ 
3   RandQuickSort( $A, p, q - 1$ );
4   RandQuickSort( $A, q + 1, r$ );
```

Algorithm 19: RandPartition(A, p, r)

Input: Array A , indici p, r

Output: q e A modificato

```

1  $i \leftarrow \text{RANDOM}(p, r);$ 
2 scambia  $A[r]$  con  $A[i]$ ;
3 return Partition( $A, p, r$ );
```

5.10 Analisi del Caso Medio di QuickSort

RandQuickSort garantisce una scelta uniforme del pivot ad ogni chiamata ricorsiva. Di seguito si presenta l'analisi formale della complessità nel caso medio seguendo [CLRS cap. 7].

Il costo reale dell'algoritmo è proporzionale al numero totale di confronti (istruzione 4) eseguiti dalla procedura **Partition**. Si deve quindi determinare il numero totale di confronti fatti da

Partition durante tutta una computazione di QuickSort.

Dato A , si rinominano tutti gli elementi come $z_1, z_2, \dots, z_n$ dove z_i è l' i -esimo elemento più piccolo di A . QuickSort confronta in **Partition** una coppia z_i, z_j **al più una volta** in tutta la sua esecuzione. Se uno dei due diventa pivot, può essere confrontato con l'altro; un elemento pivot non viene poi più confrontato con elementi appartenenti a chiamate ricorsive successive.

Si definisce la variabile casuale indicatrice:

$$X_{ij} = I\{z_i \text{ confrontato con } z_j\}.$$

Il numero totale di confronti è:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}.$$

Il numero totale atteso di confronti:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ confrontato con } z_j\}.$$

Calcolo di $\Pr\{z_i \text{ confrontato con } z_j\}$. Si consideri l'insieme $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$.

- Se **Partition** sceglie un elemento x tale che $z_i < x < z_j$, allora z_i e z_j andranno su due sotto-array distinti e **non** verranno mai confrontati.
- Se $x = z_i$ oppure $x = z_j$, allora z_i e z_j verranno confrontati **una volta**.

Gli elementi di Z_{ij} sono equi-probabili. La probabilità di scegliere un elemento particolare è quindi $\frac{1}{j-i+1}$ (quando gli elementi sono distinti):

$$\Pr\{z_i \text{ confrontato con } z_j\} = \Pr\{z_i \text{ è il primo pivot di } Z_{ij}\} + \Pr\{z_j \text{ è il primo pivot di } Z_{ij}\} = \frac{2}{j-i+1}.$$

Calcolo finale.

$$\begin{aligned} E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} \\ &= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \quad (\text{cambio variabile } k = j - i) \\ &\leq \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} = \sum_{i=1}^{n-1} O(\log n) = O(n \log n). \end{aligned}$$

Teorema 5.4 (Complessità media di QuickSort).

$$E[T(n)] = O(n \log n). \quad (\text{caso medio})$$

Caso	Input	Complessità
Peggior	Array ordinato (o inversamente ordinato)	$\Theta(n^2)$
Miglior	Pivot sempre mediano	$\Theta(n \log n)$
Medio	Input equiprobabili / pivot casuale	$\Theta(n \log n)$

Da ricordare all'esame: il caso peggiore di QuickSort è $\Theta(n^2)$ e si verifica su array *già ordinati* con il pivot fisso all'ultimo elemento. La versione randomizzata **RandQuickSort** rende tale caso molto improbabile e garantisce $O(n \log n)$ in media per qualsiasi input.

6. Statistiche d'Ordine e Selezione

Questo problema mostra un altro esempio di divide et impera con buona qualità nel caso medio: trovare il k -esimo elemento più piccolo di un array senza doverlo ordinare completamente.

6.1 Definizioni

Definizione 6.1 (Statistica d'ordine). Dato un insieme di n elementi, la i -esima **statistica d'ordine** è l' i -esimo elemento più piccolo.

Esempio 6.1. Dato l'insieme $\{3, 5, 9, 4, 6, 1\}$ di $n = 6$ elementi:

- I statistica d'ordine = 1 (elemento minimo)
- VI statistica d'ordine = 9 (elemento massimo)
- IV statistica d'ordine = 5

Definizione 6.2 (Mediana). Rappresenta un valore v tale che almeno il 50% degli elementi $\leq v$ e almeno il 50% $\geq v$.

- Se n è dispari, la mediana è la $\lfloor n/2 \rfloor + 1$ statistica d'ordine (unica).
- Se n è pari, le mediane possono essere due: la statistica d'ordine $\lfloor n/2 \rfloor$ (**mediana inferiore**) e quella $\lceil n/2 \rceil$ (**mediana superiore**).

Per convenzione, scrivendo *mediana* si intende sempre la **mediana inferiore**: la statistica d'ordine $\lfloor (n+1)/2 \rfloor$.

Esempio 6.2. Dato $\{3, 5, 9, 4, 6, 1\}$ con $n = 6$: mediana inferiore = III statistica d'ordine = 4; mediana superiore = IV statistica d'ordine = 5.

6.2 Problema della Selezione

Definizione 6.3 (Problema Selezione). **Input:** un insieme A di n numeri distinti e un intero $1 \leq i \leq n$.

Output: determinare l' i -esimo elemento più piccolo di A .

La selezione si può risolvere in tempo $\Theta(n \log n)$ ordinando gli elementi, ma non ha senso dover ordinare spostare gli elementi quando si vuole conoscere solo la i -esima statistica d'ordine. In questa sezione vediamo come è possibile risolvere il problema Selezione in tempo atteso $\Theta(n)$.

Nota

[CLRS] assume che i numeri siano distinti per comodità. Si può dimostrare in seguito che i risultati sono validi anche quando i numeri non sono distinti.

6.3 Minimo, Massimo e MinMax

Minimo (o massimo). Trovare il minimo o il massimo è facile: bastano $n - 1$ confronti, ed è ottimale perché ogni elemento tranne il minimo deve confrontarsi con almeno un altro elemento per stabilire che non è il minimo.

Algorithm 20: Minimum(A)

Input: Array $A[1 : n]$

Output: Valore minimo

```
1  $m \leftarrow A[1];$ 
2 for  $i \leftarrow 2$  to  $n$  do
3   if  $m > A[i]$  then
4      $m \leftarrow A[i];$ 
5 return  $m;$ 
```

$T_{\text{Minimum}}(n) = \Theta(n)$. L'algoritmo MAXIMUM(A) è analogo (basta cambiare $>$ con $<$).

Minimo e massimo contemporaneamente. Fondendo MINIMUM e MAXIMUM si risolvono entrambi con $2(n - 1)$ confronti. Si può fare meglio: $\lceil 3n/2 \rceil - 2$ **confronti**, confrontando gli elementi *a coppie* anziché singolarmente.

Algorithm 21: MinMax(A)

Input: Array $A[1 : n]$
Output: Valori minimo m e massimo M

```

1 if  $n$  è dispari then
2    $m \leftarrow A[1]; M \leftarrow A[1]; i \leftarrow 2;$ 
3 if  $A[1] < A[2]$  then
4    $m \leftarrow A[1]; M \leftarrow A[2];$ 
5 else
6    $M \leftarrow A[1]; m \leftarrow A[2];$ 
7  $i \leftarrow 3;$ 
8 while  $i \leq n - 1$  do
9   // Usa  $A[i]$  e  $A[i + 1]$  per aggiornare  $m$  e  $M$ 
10  if  $A[i] < A[i + 1]$  then
11    if  $A[i] < m$  then
12       $m \leftarrow A[i];$ 
13    if  $A[i + 1] > M$  then
14       $M \leftarrow A[i + 1];$ 
15  else
16    if  $A[i + 1] < m$  then
17       $m \leftarrow A[i + 1];$ 
18    if  $A[i] > M$  then
19       $M \leftarrow A[i];$ 
20   $i \leftarrow i + 2;$ 
21 return  $m, M;$ 

```

Nota

Ogni coppia di elementi richiede 3 confronti (1 tra loro + 1 con il minimo corrente + 1 con il massimo corrente). Con $\lfloor n/2 \rfloor$ coppie si ottengono $\lceil 3n/2 \rceil - 2$ confronti totali.

6.4 Idea Divide et Impera per la Selezione

L'idea divide et impera per risolvere il problema Selezione(A, i) è la seguente. Dato un numero casuale v di A , si suddivide A in 3 sottoinsiemi:

$$A_L = \{x \in A \mid x < v\}, \quad A_v = \{x \in A \mid x = v\}, \quad A_R = \{x \in A \mid x > v\}.$$

La ricerca dell' i -esimo elemento più piccolo può essere ristretta:

$$\text{Selezione}(A, i) = \begin{cases} \text{Selezione}(A_L, i) & \text{se } i \leq |A_L| \\ v & \text{se } |A_L| < i \leq |A_L| + |A_v| \\ \text{Selezione}(A_R, i - |A_L| - |A_v|) & \text{altrimenti} \end{cases}$$

Le tre liste si determinano con un numero di confronti $\Theta(n)$. Dato A , la dimensione del sotto-problema è $\max(|A_L|, |A_R|)$.

Esempio 6.3. Funzionamento divide et impera per $\text{Selezione}(A, \lfloor n/2 \rfloor)$ (problema della mediana) con $A = \{3, 5, 9, 4, 6, 1\}$, $n = 6$, $i = 3$:

1. Valore casuale $v = 3$: $A_L = \{1\}$, $A_v = \{3\}$, $A_R = \{5, 9, 4, 6\}$. Vale $|A_L| = 1 < i = 3 \leq |A_L| + |A_v| = 2$? No. Si deve risolvere $\text{Selezione}(A_R, 3 - 1 - 1) = \text{Selezione}(\{5, 9, 4, 6\}, 1)$.
2. Valore casuale $v = 4$: $A_L = \{\}$, $A_v = \{4\}$, $A_R = \{5, 9, 6\}$. Vale $|A_L| = 0 < i = 1 \leq |A_L| + |A_v| = 1$? Sì. La mediana è soluzione $= 4$.

6.5 RandSelect

Questa implementazione riusa $\text{RandPartition}(A, p, r)$ vista per QuickSort. Versione di [CLRS] che assume tutti valori distinti.

Algorithm 22: RandSelect(A, p, r, i)

Input: Array A , indici p, r , rango i
Output: i -esimo elemento più piccolo di $A[p : r]$

```

1 if  $p = r$  then
2   return  $A[p]$ ;
3  $q \leftarrow \text{RandPartition}(A, p, r)$ ;
4  $k \leftarrow q - p + 1$ ;           // Rango del pivot nel sotto-array corrente
5 if  $i = k$  then
6   return  $A[q]$ ;               // Il pivot è la risposta
7 if  $i < k$  then
8   return RandSelect( $A, p, q - 1, i$ );
9 else
10  return RandSelect( $A, q + 1, r, i - k$ );
```

Nota

RandSelect ha il vantaggio di non richiedere memoria ausiliaria, ma ha lo svantaggio che altera l'ordine originale dell'array di input.

6.6 Analisi della Complessità di RandSelect

6.6.1 Caso peggiore

Il pivot è sempre l'elemento massimo $\Rightarrow$ la dimensione scende di 1 a ogni passo:

$$T(n) = T(n-1) + \Theta(n) \Rightarrow T(n) = \Theta(n^2).$$

6.6.2 Caso medio

Si definisce $A[q]$ **buono** se risiede tra il 25° e il 75° percentile, i.e., $|A_L|, |A_R| \leq \frac{3}{4}|A|$. Il 50% degli elementi sono buoni.

Sia $X_v = I\{v \text{ è un elemento buono}\}$:

$$\Pr\{X_v = 1\} = E[X_v] = \frac{1}{2}.$$

Dati due pivot scelti casualmente uno dopo l'altro, il valore atteso di pivot buoni è $E[X_v + X_{v'}] = 1$. Quindi mediamente ogni 2 scelte casuali di v si ottiene un valore buono.

Sia $s(n)$ il costo di una chiamata a **RandPartition** e $w(n)$ il costo totale delle chiamate a **RandPartition** fino a ottenere un valore buono per $A[q]$; allora $E[w(n)] = \Theta(n)$.

La complessità media soddisfa:

$$E[T(n)] = E\left[T\left(\frac{3}{4}n\right)\right] + \Theta(n).$$

Per sostituzione si ottiene $E[T(n)] = \Theta(n)$. Inoltre $E[T(n)] = \Omega(n)$ perché almeno una partizione completa va sempre eseguita.

Teorema 6.1 (Complessità media di RandSelect).

$$E[T(n)] = \Theta(n).$$

Da ricordare all'esame: **RandSelect** risolve il problema della selezione (e quindi della mediana) in tempo atteso $\Theta(n)$, molto meglio del $\Theta(n \log n)$ ottenuto ordinando l'array. Il caso peggiore rimane $\Theta(n^2)$, ma si verifica con probabilità trascurabile grazie alla scelta casuale del pivot.

Approccio	Complessità	Note
Ordinamento + accesso	$\Theta(n \log n)$	Troppo costoso
RandSelect caso peggiore	$\Theta(n^2)$	Molto improbabile
RandSelect caso medio	$\Theta(n)$	Garantito in attesa

7. HeapSort

HeapSort è l'ultimo algoritmo di ordinamento per confronti che studiamo. Combina il meglio di InsertionSort (ordina **sul posto**) e di MergeSort (complessità $O(n \lg n)$), ma introduce una tecnica nuova: si basa su una **struttura dati** dedicata, lo *heap*, che trova applicazione anche nelle code di priorità.

7.1 Idea Generale

HeapSort:

- ordina **sul posto** come InsertionSort;
- ha complessità $O(n \lg n)$ nel caso peggiore, come MergeSort;
- nel caso medio è **meno veloce** di QuickSort, perché le costanti nascoste in $O(n \lg n)$ sono più piccole in QuickSort che in HeapSort;
- è basato sulla struttura dati **heap**, usata anche in altri algoritmi importanti come le code di priorità.

7.2 Struttura Dati Heap

Definizione 7.1 (Heap binario). Uno **heap** è un array che rappresenta un albero binario *quasi completo*:

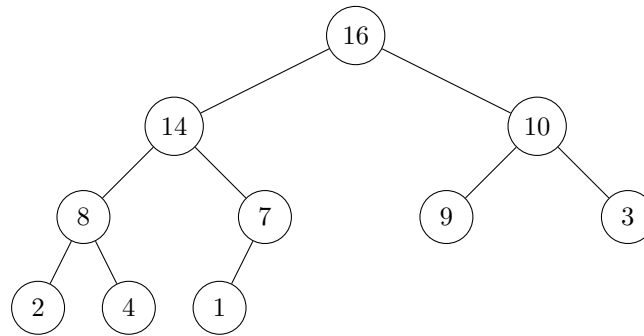
- ogni nodo dell'albero corrisponde a un elemento dell'array;
- tutti i livelli devono essere completamente riempiti tranne eventualmente l'ultimo, che può essere parziale ma sempre riempito da sinistra verso destra;
- se A è uno heap: $A.length$ è il numero di elementi dell'array, $A.heapSize$ è il numero di nodi dell'albero rappresentati ($0 \leq A.heapSize \leq A.length$);
- la radice dell'albero è in $A[1]$.

Nota

Albero binario completo vs. quasi completo:

- **Completo:** ciascun nodo interno ha esattamente due figli; le foglie sono tutte allo stesso livello.
- **Quasi completo:** tutti i livelli riempiti tranne eventualmente l'ultimo, sempre riempito da sinistra. (Se si numerano i nodi per livelli, la numerazione non ha “buchi”).

Esempio 7.1 (Rappresentazione Heap: array e albero). Dato l'array $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$, la corrispondente struttura ad albero è:



Le posizioni dell'array corrispondono ai nodi per livelli (BFS):

16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10

Regola di navigazione (indice 1-based):

$$\text{left}(i) = 2i, \quad \text{right}(i) = 2i + 1, \quad \text{parent}(i) = \lfloor i/2 \rfloor.$$

Il nodo $i = 2$ (valore 14) ha figlio sinistro in posizione 4 (valore 8) e figlio destro in posizione 5 (valore 7).

7.2.1 Navigazione dell'albero tramite indici

Dato il nodo in posizione i , genitore e figli si calcolano con:

$$\text{PARENT}(i) = \lfloor i/2 \rfloor, \quad \text{LEFT}(i) = 2i, \quad \text{RIGHT}(i) = 2i + 1.$$

Nota

Moltiplicare/dividere per 2 richiede solo di spostare di 1 bit a sinistra/destra: operazione estremamente efficiente a livello hardware.

7.2.2 Tipi di heap e proprietà

Definizione 7.2 (Max-heap e min-heap). • **Max-heap:** $A[\text{PARENT}(i)] \geq A[i]$ per ogni $i \neq 1$ (il genitore è sempre $\geq$ dei figli).

- **Min-heap:** $A[\text{PARENT}(i)] \leq A[i]$ per ogni $i \neq 1$ (il genitore è sempre $\leq$ dei figli).

HEAPSORT usa i **max-heap**; i min-heap sono usati per realizzare le code di priorità.

7.2.3 Altezza dello heap

Definizione 7.3 (Altezza di un nodo). L'**altezza** di un nodo è il numero di archi del cammino semplice più lungo che parte dal nodo e termina in una foglia (numero di livelli sottostanti al livello del nodo). L'**altezza dello heap** è l'altezza della radice.

Teorema 7.1 (Altezza di un heap di n nodi). *Un albero binario quasi completo di n nodi ha altezza $h = \lfloor \lg n \rfloor$.*

Dimostrazione. Vale $2^h \leq n < 2^{h+1}$. Applicando $\lg$: $h \leq \lg n < h + 1$, quindi $h = \lfloor \lg n \rfloor$. $\square$

Esempio 7.2 (Altezza con $n = 10$). $h = \lfloor \lg 10 \rfloor = \lfloor 3,32 \rfloor = 3$.

Nota

Quasi tutte le operazioni fondamentali sugli heap sono eseguite in un tempo proporzionale all'altezza dell'albero, quindi $O(\lg n)$.

7.2.4 Operazioni fondamentali sullo heap

Sono 6 le operazioni fondamentali:

1. BUILDMAXHEAP — genera un max-heap da un array generico in tempo $O(n)$;
2. MAXHEAPIFY — ripristina la proprietà di max-heap quando a un elemento viene diminuito il valore; tempo $O(\lg n)$ (necessaria a BUILDMAXHEAP);
3. MAXHEAPINSERT, HEAPEXTRACTMAX, HEAPINCREASEKEY e HEAPMAX — consentono di usare uno heap per realizzare una coda di priorità; complessità $O(\lg n)$ tranne HEAPMAX che è $O(1)$.

7.3 MaxHeapify

MAXHEAPIFY è l'operazione di base: dato un nodo i il cui valore è stato diminuito, “fa scendere” $A[i]$ fino alla posizione corretta ripristinando la proprietà di max-heap nel sottoalbero radicato in i .

Precondizione. I sottoalberi radicati in $\text{LEFT}(i)$ e $\text{RIGHT}(i)$ sono già max-heap; la proprietà può essere violata solo nel nodo i .

Algorithm 23: MAXHEAPIFY(A, i)

Input: Array A , indice i ; sottoalberi di $\text{LEFT}(i)$ e $\text{RIGHT}(i)$ sono max-heap

Output: A con la proprietà di max-heap ristabilita nel sottoalbero radicato in i

```

1  $l \leftarrow \text{LEFT}(i)$ ;
2  $max \leftarrow (l \leq A.\text{heapSize} \text{ and } A[l] > A[i]) ? l : i$ ;           // Figlio sx vs. radice
3  $r \leftarrow \text{RIGHT}(i)$ ;
4 if  $r \leq A.\text{heapSize}$  and  $A[r] > A[max]$  then
5    $max \leftarrow r$ ;
6 if  $max \neq i$  then
7   scambia  $A[i]$  con  $A[max]$ ;
8   MAXHEAPIFY( $A, max$ );                                           // Ricorsione sul figlio
```

7.3.1 Analisi di complessità di MaxHeapify

Sia n il numero di nodi del sottoalbero da sistemare.

Ricorrenza. MAXHEAPIFY esegue $\Theta(1)$ operazioni più una sola chiamata ricorsiva sul sottoalbero più grande:

$$T(n) \leq T\left(\frac{2}{3}n\right) + \Theta(1).$$

Giustificazione del fattore $\frac{2}{3}$. Nel caso peggiore l'ultimo livello dell'albero è riempito esattamente a metà: il sottoalbero sinistro ha un livello in più del destro. Se h è l'altezza della radice, il sottoalbero sinistro (completo) ha $2^h - 1$ nodi, quello destro (completo) $2^{h-1} - 1$ nodi e il totale è $n = 3 \cdot 2^{h-1} - 1$. La proporzione del sottoalbero sinistro è:

$$\frac{2^h - 1}{3 \cdot 2^{h-1} - 1} \leq \frac{2^h}{3 \cdot 2^{h-1}} = \frac{2}{3}.$$

Soluzione. Applicando il Teorema dell'Esperto (caso 2), la ricorrenza $T(n) \leq T(\alpha n) + \Theta(1)$ ha soluzione $T(n) = O(\lg n)$ per qualsiasi $\alpha < 1$:

$$T_{\text{MAXHEAPIFY}}(n) = O(\lg n) = O(h).$$

Caso peggiore concreto: un elemento posto alla radice deve scendere fino a una foglia, eseguendo $\lfloor \lg n \rfloor$ chiamate ricorsive. Quindi la complessità è anche $\Omega(\lg n)$, cioè $\Theta(\lg n)$ nel caso peggiore.

7.4 BuildMaxHeap

Per trasformare un array generico in un max-heap si osserva che tutti gli elementi in $A[\lfloor n/2 \rfloor + 1 : n]$ sono **foglie**, quindi già heap di un elemento. Basta applicare MAXHEAPIFY a tutti i nodi interni, risalendo dall'ultimo verso la radice.

Algorithm 24: BUILDMAXHEAP(A)

Input: Array A di n elementi

Output: A riordinato come max-heap

```
1  $A.heapSize \leftarrow A.length;$ 
2 for  $i \leftarrow \lfloor A.length/2 \rfloor$  to 1 do
3   MAXHEAPIFY( $A, i$ );
```

7.4.1 Correttezza di BuildMaxHeap

Definizione 7.4 (Invariante di BuildMaxHeap). All'inizio di ogni iterazione del ciclo `for`, ogni nodo di indice $i + 1, \dots, n$ è la radice di un max-heap.

Dimostrazione. 1. **Inizializzazione:** prima del I ciclo, $i = \lfloor n/2 \rfloor$. Tutti i nodi successivi sono foglie, dunque radici di max-heap di un solo elemento. ✓

2. **Conservazione:** i figli del nodo i hanno indice maggiore di i , quindi per l'invariante sono radici di max-heap. `MAXHEAPIFY(A, i)` ripristina la proprietà nel sottoalbero radicato in i , garantendo che all'iterazione $i - 1$ i nodi $i, \dots, n$ siano tutti radici di max-heap. ✓

3. **Conclusioni:** alla fine del ciclo $i = 0$, quindi il nodo 1 è la radice di un max-heap che comprende tutti i nodi da 1 a n : A è un max-heap. ✓

□

7.4.2 Complessità di BuildMaxHeap

Una stima immediata è $O(n \lg n)$ (si chiama `MAXHEAPIFY` $n/2$ volte, ciascuna $O(\lg n)$). Ma si può dimostrare un limite più stretto: $O(n)$.

Per ogni altezza h' con $0 \leq h' \leq \lfloor \lg n \rfloor$, il numero di nodi ad altezza h' è al più $n_{h'} = \lceil n/2^{h'+1} \rceil$.

Dimostrazione per induzione. Le foglie ($h' = 0$) sono $\lceil n/2 \rceil = \lceil n/2^{0+1} \rceil$. Se $n_{h'-1} \leq \lceil n/2^{h'} \rceil$, allora $n_{h'} = \lceil n_{h'-1}/2 \rceil \leq \lceil n/2^{h'+1} \rceil$. □

`MAXHEAPIFY` usa tempo $O(h')$ per ciascun nodo ad altezza h' . Il costo totale è:

$$\sum_{h'=1}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h'+1}} \right\rceil O(h') \leq O\left(n \sum_{h'=0}^{\lfloor \lg n \rfloor} \frac{h'}{2^{h'}}\right) = O\left(n \cdot \frac{1/2}{(1 - 1/2)^2}\right) = O(n).$$

Nota

Si usa la somma della serie $\sum_{h'=0}^{\infty} h' x^{h'} = x/(1-x)^2$ con $x = 1/2$, ottenendo il valore finito 2.

Teorema 7.2 (Complessità di BuildMaxHeap).

$$T_{\text{BUILDMAXHEAP}}(n) = O(n).$$

7.5 Algoritmo HeapSort

Una volta costruito il max-heap, l'elemento massimo si trova sempre in $A[1]$. L'idea è estrarlo iterativamente: si scambia la radice con l'ultimo elemento, si riduce l'*heapSize* di 1 e si ripristina la proprietà con MAXHEAPIFY. Al termine l'array è ordinato in modo crescente.

Algorithm 25: HEAPSORT(A)

Input: Array A di n elementi

Output: A ordinato in loco

```

1 BUILDMAXHEAP( $A$ );
2 for  $i \leftarrow A.length$  to 2 do
3   |   scambia  $A[1]$  con  $A[i]$ ;
4   |    $A.heapSize \leftarrow A.heapSize - 1$ ;
5   |   MAXHEAPIFY( $A, 1$ );
```

7.5.1 Correttezza di HeapSort

Definizione 7.5 (Invariante di HeapSort). All'inizio di ogni iterazione del ciclo **for**, il sottoarray $A[1 : i]$ è un max-heap contenente gli i elementi più piccoli di $A[1 : n]$, e il sottoarray $A[i + 1 : n]$ contiene gli $n - i$ elementi più grandi di $A[1 : n]$, ordinati in modo crescente.

7.5.2 Complessità di HeapSort

- BUILDMAXHEAP: $O(n)$.
- Ciclo **for**: $n - 1$ chiamate a MAXHEAPIFY, ciascuna $O(\lg n)$.

$$T_{\text{HEAPSORT}}(n) = O(n) + (n - 1) O(\lg n) = O(n \lg n).$$

Caso	Input	Complessità
Peggior	qualsiasi	$O(n \lg n)$
Medio	qualsiasi	$O(n \lg n)$
Migliore	qualsiasi	$O(n \lg n)$

Da ricordare all'esame: HeapSort è $O(n \lg n)$ in tutti i casi e ordina sul posto. Nel caso medio è più lento di QuickSort (costanti peggiori), ma non soffre del caso peggiore $\Theta(n^2)$ di QuickSort non randomizzato.

7.6 Code di Priorità

Lo heap non serve solo per ordinare: è la struttura dati ideale per realizzare una **coda di priorità**, che consente di estrarre sempre l'elemento con priorità massima (o minima) in modo efficiente.

Definizione 7.6 (Coda di max-priorità). Una **coda di max-priorità** è una struttura dati che gestisce un insieme S di elementi, ciascuno associato a un valore detto *chiave* o *priorità*, in modo da estrarre l'elemento di priorità massima o modificare la priorità di un elemento nel modo più efficiente possibile. Analogamente si definisce la coda di **min-priorità**.

7.6.1 Operazioni fondamentali

Se x è un elemento, $x.key$ è la sua chiave/priorità. Le operazioni fondamentali sono:

- **Insert**(S, x): inserisce x in S .
- **Max**(S): restituisce l'elemento di S con priorità massima.
- **ExtractMax**(S): rimuove e restituisce l'elemento con priorità massima.
- **IncreaseKey**(S, x, k): aumenta la chiave di x a k (con $k \geq x.key$).

Nota

Se si vuole **diminuire** il valore di un elemento, è sufficiente farlo direttamente e poi invocare MAXHEAPIFY per ripristinare la proprietà di max-heap.

7.6.2 Implementazione delle operazioni

Algorithm 26: MAX(S)

Input: S coda di max-priorità

Output: Elemento con priorità massima

1 **return** $S[1]$;

Complessità: $O(1)$.

Algorithm 27: EXTRACTMAX(S)

Input: S coda di max-priorità

Output: Elemento con priorità massima, rimosso da S

1 **if** $S.heapSize < 1$ **then**

2 **return** "Coda vuota!";

3 $max \leftarrow S[1]$;

4 $S[1] \leftarrow S[S.heapSize]$;

5 $S.heapSize \leftarrow S.heapSize - 1$;

6 MAXHEAPIFY($S, 1$);

7 **return** max ;

Complessità: $O(\lg n)$.

Algorithm 28: INCREASEKEY(S, i, key)

Input: S coda di max-priorità, posizione i , nuovo valore key

Output: Priorità $S[i].key$ aggiornata in S

```

1 if  $key < S[i].key$  then
2   return "Errore: nuova priorità minore dell'attuale";
3  $S[i].key \leftarrow key$ ;
4 while  $i > 1$  and  $S[PARENT(i)].key < S[i].key$  do
5   scambia  $S[i]$  con  $S[PARENT(i)]$ ;
6    $i \leftarrow PARENT(i)$ ;
```

Se la nuova chiave rompe la proprietà di max-heap, l'elemento viene scambiato con il genitore fino a trovare la posizione corretta (risalita verso la radice).

Complessità: $O(\lg n)$ (al più si sale dalla foglia più profonda alla radice).

Algorithm 29: INSERT(S, x)

Input: S coda di max-priorità, nuovo elemento x

Output: S aggiornato con x

```

1  $S.heapSize \leftarrow S.heapSize + 1$ ;
2  $key \leftarrow x.key$ ;
3  $x.key \leftarrow -\infty$ ;                                // Inserisce in fondo con priorità minima
4  $S[S.heapSize] \leftarrow x$ ;
5 INCREASEKEY( $S, S.heapSize, key$ );                      // Risale alla posizione corretta
```

Complessità: $O(\lg n)$ (l'elemento potrebbe risalire dalla foglia più profonda alla radice).

Operazione	Complessità
MAX	$O(1)$
EXTRACTMAX	$O(\lg n)$
INCREASEKEY	$O(\lg n)$
INSERT	$O(\lg n)$
BUILDMAXHEAP	$O(n)$
MAXHEAPIFY	$O(\lg n)$

8. Ordinamento in Tempo Lineare

Fino ad ora abbiamo considerato algoritmi di ordinamento *basati sul confronto* (InsertionSort, MergeSort, HeapSort, QuickSort). In questa sezione si dimostrerà che **tutti** questi algoritmi hanno complessità $\Omega(n \lg n)$ nel caso peggiore. Si presenteranno poi due algoritmi che *non* si basano sul confronto: COUNTINGSORT e BUCKETSORT, che raggiungono la complessità lineare $\Theta(n)$ sotto opportune ipotesi sull'input.

8.1 Limiti Inferiori per l'Ordinamento

8.1.1 Alberi di Decisione

Gli algoritmi di ordinamento per confronto eseguono test del tipo $x < y$, $x == y$, $x > y$. Ogni algoritmo di questo tipo può essere rappresentato tramite un **albero di decisione**: un modello che cattura tutti i possibili percorsi di confronto per un dato input.

Definizione 8.1 (Albero di decisione per algoritmi di ordinamento). Un *albero di decisione* è un albero binario *pieno* che rappresenta i confronti fra gli elementi di un input di dimensione n di un algoritmo di ordinamento. Ogni nodo interno considera una coppia di elementi (a, b) :

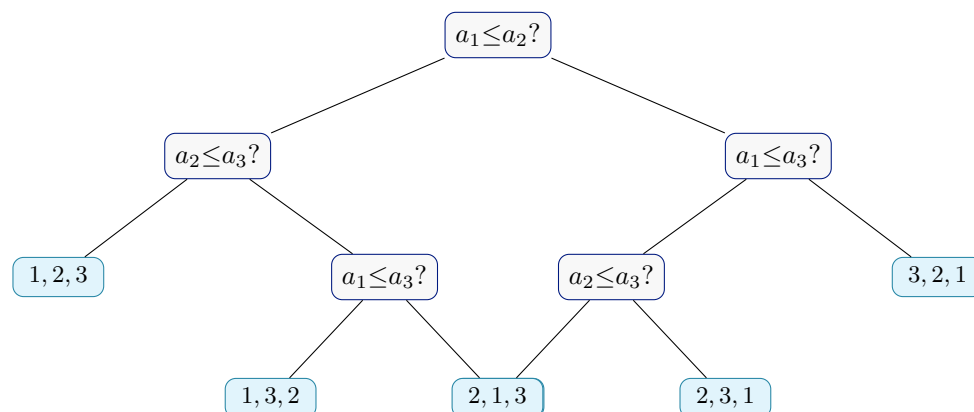
- il figlio **sinistro** rappresenta il caso $a \leq b$;
- il figlio **destro** rappresenta il caso $a > b$.

In un albero binario pieno ogni nodo interno ha esattamente 0 o 2 figli.

Nota

Ogni algoritmo di ordinamento per confronto deve essere in grado di produrre ogni permutazione di n elementi distinti. Con n elementi distinti esistono $n!$ permutazioni possibili, ognuna delle quali deve essere rappresentata come foglia dell'albero di decisione.

Esempio 8.1 (Albero di decisione per InsertionSort su 3 elementi). Con input $A = [6, 8, 5]$, INSERTIONSORT esegue i confronti seguendo un percorso specifico nell'albero. L'albero per $n = 3$ ha $3! = 6$ foglie (una per ogni permutazione) e altezza almeno $\lceil \lg 6 \rceil = 3$.



Il percorso evidenziato per $[6, 8, 5]$: prima $6 \leq 8$ (sinistra), poi $8 > 5$ (destra), poi $6 > 5$ (destra) $\Rightarrow$ permutazione $[5, 6, 8]$.

8.1.2 Limite Inferiore per il Caso Peggior

Teorema 8.1 (Limite inferiore $\Omega(n \lg n)$). *Qualsiasi algoritmo di ordinamento basato sul confronto richiede nel caso peggiore $\Omega(n \lg n)$ confronti.*

Dimostrazione. Si fissi un algoritmo di ordinamento per confronto e il suo albero di decisione per un dato n .

- La lunghezza del cammino più lungo dalla radice a una foglia è l'altezza h dell'albero, e corrisponde al numero di confronti nel caso peggiore.
- L'albero deve avere almeno $n!$ foglie (una per ogni permutazione): $n! \leq l$.
- Un albero binario di altezza h ha al più 2^h foglie: $l \leq 2^h$.
- Combinando: $n! \leq 2^h$, quindi $h \geq \lg(n!)$.
- Per l'approssimazione di Stirling ($n! \approx \sqrt{2\pi n}(n/e)^n$):

$$h \geq \lg(n!) = \Omega(n \lg n)$$

□

Il limite $\Omega(n \lg n)$ vale per **tutti** gli algoritmi basati sul confronto: InsertionSort, MergeSort, HeapSort, QuickSort, ecc. Gli algoritmi COUNTINGSORT e BUCKETSORT aggirano questo limite perché **non** usano confronti diretti tra elementi, ma sfruttano proprietà numeriche dell'input.

8.2 Counting Sort

8.2.1 Idea Generale

COUNTINGSORT è un algoritmo di ordinamento non basato sul confronto. L'input deve essere un array di interi nell'intervallo $[0 : k]$ per un intero fisso k . L'idea è: dato un elemento x dell'input, **conta** quanti elementi minori di x ci sono, e usa tale numero per collocare x nella posizione finale corretta nell'array di output. Per contare, si usa un array ausiliario $C[0 : k]$: ogni volta che si incontra il valore x nell'input, si incrementa $C[x]$ di 1. La complessità è $\Theta(k + n)$; se $k = O(n)$, diventa $\Theta(n)$.

8.2.2 Algoritmo

Algorithm 30: COUNTINGSORT(A, k)**Input:** $A[1 : n]$ array di interi in $[0, k]$, k valore massimo**Output:** $B[1 : n]$, array ordinato

```

1  $B[1 : n] \leftarrow$  nuovo array;
2  $C[0 : k] \leftarrow$  nuovo array inizializzato a 0;
3 for  $j = 1$  to  $A.length$  do
4    $C[A[j]]++$ ; //  $C[i] = \#$  volte che  $i$  compare in  $A$ 
5 for  $i = 1$  to  $k$  do
6    $C[i] = C[i] + C[i - 1]$ ; //  $C[i] = \#$  elementi  $\leq i$  in  $A$ 
7 for  $j = A.length$  to 1 do
8    $B[C[A[j]]] = A[j]$ ;
9    $C[A[j]]-$ ; // se ci sono più copie di  $A[j]$ , la prossima va in posizione
    precedente
10 return  $B$ 

```

8.2.3 Esempio Visivo

Esempio 8.2 (Esecuzione di CountingSort, $n = 8$, $k = 5$). Sia $A = [2, 5, 3, 0, 2, 3, 0, 3]$.

Array iniziale A .

indice	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

Passo 1 — Conteggio occorrenze (primo ciclo for). Per ogni j , si incrementa $C[A[j]]$:

indice i	0	1	2	3	4	5
$C[i]$ (occorrenze)	2	0	2	3	0	1

Il valore 0 compare 2 volte, 2 compare 2 volte, 3 compare 3 volte, 5 una volta.

Passo 2 — Prefissi cumulativi (secondo ciclo for). $C[i] += C[i - 1]$ per $i = 1, \dots, 5$:

indice i	0	1	2	3	4	5
$C[i]$ (elementi $\leq i$)	2	2	4	7	7	8

Lettura: ci sono 4 elementi con valore ≤ 2 , quindi l'ultimo 2 andrà in posizione 4. Ci sono 7 elementi con valore ≤ 3 , quindi l'ultimo 3 andrà in posizione 7.

Passo 3 — Costruzione di B (terzo ciclo for, da $j = 8$ a $j = 1$).

Step	$B[1]$	$B[2]$	$B[3]$	$B[4]$	$B[5]$	$B[6]$	$B[7]$	$B[8]$	Operazione
Inizio	—	—	—	—	—	—	—	—	
$j = 8: A[8] = 3$	—	—	—	—	—	—	3	—	$B[C[3]]=B[7]=3; C[3] \rightarrow 6$
$j = 7: A[7] = 0$	—	0	—	—	—	—	3	—	$B[C[0]]=B[2]=0; C[0] \rightarrow 1$
$j = 6: A[6] = 3$	—	0	—	—	—	3	3	—	$B[C[3]]=B[6]=3; C[3] \rightarrow 5$
$j = 5: A[5] = 2$	—	0	—	2	—	3	3	—	$B[C[2]]=B[4]=2; C[2] \rightarrow 3$
$j = 4: A[4] = 0$	0	0	—	2	—	3	3	—	$B[C[0]]=B[1]=0; C[0] \rightarrow 0$
$j = 3: A[3] = 3$	0	0	—	2	3	3	3	—	$B[C[3]]=B[5]=3; C[3] \rightarrow 4$
$j = 2: A[2] = 5$	0	0	—	2	3	3	3	5	$B[C[5]]=B[8]=5; C[5] \rightarrow 7$
$j = 1: A[1] = 2$	0	0	2	2	3	3	3	5	$B[C[2]]=B[3]=2; C[2] \rightarrow 2$
Finale	0	0	2	2	3	3	3	5	

Il risultato $B = [0, 0, 2, 2, 3, 3, 3, 5]$ è correttamente ordinato. I due valori 3 (originariamente in posizione 3 e 6) e i due valori 2 (in posizione 1 e 5) mantengono l'ordine relativo: **stabilità garantita**.

8.2.4 Complessità

- Ciclo **for** righe 3–4: $\Theta(n)$
- Ciclo **for** righe 5–6: $\Theta(k)$
- Ciclo **for** righe 7–9: $\Theta(n)$

$$T(n) = \Theta(k + n)$$

Se $k = O(n)$, allora $T(n) = \Theta(n)$.

Il tempo è inferiore al limite $\Omega(n \lg n)$ perché COUNTINGSORT **non** si basa sul confronto. Il prerequisito fondamentale è che i valori in input siano interi in un intervallo $[0, k]$ noto a priori.

8.2.5 Stabilità

Nota

COUNTINGSORT è un algoritmo **stabile**: gli elementi con lo stesso valore mantengono la loro posizione relativa. Questo è garantito dal terzo ciclo, che scorre A da *destra a sinistra*: il primo elemento con valore v incontrato (partendo da destra) occupa la posizione più alta disponibile, il successivo quella immediatamente precedente, e così via. La stabilità è fondamentale quando COUNTINGSORT è usato come subroutine di RADIXSORT.

8.3 Bucket Sort

8.3.1 Idea Generale

BUCKETSORT ordina valori decimali positivi assumendo che siano *uniformemente distribuiti* in $[0, 1)$. L'idea è:

1. Dividere $[0, 1)$ in n sotto-intervalli di uguale dimensione (*bucket*).
2. Distribuire gli n elementi nei bucket.
3. Grazie all'ipotesi di distribuzione uniforme, ogni bucket contiene in media ≈ 1 elemento.
4. Ordinare ogni bucket con INSERTIONSORT (efficiente su input piccoli).
5. Concatenare i bucket ordinati da $B[0]$ a $B[n - 1]$.

8.3.2 Algoritmo

Algorithm 31: BUCKETSORT(A)

Input: $A[1 : n]$, valori in $[0, 1)$, distribuzione uniforme

Output: $C[1 : n]$, array ordinato

```

1  $n = A.length$ ;
2  $B[0 : n - 1] \leftarrow$  nuovo array di liste vuote;
3  $C[1 : n] \leftarrow$  nuovo array vuoto;
4 for  $i = 1$  to  $n$  do
5    $B[\lfloor n \cdot A[i] \rfloor].add(A[i])$ ;           // inserisce  $A[i]$  nel bucket corrispondente
6 for  $i = 0$  to  $n - 1$  do
7   INSERTIONSORT( $B[i]$ );
8   Aggiungi a  $C$  tutti i valori ordinati di  $B[i]$ ;
9 return  $C$ 
```

8.3.3 Esempio Visivo

Esempio 8.3 (Esecuzione di BucketSort, $n = 10$). Sia:

$$A = [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68]$$

Passo 1 — Distribuzione nei bucket. La regola $\lfloor 10 \cdot A[i] \rfloor$ assegna ogni elemento al bucket corrispondente alla sua prima cifra decimale:

Bucket i	Intervallo	Elementi inseriti	Dopo InsertionSort
$B[0]$	$[0.0, 0.1)$	—	—
$B[1]$	$[0.1, 0.2)$	0.17, 0.12	0.12, 0.17
$B[2]$	$[0.2, 0.3)$	0.26, 0.21, 0.23	0.21, 0.23, 0.26
$B[3]$	$[0.3, 0.4)$	0.39	0.39
$B[4]$	$[0.4, 0.5)$	—	—
$B[5]$	$[0.5, 0.6)$	—	—
$B[6]$	$[0.6, 0.7)$	0.68	0.68
$B[7]$	$[0.7, 0.8)$	0.78, 0.72	0.72, 0.78
$B[8]$	$[0.8, 0.9)$	—	—
$B[9]$	$[0.9, 1.0)$	0.94	0.94

Passo 2 — Concatenazione.

$$C = [0.12, 0.17, 0.21, 0.23, 0.26, 0.39, 0.68, 0.72, 0.78, 0.94]$$

Nota

Il bucket $B[2]$ contiene 3 elementi, mentre la maggior parte ne contiene 0 o 1. Con distribuzione uniforme, questo è atteso: mediamente ogni bucket ha $n/n = 1$ elemento. Il costo medio di INSERTIONSORT su ogni bucket è $O(1)$, da cui la complessità totale $\Theta(n)$.

8.3.4 Correttezza

Dimostrazione. Si considerino due elementi $A[i]$ e $A[j]$ con $A[i] \leq A[j]$. Moltiplicando per $n > 0$:

$$\lfloor n \cdot A[i] \rfloor \leq \lfloor n \cdot A[j] \rfloor$$

Quindi $A[i]$ viene messo nello stesso bucket di $A[j]$ o in uno precedente.

- **Stesso bucket:** INSERTIONSORT li ordina correttamente.
- **Bucket diversi:** la concatenazione in ordine crescente degli indici li posiziona nell'ordine corretto nell'output.

□

8.3.5 Complessità: Analisi del Caso Medio

Sia n_i la variabile casuale che rappresenta il numero di elementi nel bucket i . INSERTIONSORT sul bucket i ha complessità $O(n_i^2)$, quindi:

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$$

Calcolo di $E[n_i^2]$. Assumendo i valori distribuiti uniformemente in $[0, 1)$, si definisce X_{ij} come la variabile casuale indicatrice $\mathbf{I}\{A[j] \text{ entra nel bucket } i\}$, con $\Pr\{X_{ij} = 1\} = E[X_{ij}] = 1/n$. Il numero di elementi nel bucket i è $n_i = \sum_{j=1}^n X_{ij}$, quindi:

$$E[n_i^2] = \sum_{j=1}^n E[X_{ij}^2] + \sum_{j=1}^n \sum_{\substack{k=1 \\ k \neq j}}^n E[X_{ij} X_{ik}]$$

- **Primo termine:** $E[X_{ij}^2] = 1^2 \cdot \frac{1}{n} + 0^2 \cdot (1 - \frac{1}{n}) = \frac{1}{n}$
- **Secondo termine:** X_{ij} e X_{ik} indipendenti (per $k \neq j$): $E[X_{ij} X_{ik}] = \frac{1}{n} \cdot \frac{1}{n} = \frac{1}{n^2}$

Sostituendo:

$$E[n_i^2] = n \cdot \frac{1}{n} + n(n-1) \cdot \frac{1}{n^2} = 1 + \frac{n-1}{n} = 2 - \frac{1}{n}$$

Il contributo medio di ciascun bucket è $O\left(2 - \frac{1}{n}\right) = O(1)$.

Complessità media totale.

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2]) = \Theta(n) + n \cdot O(1) = \Theta(n)$$

Se l'input *non* ha distribuzione uniforme, ma la sommatoria $\sum_{i=0}^{n-1} n_i^2$ è comunque lineare in n , BUCKETSORT mantiene complessità $\Theta(n)$. Questa condizione è soddisfatta, ad esempio, quando i valori sono distribuiti uniformemente su un qualsiasi intervallo $[a, b]$ (basta riscalare).

8.4 Confronto tra gli Algoritmi

Algoritmo	Caso peggiore	Caso medio	Ipotesi sull'input
INSERTIONSORT	$\Theta(n^2)$	$\Theta(n^2)$	nessuna
MERGESORT	$\Theta(n \lg n)$	$\Theta(n \lg n)$	nessuna
HEAPSORT	$\Theta(n \lg n)$	$\Theta(n \lg n)$	nessuna
QUICKSORT	$\Theta(n^2)$	$\Theta(n \lg n)$	nessuna
COUNTINGSORT	$\Theta(k + n)$	$\Theta(k + n)$	interi in $[0, k]$
BUCKETSORT	$\Theta(n^2)$	$\Theta(n)$	decimali in $[0, 1)$, distr. uniforme

Nota

Il limite $\Omega(n \lg n)$ è un **lower bound per gli algoritmi basati sul confronto**: non si può fare meglio con quel modello. COUNTINGSORT e BUCKETSORT lo violano sfruttando informazioni aggiuntive sull'input (tipo e distribuzione dei valori), non disponibili nel modello generale del confronto.

9. Hashing e Tabelle Hash

Le tabelle hash sono strutture dati efficienti per implementare dizionari che supportano le sole operazioni INSERT, SEARCH e DELETE. Analogamente a un array indicizzato da interi, una tabella hash indicizza gli elementi tramite la *chiave* dell'elemento stesso. Nel caso medio, sotto ipotesi ragionevoli, il tempo di ricerca è $O(1)$, anche se nel caso peggiore è $O(n)$.

9.1 Tavole a indirizzamento diretto

Definizione 9.1 (Tavola a indirizzamento diretto). Sia U l'insieme di tutti i possibili valori delle chiavi. Se $|U|$ non è troppo grande e nessun elemento ha chiave uguale a un altro, è possibile convertire ogni chiave in un intero e usare un array T di lunghezza $|U|$ per memorizzare i riferimenti agli elementi. La funzione di indicizzazione è:

$$I(k) = \sum_{i=0}^{n-1} (k[i] - 'A') \cdot 26^i$$

(nell'esempio con chiavi alfanumeriche di 2 caratteri in $[A, \dots, Z]$, $|U| = 26^2 = 676$).

L'indirizzamento diretto è praticabile solo quando $|U|$ è piccolo. Se $n \ll |U|$ (poche chiavi usate su un universo enorme) si spreca molta memoria: si deve allocare un array di dimensione $|U|$ anche se si memorizzano solo n elementi.

9.2 Tabelle hash

Quando $|U|$ è troppo grande, si usa una tabella di dimensione $m \approx n$ (con $m \ll |U|$) e una **funzione hash**

$$h : U \rightarrow \{0, \dots, m-1\}$$

che mappa la chiave k nell'indice $h(k)$ (*valore hash* di k). Poiché $m \ll |U|$, h non può essere iniettiva: possono quindi verificarsi **collisioni**.

Definizione 9.2 (Funzione hash perfetta). Una funzione hash h è *perfetta* per un insieme di chiavi S se non produce collisioni sulle sole chiavi di S , ovvero se h è iniettiva su S .

9.3 Funzioni hash: scelta e qualità

Una funzione hash ideale determina ogni valore $h(k)$ in modo casuale e indipendente nell'insieme $\{0, \dots, m-1\}$, restituendo sempre lo stesso valore per la stessa chiave (*funzione di hash uniforme e indipendente* o *oracolo casuale*). Tale funzione è solo teorica; in pratica si usano funzioni che la approssimano.

9.3.1 Conversione verso gli interi

Quando le chiavi non sono interi, si converte prima la chiave in un intero:

- **Caratteri Unicode (Java):** base 65536 (16 bit per char):

$$k' = \sum_i c_i \cdot 65536^i$$

- **Caratteri Latin-1:** base 256.
- **Subset ristretto:** base ulteriormente ridotta riassegnando gli ordinali a partire da 0.

Nota

Per stringhe di anche soli 2 caratteri, la somma k' supera il limite di un `int` Java (32 bit). Il risultato viene troncato a $w = 32$ bit, equivalente a calcolare $k' \bmod 2^{32}$. La conversione perde biettività (possibili collisioni), ma questo è accettabile: anche h stessa non è iniettiva. L'obiettivo è semplicemente *mescolare* i caratteri in un intero prima di applicare h .

9.4 Hashing statico

9.4.1 Metodo della divisione

Definizione 9.3 (Metodo della divisione).

$$h(k) = k \bmod m$$

Richiede una sola operazione. La scelta di m è critica: si preferisce un **numero primo non vicino a potenze di 2**, oppure un numero con fattori primi > 20 (Lum, 1971).

Se $m = 2^p$, allora $h(k)$ dipende solo dagli p bit meno significativi di k , ignorando completamente la parte alta della chiave. Ad esempio, con $m = 2^{12} = 4096$:

$$h(40960) = h(45056) = h(49152) = h(53248) = 0$$

poiché questi valori hanno tutti i 12 bit meno significativi pari a zero.

9.4.2 Metodo della moltiplicazione

Definizione 9.4 (Metodo della moltiplicazione). Sia $0 < A < 1$ una costante. Il valore hash è:

$$h(k) = \lfloor m \cdot ((k \cdot A) \bmod 1) \rfloor$$

dove $(k \cdot A) \bmod 1$ è la parte frazionaria di $k \cdot A$. Il valore m non è critico; si usa tipicamente $m = 2^p$.

Nota

Donald Knuth suggerisce $A \approx \frac{\sqrt{5} - 1}{2} \approx 0.6180$ (sezione aurea).

9.4.3 Metodo moltiplica-e-scorri

Il metodo della moltiplicazione può essere reso più efficiente fissando $m = 2^l$ con $0 < l \leq w$ (w = numero di bit della parola macchina). Si pone $a = A \cdot 2^w$ (intero dispari), e si calcola:

$$h_a(k) = ((k \cdot a) \bmod 2^w) \gg (w - l)$$

dove $\gg$ è lo scorrimento logico a destra (inserisce zeri a sinistra, a differenza di $\gg$ che mantiene il bit di segno). Si estraggono così gli l bit più significativi di $r_0 = (k \cdot a) \bmod 2^w$.

Esempio 9.1 (Moltiplica-e-scorri con $w = 8$, $m = 16$). Dati $w = 8$, $m = 2^4 = 16$ (dunque $l = 4$), $k = 202$ (11001010_2), $a = 173$ (10101101_2):

$$k \cdot a = 202 \cdot 173 = 34946, \quad r_0 = 34946 \bmod 256 = 130 = 10000010_2$$

$$h_a(k) = 10000010_2 \gg 4 = 00001000_2 = 8$$

La chiave viene quindi assegnata alla cella $8 \in \{0, \dots, 15\}$.

9.5 Hashing randomizzato

Con una funzione hash statica, un avversario può costruire n chiavi che collidono tutte nella stessa cella, portando a $n - 1$ collisioni (caso peggiore sistematico). L'hashing randomizzato risolve questo problema scegliendo casualmente la funzione hash *all'inizio di ogni esecuzione* da una famiglia opportuna: nessun input può essere sistematicamente il caso peggiore.

Definizione 9.5 (Hashing universale). Sia $\mathcal{H}$ una famiglia finita di funzioni hash con dominio U e codominio $\{0, 1, \dots, m - 1\}$. $\mathcal{H}$ è *universale* se per ogni coppia di chiavi distinte $k_i, k_j \in U$:

$$\Pr_{h \in \mathcal{H}} \{h(k_i) = h(k_j)\} \leq \frac{1}{m}$$

Equivalentemente, il numero di funzioni $h \in \mathcal{H}$ che fanno collidere k_i e k_j è al più $|\mathcal{H}|/m$.

Definizione 9.6 (Classificazione delle famiglie). Sia $\mathcal{H}$ una famiglia finita di funzioni hash ($U \rightarrow \{0, \dots, m - 1\}$) e $h \in \mathcal{H}$ scelta casualmente:

- $\mathcal{H}$ è **uniforme** se $\forall k \in U$ e $\forall q \in \{0, \dots, m - 1\}$: $\Pr\{h(k) = q\} = 1/m$.
- $\mathcal{H}$ è **ε -universale** se $\forall k_i \neq k_j$: $\Pr\{h(k_i) = h(k_j)\} \leq \varepsilon$.
- $\mathcal{H}$ è **universale** se è ε -universale con $\varepsilon = 1/m$.

Una famiglia uniforme e indipendente è anche universale (ma non viceversa).

Famiglia basata sulla teoria dei numeri. Sia p un numero primo tale che ogni chiave $k \in U$ sia associabile a un intero in $[0, \dots, p - 1]$, con $p > m$. Siano $\mathbb{Z}_p = \{0, \dots, p - 1\}$ e $\mathbb{Z}_p^* = \{1, \dots, p - 1\}$. Per ogni $a \in \mathbb{Z}_p^*$ e $b \in \mathbb{Z}_p$ si definisce:

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m$$

La famiglia $\mathcal{H}_{pm} = \{h_{ab} \mid a \in \mathbb{Z}_p^*, b \in \mathbb{Z}_p\}$ è universale, con $|\mathcal{H}_{pm}| = p(p - 1)$ e m arbitrario.

Famiglia $2/m$ -universale basata su moltiplica-e-scorri. La famiglia:

$$\mathcal{H} = \{h_a(\cdot) \mid 1 \leq a < 2^w, a \text{ dispari}\}$$

è $2/m$ -universale (con $m = 2^l$). La velocità di calcolo di $h_a(k)$ (una moltiplicazione e uno scorrimento) compensa la probabilità di collisione $2/m$ invece di $1/m$.

Nella versione italiana di [CLRS] sono presenti due errori nella definizione di questa famiglia: verificare sempre la definizione originale (edizione inglese).

9.6 Gestione delle collisioni

9.6.1 Risoluzione mediante concatenamento

Nel concatenamento (*chaining*), ogni cella $T[j]$ della tabella è la testa di una lista concatenata. Tutti gli elementi con lo stesso valore hash finiscono nella stessa lista.

Le operazioni sono:

- $\text{INSERT}(o)$: inserisce o in testa alla lista $T[h(o.\text{key})] — \Theta(1)$.
- $\text{SEARCH}(k)$: scorre la lista $T[h(k)]$ cercando la chiave k .
- $\text{DELETE}(o)$: rimuove o dalla lista $T[h(o.\text{key})]$; poiché o contiene i puntatori **prev/next**, l'operazione è $\Theta(1)$.

Definizione 9.7 (Fattore di carico). Il *fattore di carico* (load factor) è:

$$\alpha = \frac{n}{m}$$

dove n è il numero di elementi memorizzati e m la dimensione della tabella. Nel concatenamento $\alpha > 1$ è accettabile (preferibilmente $\alpha \approx 1$).

Teorema 9.1 (Costo medio — ricerca senza successo). *In una tabella hash con concatenamento, sotto l'ipotesi di hashing uniforme e indipendente, una ricerca senza successo richiede in media tempo $\Theta(1 + \alpha)$.*

Dimostrazione. Ogni lista delle m celle ha lunghezza media α . La ricerca di una chiave non presente richiede $\Theta(1)$ per calcolare il valore hash e $\Theta(\alpha)$ per scorrere la lista. $\square$

Teorema 9.2 (Costo medio — ricerca con successo). *Sotto le stesse ipotesi, una ricerca con successo richiede in media tempo $\Theta(1 + \alpha)$.*

Dimostrazione. Sia x_i l' i -esimo elemento inserito. Per ogni coppia (i, j) con $j > i$ e ogni cella q , sia X_{ijq} la variabile indicatrice dell'evento $\{ \text{si cerca } x_i, h(k_i) = q \text{ e } h(k_j) = q \}$. Sotto hashing uniforme e indipendente:

$$\Pr\{X_{ijq} = 1\} = \frac{1}{n} \cdot \frac{1}{m^2} \quad \Rightarrow \quad E[X_{ijq}] = \frac{1}{nm^2}$$

Il numero atteso di elementi che precedono l'elemento cercato nella lista vale:

$$E[1 + Z] = 1 + \sum_{j=1}^n \sum_{q=0}^{m-1} \sum_{i=1}^{j-1} \frac{1}{nm^2} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

Aggiungendo il costo $\Theta(1)$ per il calcolo dell'hash, il costo totale è $\Theta\left(2 + \frac{\alpha}{2} - \frac{\alpha}{2n}\right) = \Theta(1 + \alpha)$. $\square$

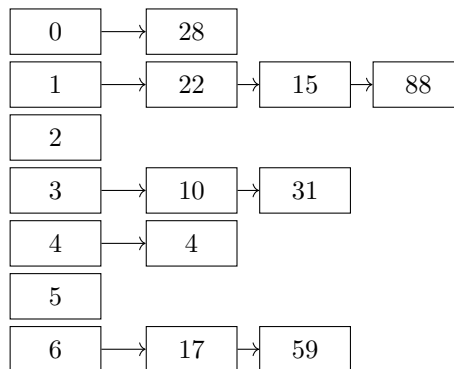
Tabella 1. Prestazioni della tabella hash con concatenamento

Operazione	Caso peggiore	Caso medio
INSERT	$\Theta(1)$	$\Theta(1)$
SEARCH	$\Theta(n)$	$\Theta(1 + \alpha)$
DELETE	$\Theta(1)$	$\Theta(1)$

Nota

Se $n = O(m)$ (il numero di elementi è proporzionale al numero di celle), allora $\alpha = n/m = O(1)$ e nel caso medio tutte le operazioni costano $O(1)$.

Esempio 9.2 (Tabella hash con concatenamento, $m = 7$). Funzione hash: $h(k) = k \bmod 7$. Chiavi da inserire: $\{10, 22, 31, 4, 15, 28, 17, 88, 59\}$.



Il **fattore di carico** è $\alpha = n/m = 9/7 \approx 1.3$. La ricerca di una chiave richiede in media $O(1 + \alpha) = O(1)$ se $m = \Theta(n)$.

9.6.2 Risoluzione mediante indirizzamento aperto

Nell'indirizzamento aperto (*open addressing*), tutti gli elementi sono memorizzati nella tabella stessa (niente liste esterne). In caso di collisione si cerca un'altra cella libera tramite una sequenza di *ispezioni* (*probing*). La funzione hash viene estesa come:

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}$$

La funzione hash non estesa è denotata $h_0(\cdot)$. Poiché tutti gli slot sono nella tabella, il fattore di carico deve soddisfare $\alpha < 1$.

Algorithm 32: INSERT(T, o)**Input:** Tabella hash T ; oggetto o da inserire (non già presente).**Output:** Oggetto inserito, oppure errore “Overflow”.

```

1  $i \leftarrow 0$ ;
2 while  $i \neq m$  do
3    $j \leftarrow h(o.key, i)$ ;
4   if  $T[j] = \text{NULL}$  or  $T[j] = \text{DELETE}$  then
5      $T[j] \leftarrow o$ ;
6     return “Inserito”;
7    $i \leftarrow i + 1$ ;
8 return “Overflow”;

```

Algorithm 33: SEARCH(T, k)**Input:** Tabella hash T ; chiave k da cercare.**Output:** Indice di k in T se presente, NULL altrimenti.

```

1  $i \leftarrow 0$ ;
2 while  $i \neq m$  do
3    $j \leftarrow h(k, i)$ ;
4   if  $T[j] = \text{NULL}$  then return NULL;
5   if  $T[j] \neq \text{DELETE}$  and  $T[j].key = k$  then return  $j$ ;
6    $i \leftarrow i + 1$ ;
7 return NULL;

```

Algorithm 34: DELETE(T, i)**Input:** Tabella hash T ; indice i della cella da cancellare.

```

1  $T[i] \leftarrow \text{DELETE}$ ; // marcatore tombstone

```

Il marcatore DELETE (*tombstone*) è necessario per non interrompere le sequenze di ispezione. Se si sostituisse con NULL, la SEARCH si fermerebbe su quella cella producendo falsi negativi. SEARCH non si ferma su DELETE e continua l’ispezione; INSERT può riutilizzare una cella DELETE. Con molte cancellazioni le ispezioni peggiorano: si risolve con un **rehash** periodico (ricostruzione della tabella con i soli elementi presenti).

Analisi (ipotesi). L’analisi classica [CLRS] assume *hashing con permutazioni indipendenti e uniformi*: per ogni chiave k , la sequenza $\langle h(k, 0), \dots, h(k, m-1) \rangle$ è una permutazione casuale uniforme di $\{0, \dots, m-1\}$, e le sequenze per chiavi diverse sono indipendenti. Si assume inoltre assenza di cancellazioni (i marcatori DELETE degradano le prestazioni).

Esempio 9.3 (Ispezioni in funzione di α). Con $\alpha = 1/2$ (tabella riempita al 50%): ricerca con insuccesso $\rightarrow 1/(1 - 1/2) = 2$ ispezioni; ricerca con successo $\rightarrow 2 \ln 2 \approx 1.387$ ispezioni.

Tabella 2. Numero medio di ispezioni con indirizzamento aperto

Operazione	Caso medio
Ricerca con successo	$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$
Ricerca con insuccesso	$\frac{1}{1-\alpha}$

Con $\alpha = 0.75$ (tabella riempita al 75%): ricerca con insuccesso $\rightarrow 4$ ispezioni; ricerca con successo $\rightarrow \frac{4}{3} \ln 4 \approx 1.848$ ispezioni.

9.6.3 Ispezione lineare

Definizione 9.8 (Ispezione lineare).

$$h(k, i) = (h_0(k) + i) \bmod m, \quad i = 0, 1, \dots, m-1$$

In caso di collisione si scansionano le celle una ad una a partire da quella adiacente alla posizione originale.

L'ispezione lineare soffre del **problema del clustering primario**: le sequenze di celle adiacenti occupate (*cluster*) tendono a crescere rapidamente, vanificando la distribuzione uniforme. Il metodo genera al più m permutazioni distinte invece di $m!$, quindi non soddisfa l'ipotesi di hashing con permutazioni indipendenti. È comunque utile su architetture con memoria gerarchica (cache/RAM/disco) grazie alla località spaziale degli accessi.

9.6.4 Doppio hashing

Definizione 9.9 (Doppio hashing).

$$h(k, i) = (h_0(k) + i \cdot p(k)) \bmod m, \quad i = 0, 1, \dots, m-1$$

dove $p(k)$ è una seconda funzione hash (*funzione probe*). Affinché la sequenza copra tutte le m celle è sufficiente che $\gcd(p(k), m) = 1$.

Nota

Due costruzioni che garantiscono la copertura completa:

- m primo e $0 < p(k) < m$: ad esempio $m = 701$, $p(k) = 1 + (k \bmod 500)$.
- $m = 2^c$ e $p(k)$ dispari: ad esempio $m = 512$, $p(k) = 1 + 2(k \bmod (m/2))$.

Il doppio hashing non soffre del clustering primario ed è in generale migliore dell'ispezione lineare e quadratica. Pur non soddisfacendo l'ipotesi di hashing con permutazioni indipendenti, le permutazioni prodotte ne hanno molte caratteristiche.

10. Alberi Binari di Ricerca

Gli **alberi binari di ricerca** (ABR) sono strutture dati che possono essere usate come *dizionari* o *code di priorità*. Le operazioni di base hanno costo proporzionale all'altezza dell'albero, quindi è importante mantenerli il più possibile bilanciati (altezza $\approx \lg n$). Quando non è possibile costruire ABR bilanciati si usano varianti come gli **alberi rosso-neri**, che garantiscono altezza $O(\lg n)$ nel caso peggiore. Un altro tipo importante sono i **B-alberi**, adatti a gestire grandi quantità di dati nella memoria secondaria (in programma in “Basi di dati”).

10.1 Definizione di ABR

Definizione 10.1 (Albero Binario di Ricerca). Un **albero binario di ricerca** (ABR) è un albero binario in cui ogni nodo x contiene:

- una chiave $x.key$ (che individua i dati satellite),
- i riferimenti $x.left$, $x.right$ e $x.p$ al figlio sinistro, destro e al genitore rispettivamente.

La radice $root$ è l'unico nodo con $x.p = \text{NULL}$.

Definizione 10.2 (Proprietà fondante). Le chiavi in un ABR soddisfano la seguente proprietà. Sia x un nodo:

- se y è un nodo del sottoalbero **sinistro** di x , allora $y.key < x.key$;
- se y è un nodo del sottoalbero **destro** di x , allora $y.key > x.key$.

La proprietà fondante vale per *tutti* i nodi del sottoalbero, non solo per i figli diretti. Lo stesso insieme di chiavi può essere rappresentato da ABR di forma diversa: la struttura dipende dall'ordine degli inserimenti.

Definizione 10.3 (Albero bilanciato in altezza). Un albero binario T è detto **bilanciato in altezza** (*height-balanced*) se

$$\forall t \in T, \quad |h(t_{sx}) - h(t_{dx})| \leq k,$$

dove t_{sx} e t_{dx} sono i sottoalberi sinistro e destro di t , e $h(t)$ è l'altezza del sottoalbero radicato in t . Solitamente $k = 1$. In generale, T è detto bilanciato quando la sua altezza è $O(\lg n)$, dove n è il numero di nodi.

10.2 Attraversamento

10.2.1 Attraversamento Simmetrico (in-order)

L'attraversamento **simmetrico** (in-order) sfrutta la proprietà fondante per visitare le chiavi in ordine crescente: si visita ricorsivamente il sottoalbero sinistro, poi la radice, poi il sottoalbero destro.

Algorithm 35: INORDERTREEWALK(x)

Input: Un nodo x di un ABR

```

1 if  $x = \text{NULL}$  then
2   return
3 INORDERTREEWALK( $x.\text{left}$ );
4 visit( $x.\text{key}$ ) ;                      // visit è una procedura esterna
5 INORDERTREEWALK( $x.\text{right}$ );
```

Nota

Su qualunque ABR che rappresenti l'insieme $\{2, 5, 5, 6, 7, 8\}$, INORDERTREEWALK produce sempre l'output ordinato $2, 5, 5, 6, 7, 8$, indipendentemente dalla forma (bilanciata o non) dell'albero.

Teorema 10.1 (Costo di INORDERTREEWALK). *Se x è la radice di un ABR di n nodi e il tempo di esecuzione di una singola chiamata a visit è $O(1)$, l'esecuzione di INORDERTREEWALK richiede tempo $\Theta(n)$.*

Dimostrazione. Sia $T(n)$ il tempo di esecuzione di INORDERTREEWALK su un albero di n nodi.

- $T(n) = \Omega(n)$: banale, si visitano n nodi.
- Si deve dimostrare $T(n) = O(n)$.
- $T(0) = c$ per il controllo che il nodo sia NULL.
- Per $n > 0$, con il sottoalbero sinistro di k nodi e quello destro di $n - k - 1$ nodi:

$$T(n) \leq T(k) + T(n - k - 1) + d,$$

dove d è un limite superiore al tempo costante del corpo della chiamata.

- Metodo di sostituzione: si assume $T(n) \leq (c + d)n + c$. Per $n = 0$ è banale. Per $n > 0$:

$$T(n) \leq [(c + d)k + c] + [(c + d)(n - k - 1) + c] + d = (c + d)n + c. \quad \square$$

10.2.2 Pre-ordine e Post-ordine

Gli attraversamenti in **pre-ordine** (radice $\rightarrow$ sx $\rightarrow$ dx) e in **post-ordine** (sx $\rightarrow$ dx $\rightarrow$ radice) non producono una sequenza ordinata come l'in-order.

Nota

Per esercizio: scrivere sia la versione ricorsiva sia quella iterativa di entrambi gli attraversamenti.

10.3 Interrogazione di un ABR

Gli ABR supportano le operazioni SEARCH, MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR. Ciascuna richiede un tempo $O(h)$, dove h è l'altezza dell'ABR in input. Per i dettagli si veda [?].

10.3.1 Search**Algorithm 36:** SEARCH(x, k)

Input: Un nodo x di un ABR, una chiave k

Output: Un nodo che contiene k se esiste, NULL altrimenti

```

1 if  $x = \text{NULL}$  or  $k = x.\text{key}$  then
2   return  $x$ 
3 if  $k < x.\text{key}$  then
4   return SEARCH( $x.\text{left}, k$ )
5 return SEARCH( $x.\text{right}, k$ );
```

I nodi incontrati durante la ricerca formano un cammino verso il basso dell'albero, quindi il tempo di esecuzione è $O(h)$.

Esempio 10.1 (Ricerca di 13 in un ABR). Nell'ABR con radice 15 e nodi $\{2, 3, 4, 6, 7, 9, 13, 15, 17, 18, 20\}$, la ricerca di 13 esegue il percorso $15 \rightarrow 6 \rightarrow 7 \rightarrow 13$: tre chiamate ricorsive.

La versione **iterativa** è più efficiente in pratica (stessa complessità $O(h)$, ma costante nascosta minore poiché evita l'overhead dello stack delle chiamate):

Algorithm 37: ITERSEARCH(x, k)

Input: Un nodo x di un ABR, una chiave k

Output: Un nodo che contiene k se esiste, NULL altrimenti

```

1 while  $x \neq \text{NULL}$  and  $k \neq x.\text{key}$  do
2    $x \leftarrow (k < x.\text{key}) ? x.\text{left} : x.\text{right};$ 
3 return  $x$ ;
```

10.3.2 Minimum e Maximum

Per la proprietà fondante:

- la chiave **minima** si trova nel nodo più a sinistra (si scende sempre nel figlio sinistro fino a $x.\text{left} = \text{NULL}$);

- la chiave **massima** si trova nel nodo più a destra.

Entrambe le operazioni hanno costo $O(h)$.

Algorithm 38: MINIMUM(x)

Input: Un nodo x di un ABR

Output: Il nodo con la chiave minima nel sottoalbero radicato in x

1 **while** $x.left \neq NULL$ **do**

2 $x \leftarrow x.left;$

3 **return** $x;$

Nota

La procedura MAXIMUM è simmetrica: si scende seguendo i figli destri. È lasciata per esercizio.

10.3.3 Successor e Predecessor

Definizione 10.4 (Successore). Il **successore** di un nodo x è il nodo con la più piccola chiave k' tale che $k' > x.key$ nella sequenza in-order. In un ABR si può trovare senza confrontare le chiavi:

- se x ha il sottoalbero destro, il successore è MINIMUM($x.right$);
- altrimenti, è il primo antenato y per il quale x appartiene al sottoalbero *sinistro* di y (si risale verso la radice finché si proviene da un figlio sinistro).

Algorithm 39: SUCCESSOR(x)

Input: Un nodo x di un ABR

Output: Il nodo successore di x nella sequenza ordinata dell'ABR

1 **if** $x.right \neq NULL$ **then**

2 **return** MINIMUM($x.right$)

3 $y \leftarrow x.p;$

4 **while** $y \neq NULL$ **and** $x = y.right$ **do**

5 $x \leftarrow y;$

6 $y \leftarrow y.p;$

7 **return** $y;$

Nota

Nell'ABR di esempio: il successore di 4 è 6, il successore di 13 è 15. La procedura PREDECESSOR è simmetrica ed è lasciata per esercizio.

10.4 Inserimento e Cancellazione

Inserire o cancellare un elemento modifica la struttura dell'albero: occorre sempre garantire che la nuova struttura soddisfi ancora la proprietà fondante (ripristino). L'inserimento è semplice; la cancellazione richiede un ripristino più articolato.

10.4.1 Inserimento

INSERT percorre l'albero dalla radice verso il basso e inserisce z sempre come **foglia** nella posizione corretta. Non controlla la presenza di duplicati: può quindi inserire più copie dello stesso valore.

Algorithm 40: INSERT(t, z)

Input: Un ABR t e un nodo $z \notin t$ (con $z.left = z.right = \text{NULL}$)

```

1  $y \leftarrow \text{NULL};$ 
2  $x \leftarrow t.root;$ 
3 while  $x \neq \text{NULL}$  do
4    $y \leftarrow x;$ 
5    $x \leftarrow (z.key < x.key) ? x.left : x.right;$ 
6  $z.p \leftarrow y;$ 
7 if  $y = \text{NULL}$  then
8    $t.root \leftarrow z;$                                 //  $t$  era vuoto
9 else
10  if  $z.key < y.key$  then
11     $y.left \leftarrow z;$ 
12  else
13     $y.right \leftarrow z;$ 

```

10.4.2 Cancellazione

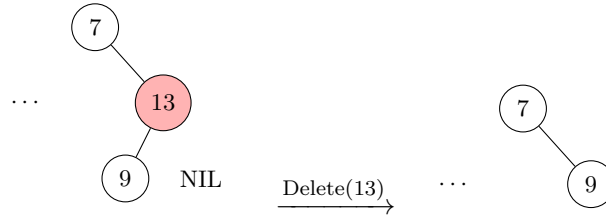
La cancellazione del nodo z gestisce i seguenti casi:

1. z **non ha figli**: si cancella direttamente z .
2. z **ha un solo figlio**: il figlio prende il posto di z .
3. z **ha entrambi i figli** e il suo successore y è figlio diretto di z (caso c): y prende semplicemente il posto di z .
4. z **ha entrambi i figli** e il successore y non è figlio diretto di z (caso d): si ricollocano y al posto di z e il figlio destro di y al posto di z . È il caso più complesso.

La soluzione CLRS utilizza la procedura ausiliaria

Esempio 10.2 (I quattro casi di DELETE). Consideriamo l'ABR con radice 15 e nodi $\{2, 3, 4, 6, 7, 9, 13, 15, 17, 18, 20\}$.

Caso (a) — z senza figlio sinistro (es. cancella 13).



TRANSPLANT($t, 13, 9$): il figlio destro (9) prende il posto di 13.

Caso (c) — entrambi i figli, successore figlio diretto (es. cancella 15). Il successore di 15 è 17 (figlio diretto del sottoalbero destro, senza figlio sinistro). TRANSPLANT($t, 15, 17$), poi si aggancia il sottoalbero sinistro di 15 come figlio sinistro di 17.

Caso (d) — entrambi i figli, successore non figlio diretto (es. cancella 6). Il successore di 6 è 7. Poiché $7.p \neq 6$, prima si esegue TRANSPLANT($t, 7, 13$) (il figlio destro di 7 prende il posto di 7), poi TRANSPLANT($t, 6, 7$) e si riagganciano i sottoalberi.

TRANSPLANT, che sostituisce il sottoalbero radicato in u con quello radicato in v :

Algorithm 41: TRANSPLANT(t, u, v)

Input: Un ABR t , due nodi $u, v \in t$ (v può essere NULL)

Output: In t , sostituisce il sottoalbero con radice u con quello con radice v

```

1 if  $u.p = \text{NULL}$  then
2    $t.\text{root} \leftarrow v$ ;
3 else
4   if  $u = u.p.\text{left}$  then
5      $u.p.\text{left} \leftarrow v$  ;                               //  $u$  è figlio sinistro
6   else
7      $u.p.\text{right} \leftarrow v$ ;
8 if  $v \neq \text{NULL}$  then
9    $v.p \leftarrow u.p$  ;                                   // Inserisci  $v$  in  $t$ 

```

TRANSPLANT ha complessità $O(1)$.

Algorithm 42: DELETE(t, z)**Input:** Un ABR t e un nodo $z \in t$ **Output:** In t , cancella z mantenendo t un ABR

```

1 if  $z.left = NULL$  then
2   TRANSPLANT( $t, z, z.right$ );
3   return ;                                     // caso (a)
4 if  $z.right = NULL$  then
5   TRANSPLANT( $t, z, z.left$ );
6   return ;                                     // caso (b)
7  $y \leftarrow \text{MINIMUM}(z.right)$ ;
8 if  $y.p \neq z$  then
9   TRANSPLANT( $t, y, y.right$ ) ; // caso (d): al posto di  $y$  metti suo figlio  $x$ 
10   $y.right \leftarrow z.right$  ;           // caso (d):  $r$  diventa figlio dx di  $y$ 
11   $y.right.p \leftarrow y$ ;
12 TRANSPLANT( $t, z, y$ ) ;                     // ultimo passo di (c) e (d)
13  $y.left \leftarrow z.left$ ;
14  $y.left.p \leftarrow y$ ;

```

DELETE ha complessità $O(h)$ perché al suo interno può eseguire MINIMUM. Non modificare la struttura prima di aver aggiornato tutti i puntatori: l'ordine delle operazioni in DELETE è cruciale.

10.5 Costruzione di un ABR

La complessità delle operazioni fondamentali è $O(h)$. Dalle proprietà degli alberi binari:

$$\lfloor \lg n \rfloor \leq h \leq n - 1,$$

dove n è il numero di nodi. Il caso peggiore ($h = n - 1$) si raggiunge inserendo le chiavi in ordine crescente (o decrescente): l'ABR degenera in una lista.

Teorema 10.2 (Altezza media di un ABR casuale). *L'altezza media di un ABR costruito in modo casuale con n chiavi **distinte** è $O(\lg n)$.*

Nota

Un ABR è *costruito in modo casuale* se, partendo dall'albero vuoto, le chiavi vengono inserite una alla volta scegliendole casualmente dall'insieme. Non sono note formule che stimano l'altezza media dopo una sequenza arbitraria di operazioni di inserimento e cancellazione.

10.6 Cenno agli ABR Rosso-Neri

Operazioni di inserimento e cancellazione possono sbilanciare un ABR fino a renderlo degenerare. Esistono varianti che mantengono il bilanciamento; la più nota è l'**ABR rosso-nero**.

Un ABR rosso-nero associa a ogni nodo un colore (rosso o nero) e rispetta le seguenti proprietà:

- ogni nodo è rosso o nero;
- la radice è nera; ogni riferimento a NULL è considerato nero;
- se un nodo è rosso, entrambi i figli sono neri;
- per ogni nodo, tutti i cammini semplici dal nodo alle foglie contengono lo stesso numero di nodi neri.

Teorema 10.3 (Altezza degli ABR rosso-neri). *L'altezza di un ABR rosso-nero con n nodi è al più $2 \lg(n + 1)$.*

Gli ABR rosso-neri **non sono in programma** per quest'anno: non si richiedono gli algoritmi di inserimento e cancellazione.

10.7 Cenno ai Radix Tree

Definizione 10.5 (Radix Tree). Un **radix tree** (o *radix trie*, *trie compresso*) è una struttura ad albero usata per rappresentare un insieme di stringhe binarie sfruttando i **prefissi comuni**:

- ogni arco è etichettato con un simbolo dell'alfabeto (0 oppure 1);
- il cammino dalla radice a un nodo rappresenta il prefisso ottenuto concatenando le etichette degli archi attraversati;
- una stringa s appartiene all'insieme se esiste un nodo marcato come **terminale** il cui cammino dalla radice è esattamente s ;
- stringhe con prefisso comune condividono la parte iniziale del cammino;
- ricerca, inserimento e visita richiedono tempo proporzionale alla lunghezza della stringa considerata.

Esempio 10.3 (Radix tree per $S = \{0, 011, 10, 1001011\}$). Le stringhe 0 e 011 condividono il prefisso 0; le stringhe 10 e 1001011 condividono il prefisso 10. I nodi terminali corrispondono esattamente agli elementi di S . La struttura ha altezza pari alla lunghezza della stringa più lunga dell'insieme (qui: 7).

10.8 Esercizi

1. Studiare il Capitolo 12 di [?].
2. Dimostrare la correttezza di `INORDERTREEWALK` per induzione.
Suggerimento: la base è un albero con un solo nodo (banale). Si suppone che la proprietà valga per alberi di dimensione $n - 1$ e si considera un albero di n nodi. Un albero di n nodi può essere visto come un nodo radice con due sottoalberi: uno di k nodi ($0 \leq k \leq n - 1$) e uno di $n - 1 - k$ nodi.
3. Scrivere le versioni ricorsiva e iterativa degli attraversamenti in pre-ordine e post-ordine.
4. Scrivere la procedura `MAXIMUM`.
5. Scrivere la procedura `PREDECESSOR`.
6. Risolvere gli esercizi 12.1-2, 12.1-3, 12.1-4, 12.2-2, 12.2-3, 12.2-4, 12.2-5, 12.3-1, 12.3-3 di [?].
Suggerimento per 12.1-3: la soluzione iterativa senza stack richiede di trasformare l'albero in un *albero threaded* (con fili). Si veda <https://www.techiedelight.com/inorder-tree-traversal-iterative-recursive>.

11. Programmazione Dinamica

La **programmazione dinamica** è un paradigma per la progettazione di algoritmi che risolvono problemi di ottimizzazione. Introdotta da Bellman (1957), si applica quando il problema presenta *sottostruttura ottima* e *sottoproblemi sovrapposti*: la soluzione ottima contiene le soluzioni ottime dei sottoproblemi, e un approccio ricorsivo naïve risolverebbe gli stessi sottoproblemi più volte, con costo esponenziale.

11.1 Idea generale

La tecnica si articola in **4 fasi**:

1. **Caratterizzazione** della struttura di una soluzione ottima.
2. **Definizione ricorsiva** del valore di una soluzione ottima.
3. **Calcolo** del valore secondo uno schema *bottom-up*.
4. **Costruzione** della soluzione ottima dalle informazioni calcolate.

In termini moderni, la programmazione dinamica descrive il problema come un insieme di **stati** (o sottoproblemi). Per ogni stato si definiscono:

- i *cas* base;
- le *transizioni*: come combinare stati più piccoli per ottenere il valore dello stato corrente;
- l'*ordine di calcolo*, che deve rispettare le dipendenze tra stati.

Il costo totale dell'algoritmo è spesso:

$$\# \text{stati} \times \text{costo per calcolare una transizione.}$$

La stessa ricorrenza può essere implementata con una tabella *bottom-up* (tabulazione) oppure con ricorsione e *memoization*.

Nota

Quando si usa la programmazione dinamica. Si usa quando:

- una soluzione ottima contiene al suo interno le soluzioni ottime dei sottoproblemi (**sottostruttura ottima**);
- un algoritmo ricorsivo richiederebbe di risolvere più volte gli stessi sottoproblemi (**sottoproblemi sovrapposti**);
- i sottoproblemi possono essere descritti da un numero limitato di parametri, cioè da uno **stato**.

11.1.1 Confronto con Divide et Impera

Proprietà	Tabulazione (bottom-up)	Divide et Impera
Tipo di risoluzione	bottom-up	top-down
Tipo di programma	iterativa	ricorsiva
Uso della memoria	esplicito per i risultati	implicito
Applicazione ideale	sottoistanze non disgiunte	sottoistanze disgiunte

Nota

La programmazione dinamica può essere implementata anche top-down tramite *memoization*, mantenendo la struttura ricorsiva del divide et impera ma riutilizzando i risultati già calcolati.

11.2 Esempi di applicazione: confronto di sequenze DNA

Una sequenza DNA si rappresenta come una stringa sull'alfabeto $\{A, C, G, T\}$. Il confronto tra due sequenze può rivelare quanto siano correlati i due organismi di origine. Esistono diversi criteri di confronto:

- verificare se una sequenza è sottostringa dell'altra;

- determinare il numero minimo di modifiche per trasformare una nell'altra (distanza di edit);
- trovare una *sottosequenza comune* di lunghezza massima.

11.3 Massima Sottosequenza Comune (LCS)

Definizione 11.1 (Sottosequenza). Data una sequenza $\langle a_1, \dots, a_n \rangle$, una **sottosequenza** è un sottoinsieme ordinato $\langle a_{i_1}, a_{i_2}, \dots, a_{i_k} \rangle$ dove la sequenza di indici $\langle i_1, i_2, \dots, i_k \rangle$ è strettamente crescente.

Esempio 11.1 (Sottosequenza). Data $X = \langle A, B, C, B, D, A, B \rangle$, la sequenza $Z = \langle B, C, D, B \rangle$ è sottosequenza di X perché i suoi indici in X formano la sequenza crescente $\langle 2, 3, 5, 7 \rangle$.

Definizione 11.2 (Sottosequenza comune — LCS). Date due sequenze X e Y , una sequenza Z è **sottosequenza comune** di X e Y se è sottosequenza di entrambe. La **Massima Sottosequenza Comune** (*Longest Common Subsequence*, LCS) è la sottosequenza comune di lunghezza massima.

Esempio 11.2 (LCS (CLRS)). Sia $X = \langle A, B, C, B, D, A, B \rangle$ e $Y = \langle B, D, C, A, B, A \rangle$. La sequenza $\langle B, C, A \rangle$ è una sottosequenza comune; la sottosequenza comune più lunga è $\langle B, C, B, A \rangle$ (lunghezza 4).

Problema formale. **Input:** due sequenze X e Y sullo stesso alfabeto. **Output:** la sottosequenza comune di lunghezza massima.

11.3.1 Fase 1 — Sottostruttura ottima

Sia $X_i = \langle x_1, \dots, x_i \rangle$ l' i -esimo prefisso di X .

Teorema 11.1 (Sottostruttura ottima della LCS). *Dati $X = \langle x_1, \dots, x_m \rangle$, $Y = \langle y_1, \dots, y_n \rangle$ e una LCS $Z = \langle z_1, \dots, z_k \rangle$:*

1. Se $x_m = y_n$, allora $z_k = x_m$ e Z_{k-1} è una LCS di X_{m-1} e Y_{n-1} .
2. Se $x_m \neq y_n$ e $z_k \neq x_m$, allora Z è una LCS di X_{m-1} e Y .
3. Se $x_m \neq y_n$ e $z_k \neq y_n$, allora Z è una LCS di X e Y_{n-1} .

Dimostrazione. Punto 1. Se $x_m = y_n$ ma Z non termina con tale carattere, si può sostituire l'ultimo elemento di Z con x_m ottenendo una LCS che termina con $x_m = y_n$. Tolto l'ultimo carattere, la parte rimanente è una LCS di X_{m-1} e Y_{n-1} .

Punto 2. Se $z_k \neq x_m$, allora Z è una LCS di X_{m-1} e Y : se esistesse una sottosequenza comune W di X_{m-1} e Y con $|W| > k$, allora W sarebbe anche sottosequenza comune di X e Y , contraddicendo l'ipotesi.

Punto 3. Dimostrazione simmetrica al punto 2. □

11.3.2 Fase 2 — Definizione ricorsiva del valore

Sia $c[i, j]$ la lunghezza di una LCS di X_i e Y_j . La soluzione del problema originale è $c[m, n]$. La ricorrenza è:

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ c[i - 1, j - 1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j \\ \max\{c[i, j - 1], c[i - 1, j]\} & \text{se } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

Perché non basta il Divide et Impera? Con $i = j = n$, il numero di chiamate ricorsive nel caso peggiore vale $T(i, j) = 2^{\binom{i+j}{i}} - 1$, valore **esponenziale** quando i e j crescono insieme. I sottoproblemi distinti sono invece soltanto $\Theta(nm)$, uno per ogni coppia di prefissi (X_i, Y_j) .

11.3.3 Fase 3 — Calcolo bottom-up: algoritmo LCS-Length

Algorithm 43: LCS-LENGTH(X, Y)

Input: $X = \langle x_1, \dots, x_m \rangle, Y = \langle y_1, \dots, y_n \rangle$

Output: Tabelle $c[0:m, 0:n]$ e $b[1:m, 1:n]$

```

1  Siano  $b[1:m, 1:n]$  e  $c[0:m, 0:n]$  due tabelle inizializzate a 0 ;           //  $b$  serve per
   ricostruire la LCS
2  for  $i \leftarrow 1$  to  $m$  do
3      for  $j \leftarrow 1$  to  $n$  do
4          if  $x_i = y_j$  then
5               $c[i, j] \leftarrow c[i - 1, j - 1] + 1$ ;
6               $b[i, j] \leftarrow \nwarrow$ ;
7          else
8              if  $c[i - 1, j] \geq c[i, j - 1]$  then
9                   $c[i, j] \leftarrow c[i - 1, j]$ ;
10                  $b[i, j] \leftarrow \uparrow$ ;
11             else
12                  $c[i, j] \leftarrow c[i, j - 1]$ ;
13                  $b[i, j] \leftarrow \leftarrow$ ;
14 return  $c, b$ 

```

Il tempo di computazione è $\Theta(nm)$.

11.3.4 Fase 4 — Ricostruzione: algoritmo Print-LCS

Algorithm 44: PRINT-LCS(b, X, i, j)

Input: Matrice b ; sequenza X ; indici correnti i, j

```

1 if  $i = 0$  and  $j = 0$  then
2   return
3 if  $b[i, j] = "\nwarrow"$  then
4   PRINT-LCS( $b, X, i - 1, j - 1$ );
5   stampa  $x_i$ ;
6 else
7   if  $b[i, j] = "\uparrow"$  then
8     PRINT-LCS( $b, X, i - 1, j$ );
9   else
10    PRINT-LCS( $b, X, i, j - 1$ );
```

Si invoca come PRINT-LCS($b, X, |X|, |Y|$). Il tempo di computazione è $O(n + m)$.

Esempio 11.3 (LCS su ABCBDAB / BDCABA). Per $X = \langle A, B, C, B, D, A, B \rangle$ e $Y = \langle B, D, C, A, B, A \rangle$, l'algoritmo restituisce la LCS $\langle B, C, B, A \rangle$ di lunghezza 4, leggendo le frecce $\nwarrow$ nella tabella b .

11.4 Massima Sottosequenza Crescente

Definizione 11.3 (Sottosequenza crescente). Una sottosequenza $a_{i_1}, a_{i_2}, \dots, a_{i_k}$ di interi è **crescente** se $a_{i_1} < a_{i_2} < \dots < a_{i_k}$.

Problema. **Input:** sequenza di interi $a_1, \dots, a_n$. **Output:** sottosequenza crescente di lunghezza massima.

Esempio 11.4 (MSC). La sequenza 5, 2, 8, 6, 3, 6, 9, 7 ha massima sottosequenza crescente 2, 3, 6, 9 (lunghezza 4).

11.4.1 Riduzione a cammino massimo in un DAG

Si costruisce un DAG $G = (V, E)$ dove esiste l'arco (i, j) se e solo se $i < j$ e $a_i < a_j$. Trovare la massima sottosequenza crescente è **equivalente** a trovare il cammino più lungo nel DAG.

11.4.2 Definizione ricorsiva

Sia $L(j)$ la lunghezza del cammino più lungo che termina nel nodo j :

$$L(j) = 1 + \max(\{0\} \cup \{L(i) \mid (i, j) \in E\}).$$

Il valore della soluzione ottima è $\max_j L(j)$.

11.4.3 Algoritmo bottom-up (versione valutativa)

Algorithm 45: MSCRESC-V($a_1, \dots, a_n$)

Input: Sequenza di interi $a_1, \dots, a_n$
Output: Lunghezza della massima sottosequenza crescente

```

1 Costruisci il DAG trasposto  $G^T = (V, E^T)$  da  $(a_1, \dots, a_n)$ ;
2  $L \leftarrow$  array di  $n$  interi ;                                // soluzioni parziali
3  $ottimo \leftarrow -\infty$ ;
4 for  $j \leftarrow 1$  to  $n$  do
5    $max \leftarrow 0$ ;
6   for ogni  $i$  tale che  $(j, i) \in E^T$  do
7     if  $L[i] > max$  then
8        $max \leftarrow L[i]$ 
9    $L[j] \leftarrow 1 + max$ ;
10  if  $L[j] > ottimo$  then
11     $ottimo \leftarrow L[j]$ 
12 return  $ottimo$ 

```

Complessità. La costruzione del grafo trasposto richiede $O(n^2)$. Il calcolo di tutti gli $L(j)$ richiede $O(|E|)$, con $|E| \leq \frac{n(n-1)}{2}$. La complessità totale è $O(n^2)$.

L'approccio Divide et Impera puro genera 2^{j-1} chiamate ricorsive per calcolare $L(j)$ nel caso peggiore (sequenza completamente crescente), rendendo l'algoritmo esponenziale. La programmazione dinamica risolve ciascuno degli n stati esattamente una volta.

11.5 String Matching Approssimato

Nella ricerca approssimata di un pattern P in un testo T si ammettono tre tipi di errore: *sostituzione* (caratteri corrispondenti diversi), *gap in P* (un carattere di T non allineato con nessun carattere di P), *gap in T* (un carattere di P non allineato con nessun carattere di T). Questo problema generalizza lo string matching esatto ed è strettamente correlato alla **distanza di edit** (Levenshtein).

Definizione 11.4 (Occorrenza k -approssimata). Data una stringa T (testo, lunghezza n) e una stringa P (pattern, lunghezza $m < n$), un'**occorrenza k -approssimata** di P in T è un allineamento di P con una sottostringa di T in cui compaiono esattamente k errori dei tipi: sostituzione, gap in P , gap in T .

Problema formale. **Input:** testo T di lunghezza n , pattern P di lunghezza $m < n$. **Output:** occorrenza k -approssimata minima (rispetto a k) di P in T .

11.5.1 Definizione ricorsiva

Sia $K[i, j]$ il minimo numero di errori nell'approssimare $P[1, \dots, i]$ con una sottostringa di $T[1, \dots, j]$ che termina in posizione j . La soluzione ottima è $k = \min_j K[m, j]$.

Equazione di ricorrenza unificata:

$$K[i, j] = \min\{ \text{diff}(i, j) + K[i-1, j-1], \ 1 + K[i-1, j], \ 1 + K[i, j-1] \}$$

dove $\text{diff}(i, j) = 0$ se $P[i] = T[j]$, altrimenti 1.

Casi base.

- $K[i, 0] = i$ (il pattern ha i caratteri in più della stringa vuota);
- $K[0, j] = 0$ (il pattern vuoto si allinea a costo 0 a qualunque prefisso di T).

11.5.2 Algoritmo SMA

Algorithm 46: SMA(T, P)

Input: Testo T , pattern P

Output: (numero minimo di errori; posizione ultimo carattere dell'occorrenza in T)

```

1  $n \leftarrow |T|;$ 
2  $m \leftarrow |P|;$ 
3 for  $i \leftarrow 0$  to  $n$  do
4    $K[0, i] \leftarrow 0$ 
5 for  $i \leftarrow 0$  to  $m$  do
6    $K[i, 0] \leftarrow i$ 
7 for  $i \leftarrow 1$  to  $m$  do
8   for  $j \leftarrow 1$  to  $n$  do
9      $\text{diff} \leftarrow (T[j] = P[i]) ? 0 : 1;$ 
10     $K[i, j] \leftarrow \min\{ \text{diff} + K[i-1, j-1], \ 1 + K[i-1, j], \ 1 + K[i, j-1] \};$ 
11  $\text{Min} \leftarrow +\infty;$ 
12  $\text{pos} \leftarrow -1;$ 
13 for  $j \leftarrow 1$  to  $n$  do
14   if  $K[m, j] < \text{Min}$  then
15      $\text{Min} \leftarrow K[m, j];$ 
16      $\text{pos} \leftarrow j;$ 
17 return ( $\text{Min}, \text{pos}$ )
```

La complessità è $\Theta(nm)$.

Esempio 11.5 (SMA su “It’s snowy” / “sunny”). Con $T = \text{It's snowy}$ e $P = \text{sunny}$, la tabella K produce $\min_j K[5, j] = 3$, confermando che il pattern si abbina al testo con 3 errori.

11.6 Problema dello Zaino (Knapsack 0-1)

Problema formale. **Input:** n oggetti con valore v_i e peso p_i (interi positivi); portata massima P . **Output:** vettore $x \in \{0, 1\}^n$ che massimizza $\sum_{i=1}^n x_i v_i$ con vincolo $\sum_{i=1}^n x_i p_i \leq P$.

Nota

Questa versione è chiamata **Zaino 0-1** perché ogni oggetto o si prende ($x_i = 1$) o si lascia ($x_i = 0$). Esistono anche le versioni *frazionaria* (risolvibile greedy) e *con ripetizione*.

11.6.1 Sottostruttura ottima

Si considera la soluzione ottima per i primi j oggetti con capacità w .

- Se l'oggetto j **non** è nella soluzione: la soluzione è ottima per i primi $j - 1$ oggetti con capacità w .
- Se l'oggetto j **è** nella soluzione: toltolo, rimane una soluzione ottima per i primi $j - 1$ oggetti con capacità $w - p_j$.

In entrambi i casi: *taglia e incolla* mostra che la soluzione ottima del problema originale contiene la soluzione ottima del sottoproblema.

11.6.2 Definizione ricorsiva

Sia $K[j, w]$ il massimo valore ottenibile usando uno zaino di capacità w e i primi j oggetti. La soluzione è $K[n, P]$.

$$K[j, w] = \begin{cases} 0 & \text{se } j = 0 \text{ o } w = 0 \\ K[j - 1, w] & \text{se } p_j > w \\ \max\{K[j - 1, w], K[j - 1, w - p_j] + v_j\} & \text{altrimenti} \end{cases}$$

11.6.3 Algoritmo Zaino

Algorithm 47: ZAINO(P, p, v)**Input:** Portata P ; array pesi p ; array valori v **Output:** Valore ottimo dello zaino

```

1  $n \leftarrow |v|$ ;
  ; // In Java le tabelle sono già inizializzate a 0
2 for  $j \leftarrow 1$  to  $n$  do
3   for  $w \leftarrow 1$  to  $P$  do
4     if  $p[j] > w$  then
5        $K[j, w] \leftarrow K[j - 1, w]$ ; // oggetto  $j$  troppo pesante
6     else
7        $K[j, w] \leftarrow \max(K[j - 1, w], K[j - 1, w - p[j]] + v[j])$ ;
8 return  $K[n, P]$ 

```

Complessità. La complessità è $\Theta(nP)$.

$\Theta(nP)$ è **pseudo-polinomiale**, non polinomiale nella dimensione dell'input. Se k è il numero di bit per rappresentare P , allora $k = \Theta(\log P)$, cioè $P = \Theta(2^k)$: il costo può essere esponenziale nella lunghezza della rappresentazione binaria di P . Il problema Zaino 0-1 è NP-difficile.

Esempio 11.6 (Zaino 0-1). Con $v = [6, 13, 4, 4]$, $p = [3, 13, 5, 3]$, $P = 13$, lo zaino ottimo contiene gli oggetti $\{x_1, x_3, x_4\}$ con valore totale $6 + 4 + 4 = 14$ e peso $3 + 5 + 3 = 11$.

11.7 Casi comuni di sottoproblemi

Nota

Caratterizzare correttamente la decomposizione in sottoproblemi richiede creatività ed esperienza. Quattro schemi ricorrono più frequentemente:

1. **Prefisso singolo.** Input $x_1, \dots, x_n$; il sottoproblema considera $x_1, \dots, x_i$. Numero di stati: $O(n)$.
2. **Doppio prefisso.** Input $x_1, \dots, x_m$ e $y_1, \dots, y_n$; il sottoproblema considera $x_1, \dots, x_i$ e $y_1, \dots, y_j$. Numero di stati: $O(nm)$.
3. **Intervallo.** Input $x_1, \dots, x_n$; il sottoproblema considera $x_i, \dots, x_j$. Numero di stati: $O(n^2)$ (tabella $n \times n$ triangolare superiore).
4. **Sottoalbero.** Input: albero radicato di n nodi; il sottoproblema è un sottoalbero radicato. Numero di stati: $O(n)$.

11.8 Memoization

Definizione 11.5 (Memoization). Tecnica (termine coniato da Donald Michie nel 1968, senza la “r” di *memorization*) che, in un programma ricorsivo, memorizza i risultati delle chiamate già eseguite in una struttura di supporto (hash table, array, matrice) indicizzata dai parametri, per riutilizzarli anziché ricalcolarli.

Confronto tra i tre approcci:

- **Tabulazione bottom-up:** risolve *tutte* le sottoistanze rispettando l'ordine di dipendenza. Iterativa, costanti moltiplicative ridotte.
- **Divide et impera:** risolve solo le sottoistanze necessarie ma ricalcola quelle comuni ogni volta.
- **Memoization:** mantiene la struttura ricorsiva del divide et impera ma riusa i risultati già incontrati. Può essere più efficiente della tabulazione se molte sottoistanze non sono necessarie; in generale ha overhead maggiore per le chiamate ricorsive.

11.8.1 Esempio: Zaino con memoization

Algorithm 48: ZAINO-MEMO(i, w)

Input: Indice oggetto i , capacità residua w ; P, v, p globali

Output: Valore ottimo per i primi i oggetti con capacità w

```

1 if  $K.CONTAINSKEY(i, w)$  then
2   return  $K.GET(i, w)$ 
3 if  $i = 0$  and  $w = 0$  then
4   return 0
5 if  $p[i] > w$  then
6    $k \leftarrow \text{ZAINO-MEMO}(i - 1, w)$  ; // troppo pesante
7 else
8    $k \leftarrow \max(\text{ZAINO-MEMO}(i - 1, w),$ 
9      $\text{ZAINO-MEMO}(i - 1, w - p[i]) + v[i])$ ;
10  $K.PUT((i, w), k)$ ;
11 return  $k$ 
```

Complessità: $O(nP)$, identica alla versione tabulare, ma con costanti moltiplicative più grandi per l'overhead delle chiamate ricorsive.

12. Può tornare utile

Questa sezione contiene materiale di riferimento (appendici) e domande d'esame.

12.1 Appendice A (Sommatorie)

12.1.1 Formule fondamentali

$$\begin{aligned}\sum_{i=1}^n 1 &= n \\ \sum_{i=1}^n i &= \frac{n(n+1)}{2} = \Theta(n^2) \\ \sum_{i=1}^n i^2 &= \frac{n(n+1)(2n+1)}{6} = \Theta(n^3) \\ \sum_{i=1}^n i^k &= \Theta(n^{k+1}) \quad \text{per } k \geq 0 \\ \sum_{i=0}^n x^i &= \frac{x^{n+1} - 1}{x - 1} \quad \text{per } x \neq 1 \\ \sum_{i=0}^{\infty} x^i &= \frac{1}{1 - x} \quad \text{per } |x| < 1\end{aligned}$$

12.1.2 Serie geometrica

Nota

La serie geometrica è fondamentale nell'analisi degli algoritmi ricorsivi.

Formula generale:

$$\sum_{i=0}^n ar^i = a \cdot \frac{r^{n+1} - 1}{r - 1}$$

Caso particolare importante ($r = 2$):

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1$$

Serie infinita convergente ($|r| < 1$):

$$\sum_{i=0}^{\infty} ar^i = \frac{a}{1 - r}$$

12.1.3 Serie armonica

$$H_n = \sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \Theta(\log n)$$

Nota

La serie armonica cresce **logaritmicamente**. Utile per analizzare:

- Quicksort (numero medio di confronti)
- Hashing (lunghezza catene)
- Alberi bilanciati (altezza media)

Approssimazione: $H_n \approx \ln n + \gamma$ dove $\gamma \approx 0.5772$ (costante di Eulero-Mascheroni)

12.1.4 Tecniche di manipolazione**1. Separazione dei termini:**

$$\sum_{i=1}^n (a_i + b_i) = \sum_{i=1}^n a_i + \sum_{i=1}^n b_i$$

2. Estrazione di costanti:

$$\sum_{i=1}^n c \cdot a_i = c \sum_{i=1}^n a_i$$

3. Telescoping (sommatoria telescopica):

$$\sum_{i=1}^n (a_i - a_{i-1}) = a_n - a_0$$

Esempio 12.1 (Telescoping applicato).

$$\sum_{i=1}^n \frac{1}{i(i+1)} = \sum_{i=1}^n \left(\frac{1}{i} - \frac{1}{i+1} \right) = 1 - \frac{1}{n+1} = \frac{n}{n+1}$$

4. Cambio di variabile:

Se $j = n - i$, allora:

$$\sum_{i=0}^n f(i) = \sum_{j=n}^0 f(n-j) = \sum_{j=0}^n f(n-j)$$

12.1.5 Approssimazione con integrali

Per f crescente:

$$\int_1^n f(x) dx \leq \sum_{i=1}^n f(i) \leq f(n) + \int_1^n f(x) dx$$

Esempio 12.2 (Somma di potenze).

$$\begin{aligned}\sum_{i=1}^n i^k &\approx \int_1^n x^k dx = \frac{n^{k+1}}{k+1} \\ \Rightarrow \sum_{i=1}^n i^k &= \Theta(n^{k+1})\end{aligned}$$

12.2 Appendice B (Insiemi, relazioni, funzioni, grafi, alberi)

12.2.1 Insiemi

Definizione 12.1 (Insieme). Collezione non ordinata di elementi distinti. Notazione: $A = \{1, 2, 3\}$

Operazioni fondamentali:

- **Unione:** $A \cup B = \{x : x \in A \text{ oppure } x \in B\}$
- **Intersezione:** $A \cap B = \{x : x \in A \text{ e } x \in B\}$
- **Differenza:** $A \setminus B = \{x : x \in A \text{ e } x \notin B\}$
- **Complemento:** $\bar{A} = \{x : x \notin A\}$ (rispetto a un universo)

Nota

Cardinalità: $|A|$ = numero di elementi in A

Insieme delle parti: $\mathcal{P}(A)$ = insieme di tutti i sottoinsiemi di A

Se $|A| = n$ allora $|\mathcal{P}(A)| = 2^n$

12.2.2 Relazioni

Definizione 12.2 (Relazione). Una relazione R da A a B è un sottoinsieme di $A \times B$ (prodotto cartesiano).

Scriviamo aRb se $(a, b) \in R$.

Proprietà di relazioni su A (relazione binaria):

- **Riflessiva:** $\forall a \in A : aRa$
- **Simmetrica:** $\forall a, b \in A : aRb \Rightarrow bRa$
- **Transitiva:** $\forall a, b, c \in A : (aRb \wedge bRc) \Rightarrow aRc$
- **Antisimmetrica:** $\forall a, b \in A : (aRb \wedge bRa) \Rightarrow a = b$

Definizione 12.3 (Relazione di equivalenza). Una relazione è di **equivalenza** se è riflessiva, simmetrica e transitiva.

Esempio: uguaglianza ($=$), congruenza modulo n

Definizione 12.4 (Ordine parziale). Una relazione è di **ordine parziale** se è riflessiva, antisimmetrica e transitiva.

Esempio: $\leq$ sui numeri reali, $\subseteq$ sugli insiemi

12.2.3 Funzioni

Definizione 12.5 (Funzione). Una funzione $f : A \rightarrow B$ assegna a ogni elemento di A esattamente un elemento di B .

- $A = \text{dominio}$
- $B = \text{codominio}$
- $f(A) = \{f(a) : a \in A\} = \text{immagine}$

Tipi di funzioni:

- **Iniettiva** (uno-a-uno): $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$
- **Suriettiva** (sopra): $\forall b \in B, \exists a \in A : f(a) = b$
- **Biettiva** (biunivoca): iniettiva e suriettiva

Nota

Una funzione biettiva ha **funzione inversa** $f^{-1} : B \rightarrow A$ tale che:

$$f^{-1}(f(a)) = a \quad \text{e} \quad f(f^{-1}(b)) = b$$

12.2.4 Grafi

Definizione 12.6 (Grafo). Un grafo $G = (V, E)$ consiste di:

- $V =$ insieme di **vertici** (o nodi)
- $E \subseteq V \times V =$ insieme di **archi**

Tipi di grafi:

- **Grafo non orientato:** $(u, v) \in E \Leftrightarrow (v, u) \in E$
- **Grafo orientato** (digrafo): gli archi hanno una direzione
- **Grafo pesato:** ogni arco ha un peso $w : E \rightarrow \mathbb{R}$

Terminologia:

- **Grado** di v : numero di archi incidenti

– Grafo orientato: *grado entrante* + *grado uscente*

- **Cammino:** sequenza di vertici $v_1, v_2, \dots, v_k$ con $(v_i, v_{i+1}) \in E$
- **Ciclo:** cammino con $v_1 = v_k$
- **Connesso:** esiste un cammino tra ogni coppia di vertici

Rappresentazioni:

1. **Matrice di adiacenza** ($n \times n$):

$$A[i][j] = \begin{cases} 1 & \text{se } (i, j) \in E \\ 0 & \text{altrimenti} \end{cases}$$

Spazio: $\Theta(n^2)$, accesso arco: $O(1)$

2. **Liste di adiacenza:** Array di n liste, $\text{Adj}[u]$ contiene i vicini di u

Spazio: $\Theta(n + m)$, iterare vicini: $O(\deg(u))$

12.2.5 Alberi

Definizione 12.7 (Albero). Un albero è un grafo **connesso** e **aciclico**.

Equivalentemente:

- Grafo connesso con n nodi e $n - 1$ archi
- Esiste un unico cammino tra ogni coppia di nodi

Albero radicato: albero con un nodo speciale detto **radice**.

- **Padre/Figlio:** u è padre di v se $(u, v) \in E$ e u è più vicino alla radice
- **Foglia:** nodo senza figli
- **Altezza:** lunghezza del cammino più lungo dalla radice a una foglia
- **Profondità** di v : distanza dalla radice a v

Definizione 12.8 (Albero binario). Albero radicato in cui ogni nodo ha al più 2 figli (sinistro e destro).

Albero binario completo: tutte le foglie allo stesso livello, ogni nodo interno ha 2 figli.

Un albero binario completo di altezza h ha $2^{h+1} - 1$ nodi.

Visite di alberi binari:

- **Preorder:** radice $\rightarrow$ sinistro $\rightarrow$ destro
- **Inorder:** sinistro $\rightarrow$ radice $\rightarrow$ destro
- **Postorder:** sinistro $\rightarrow$ destro $\rightarrow$ radice

12.3 Appendice C (Calcolo combinatorio e probabilità)

12.3.1 Principi di conteggio

1. Principio del prodotto:

Se un evento può avvenire in m modi e un altro in n modi, allora entrambi possono avvenire in $m \times n$ modi.

Esempio 12.3. Quante stringhe binarie di lunghezza 3?

Ogni posizione: 2 scelte $\Rightarrow$ totale: $2 \times 2 \times 2 = 8$

2. Principio della somma:

Se un evento può avvenire in m modi disgiunti o in n modi disgiunti, può avvenire in $m + n$ modi.

12.3.2 Permutazioni e combinazioni

Definizione 12.9 (Permutazione). Numero di modi per ordinare n oggetti distinti:

$$P(n) = n! = n \times (n-1) \times \cdots \times 2 \times 1$$

Definizione 12.10 (Permutazione con ripetizioni). Se tra n oggetti ci sono gruppi di $n_1, n_2, \dots, n_k$ oggetti identici:

$$\frac{n!}{n_1! \cdot n_2! \cdot \dots \cdot n_k!}$$

Esempio 12.4. Permutazioni della parola “MISSISSIPPI” (11 lettere: 4 I, 4 S, 2 P, 1 M):

$$\frac{11!}{4! \cdot 4! \cdot 2! \cdot 1!} = 34650$$

Definizione 12.11 (Disposizioni). Numero di modi per scegliere k oggetti da n tenendo conto dell'ordine:

$$D(n, k) = \frac{n!}{(n-k)!} = n \times (n-1) \times \cdots \times (n-k+1)$$

Definizione 12.12 (Combinazioni (coefficiente binomiale)). Numero di modi per scegliere k oggetti da n **senza** considerare l'ordine:

$$C(n, k) = \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Proprietà dei coefficienti binomiali:

$$\begin{aligned}\binom{n}{0} &= \binom{n}{n} = 1 \\ \binom{n}{k} &= \binom{n}{n-k} \quad (\text{simmetria}) \\ \binom{n}{k} &= \binom{n-1}{k-1} + \binom{n-1}{k} \quad (\text{ricorsione - Pascal}) \\ \sum_{k=0}^n \binom{n}{k} &= 2^n\end{aligned}$$

12.3.3 Probabilità

Definizione 12.13 (Spazio campionario). Insieme Ω di tutti i possibili risultati di un esperimento.

Definizione 12.14 (Evento). Sottinsieme $A \subseteq \Omega$ dello spazio campionario.

Definizione 12.15 (Probabilità). Funzione $P : \mathcal{P}(\Omega) \rightarrow [0, 1]$ tale che:

- $P(\Omega) = 1$
- $P(A \cup B) = P(A) + P(B)$ se $A \cap B = \emptyset$ (additività)

Caso equiprobabile:

$$P(A) = \frac{|A|}{|\Omega|} = \frac{\text{casi favorevoli}}{\text{casi possibili}}$$

Regole fondamentali:

- $P(\emptyset) = 0$
- $P(\bar{A}) = 1 - P(A)$ (complemento)
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ (unione)

12.3.4 Probabilità condizionata e indipendenza

Definizione 12.16 (Probabilità condizionata). La probabilità di A dato B (con $P(B) > 0$):

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

Teorema 12.1 (Regola del prodotto).

$$P(A \cap B) = P(A | B) \cdot P(B) = P(B | A) \cdot P(A)$$

Definizione 12.17 (Eventi indipendenti). A e B sono indipendenti se:

$$P(A \cap B) = P(A) \cdot P(B)$$

Equivalentemente: $P(A | B) = P(A)$

Teorema 12.2 (Teorema di Bayes).

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

12.3.5 Variabili casuali e valore atteso

Definizione 12.18 (Variabile casuale). Funzione $X : \Omega \rightarrow \mathbb{R}$ che assegna un valore numerico a ogni risultato.

Definizione 12.19 (Valore atteso). Media pesata dei valori:

$$E[X] = \sum_x x \cdot P(X = x)$$

Proprietà del valore atteso:

- $E[aX + b] = aE[X] + b$ (linearità)
- $E[X + Y] = E[X] + E[Y]$ (sempre vera)
- $E[XY] = E[X] \cdot E[Y]$ (solo se X e Y indipendenti)

Definizione 12.20 (Variabile indicatrice). Variabile casuale che vale 1 se un evento si verifica, 0 altrimenti.

$$I_A = \begin{cases} 1 & \text{se } A \text{ si verifica} \\ 0 & \text{altrimenti} \end{cases}$$

Proprietà fondamentale: $E[I_A] = P(A)$

Esempio 12.5 (Numero atteso di confronti in QuickSort). Usando variabili indicatrici si dimostra che il numero atteso di confronti è $\Theta(n \log n)$.

12.4 Appendice D (Matrici)

12.4.1 Definizioni base

Definizione 12.21 (Matrice). Una matrice A di dimensione $m \times n$ è una tabella di m righe e n colonne:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

Notazione: $A = (a_{ij})$ dove a_{ij} è l'elemento in posizione (i, j)

Tipi speciali:

- **Matrice quadrata:** $m = n$
- **Matrice identità** I_n : diagonale di 1, resto di 0

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- **Matrice diagonale:** solo la diagonale può essere non-zero
- **Matrice triangolare superiore:** $a_{ij} = 0$ per $i > j$
- **Matrice simmetrica:** $A = A^T$ (vale $a_{ij} = a_{ji}$)

12.4.2 Operazioni fondamentali

1. **Somma:** (solo se A e B hanno stesse dimensioni)

$$(A + B)_{ij} = a_{ij} + b_{ij}$$

Complessità: $\Theta(mn)$

2. **Prodotto per scalare:**

$$(cA)_{ij} = c \cdot a_{ij}$$

Complessità: $\Theta(mn)$

3. **Trasposizione:** scambia righe con colonne

$$(A^T)_{ij} = a_{ji}$$

Se A è $m \times n$, allora A^T è $n \times m$

Complessità: $\Theta(mn)$

4. **Prodotto:** A è $m \times n$, B è $n \times p$

$$(AB)_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

AB è $m \times p$

Complessità (algoritmo naive): $\Theta(mnp)$

Per matrici quadrate $n \times n$: $\Theta(n^3)$

12.4.3 Proprietà del prodotto

- **Associativa:** $(AB)C = A(BC)$

- **Distributiva:** $A(B + C) = AB + AC$
- **NON commutativa:** in generale $AB \neq BA$
- $(AB)^T = B^T A^T$ (inversione ordine)
- $IA = AI = A$ (elemento neutro)

12.4.4 Determinante e matrice inversa

Definizione 12.22 (Determinante). Per matrice 2×2 :

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

Per matrici $n \times n$: sviluppo di Laplace (ricorsivo)

Proprietà:

- $\det(AB) = \det(A) \cdot \det(B)$
- $\det(A^T) = \det(A)$
- $\det(I) = 1$
- $\det(cA) = c^n \det(A)$ per matrice $n \times n$

Definizione 12.23 (Matrice inversa). A è invertibile se esiste A^{-1} tale che:

$$AA^{-1} = A^{-1}A = I$$

Condizione necessaria e sufficiente: $\det(A) \neq 0$

Proprietà della matrice inversa:

- $(A^{-1})^{-1} = A$
- $(AB)^{-1} = B^{-1}A^{-1}$ (inversione ordine)
- $(A^T)^{-1} = (A^{-1})^T$

12.4.5 Applicazioni agli algoritmi

1. Grafi: matrice di adiacenza

$A^k[i][j]$ = numero di cammini di lunghezza k da i a j

2. Programmazione dinamica: Floyd-Warshall usa prodotto matriciale generalizzato

3. Complessità prodotto matriciale:

- Algoritmo standard: $\Theta(n^3)$
- Algoritmo di Strassen: $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$
- Algoritmi avanzati: $O(n^{2.37})$ (teorici, non pratici)

12.5 Domande d'esame (Appello 2)

Queste sono domande reali dell'esame.³

Definizione di algoritmo

D: Definizione sufficiente di algoritmo?

R: Procedura ben definita che produce output da input in tempo finito.

Problema computazionale

D: Cos'è un problema computazionale?

R: Relazione input-output desiderata. Il problema dice *cosa*, l'algoritmo dice *come*.

Efficienza

D: Cosa si intende per efficienza?

R: Come crescono le risorse al crescere di n (non il tempo in secondi, dipendente dall'hardware).

InsertionSort vs MergeSort

D: $\text{InsSort} = 10n^2$, $\text{MergeSort} = 32n \log_2 n$. Quando è più veloce InsSort?

R: Per n piccoli: $10n^2 < 32n \log_2 n$ solo inizialmente.

$100n^2$ vs 2^n

D: Minimo n tale che $100n^2 < 2^n$?

R: $n \geq 15$.

Invariante di ciclo

D: Cos'è un'invariante di ciclo?

R: Proprietà vera all'inizio di ogni iterazione. Si dimostra per inizializzazione, conservazione e conclusione.

Vale $\max(f, g) = \Theta(f + g)$?

R: Sì: $\max(f, g) \leq f + g \leq 2 \max(f, g)$, con $c_1 = 1$, $c_2 = 2$.

2^{n+1} e 2^{2n} vs $O(2^n)$

R: $2^{n+1} = O(2^n)$ (costante 2). $2^{2n} = 4^n \neq O(2^n)$ (richiederebbe $2^n \leq c$, impossibile).

³Se si studiano solo queste l'esame si passa solo se si è colpiti da un fulmine mentre si fa un salto mortale carpiato suonando il flauto

Costo costante in RAM

R: Quando la parola di memoria ha dimensione fissa (es. 64 bit).

Prestazioni su istanze note

R: Precomputare le soluzioni e restituirle direttamente quando quell'istanza viene ricevuta.

$$T(n) = T(n-1) + n^c$$

R: $T(n) = \Theta(n^{c+1})$. Dim.: $\sum_{k=1}^n k^c \approx \int_1^n x^c dx = \frac{n^{c+1}}{c+1}$.

$$T(n) = 2T(n/3) + 1$$

R: Master Theorem, caso 1: $T(n) = \Theta(n^{\log_3 2})$ ($\log_3 2 \approx 0.63$).

Divide et impera: classe di problemi

R: Problemi divisibili in 2+ sotto-istanze con combinazione a costo al più polinomiale.

Analisi probabilistica

R: Uso del calcolo della probabilità nell'analisi degli algoritmi (non solo caso medio).

permute-without-identity

R: No: con $n = 3$ produce solo 3 permutazioni invece delle 5 attese.

Hire-Assistant: probabilità

R: Esattamente 1 assunzione: $1/n$. Esattamente n assunzioni: $1/n!$.

Generatore casuale

R: Valore intero in un intervallo con probabilità uniforme.

Algoritmo randomizzato

R: Il comportamento dipende dall'input e da valori casuali.

Ricorrenza con frazioni

D: $T(n) \leq T(10n/11) + T(n/11) + O(n)$?

R: $T(n) = O(n \log n)$. Ogni livello costa $O(n)$, profondità $O(\log n)$.

Partition su array

D: Partition su $[13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11]$, pivot $A[11]$?

R: $[9, 5, 8, 7, 4, 2, 6, 11, 21, 13, 19, 12]$, pivot in pos. 7.

Quicksort con elementi uguali

R: $O(n^2)$: ogni Partition genera partizione $(n-1)$ vs 0.

Partition elementi uguali: fix

R: Restituisce indice massimo. Fix: flag booleano; se tutti uguali, restituire $\lfloor (p+r)/2 \rfloor$.

Quicksort non crescente

R: Invertire il confronto in Partition: da $\leq$ a $\geq$.

Rand-Select: caso peggiore

R: Il pivot è sempre il massimo: dimensione scende di 1 a ogni passo, $O(n^2)$.

IV statistica d'ordine

D: IV statistica di $[8, 4, 8, 3, 4, 2, 3, 2]$?

R: Ordinato: $[2, 2, 3, 3, 4, 4, 8, 8]$. IV elemento = 3.

Eliminare duplicati

D: Eliminare duplicati in $\Theta(n \log n)$?

R: Ordinare, poi scorrere linearmente rimuovendo consecutivi uguali.

Build-Max-Heap: indice decrescente

R: Le foglie sono già heap. MaxHeapify richiede figli sistemati: si parte dal basso.

Visita inorder con pila

R: Visita in ordine (inorder), $O(n)$. Pattern: push-sinistra, pop-visita-destra.

LCS senza tabella b

```
PRINT-LCS(c, X, Y, i, j)
if c[i,j] == 0 then return
if X[i] == Y[j]
  PRINT-LCS(c, X, Y, i-1, j-1)
  print X[i]
else if c[i-1,j] >= c[i,j-1]
  PRINT-LCS(c, X, Y, i-1, j)
else
  PRINT-LCS(c, X, Y, i, j-1)
```

Costo $O(m+n)$: ogni chiamata riduce i o j di 1.

Cammino minimo vs TSP

R: Cammino minimo: sorgente $\rightarrow$ destinazione, polinomiale (Dijkstra). TSP: circuito visitando ogni nodo una volta, NP-difficile.

Prima e ultima cifra (Java)

```
1  int last  = n % 10;
2  int first = n / (int) Math.pow(10, (int) (Math.log10(n)));
```

T extends Comparable

R: La ricerca binaria usa confronti: serve `compareTo()`, disponibile solo se T estende `Comparable`.

Ereditarietà e attributo omonimo

R: `super.getIdA()` per `A.idA`, `this.getIdA()` per `B.idA`. Le variabili private non sono ereditate.

Errore in makeCombination

R: `values = numbers` copia solo il riferimento locale. Fix: copia elemento per elemento.

Eccezione in try-with-resources

R: L'eccezione di `close()` è soppressa (JLS 14.20.3): diventa *suppressed exception* della prima.

Downcasting Java

D: `A value = new B(); B value1 = (A) value; value = null;?`

R: `value` $\rightarrow$ `null`; `value1` punta all'oggetto `B` ancora in memoria.

Stack con due code

Push i -esimo: enqueue in ausiliaria, svuota corrente nell'ausiliaria, scambia. Costo: $O(i)$. Totale n push: $O(n^2)$.

Pop: dequeue. Costo: $O(1)$.

Ricerca in matrice destrorsa

R: Ricerca binaria su ogni riga. Complessità: $O(n \log n)$.

```
1  public static <T extends Comparable<T>>
2  int[] search(T[][] M, T x) {
3      int n = M.length;
4      for (int i = 0; i < n; i++) {
5          int lo = 0, hi = n - 1;
6          while (lo <= hi) {
7              int mid = (lo + hi) / 2;
8              int cmp = M[i][mid].compareTo(x);
9              if (cmp == 0) return new int[]{i, mid};
10             else if (cmp < 0) lo = mid + 1;
11             else hi = mid - 1;
12         }
13     }
14     return null;
15 }
```